Semantic Version Control

Anonymous Author(s)

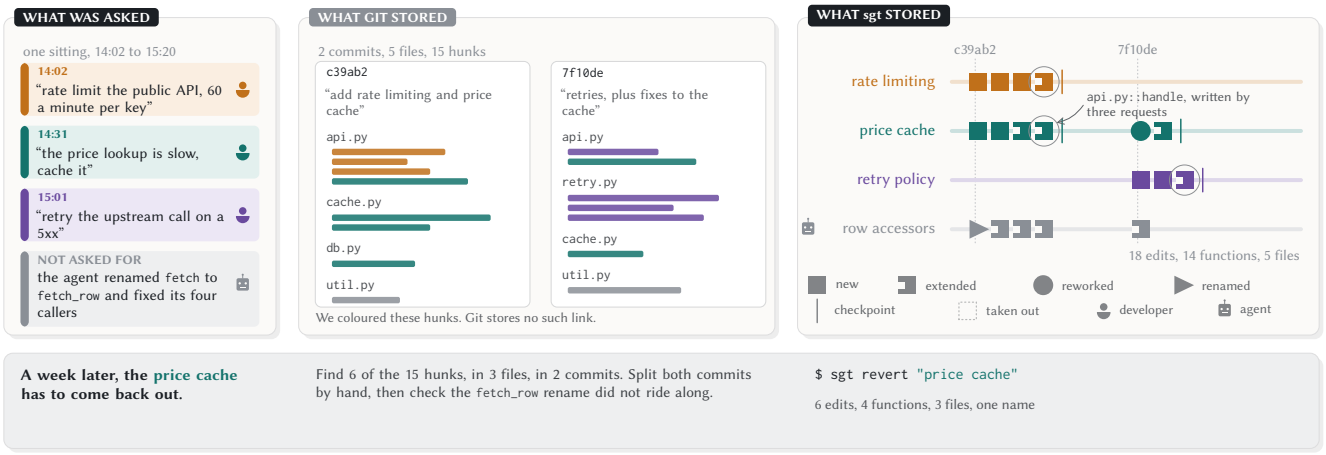

Figure 1: One hour with a coding agent, recorded two ways. The developer asked for three things and the agent did a fourth nobody asked for, and the two records differ in whether anything in the repository says which is which.

Abstract

A coding agent produces code a developer never typed; undoing one request later requires knowing which functions it produced—a grouping nobody recorded. sgt records one edit per changed function, groups edits into features by structural coupling, and rebuilds files from whichever subset the developer includes. Removing a feature is set difference; restoring it is set union.

Across 10,237 randomised operations the record lost no data, but half of all sessions produced a silent mismatch between what the tool reported and what the file contained. On 27 external repositories, reads ran everywhere; writes refused 70–79% of targets. One request becomes 4–6 features: the tool recovers sub-intent structure, not intent. In a within-subjects study, twelve developers used the system on unfamiliar agent-produced code; seeing the feature map moved their predictions of what an undo would affect toward ground truth, while the write operations remained too brittle to ship. The recommendation is temporal: build the read layer first and defer the write layer until the developer trusts the reports.

Keywords

version control, coding agents, program comprehension, developer tools, change history

1 Introduction

A version control system records changes so they can be revisited and reversed. Every widely adopted system records lines of code—whether as snapshots [141] or as patches to those lines [89, 114]—and groups them into commits a developer names. This agreement dates to the earliest systems and has survived every redesign since, for a reason that was sound from the start: the developer authored the code, held at least a rough model of what changed and why [77, 98], and could therefore supply the decomposition whenever a tool asked for it.

Developers now specify intent in natural language and models translate it into edits across files [37]. Three controlled field experiments show this shift produces 26% more code per week with no measurable quality decline [29]. A two-year longitudinal study of 800 developers found that the extra code comes with proportionally more deletion—developers produce more and discard more, while perceiving minimal change in anything but speed [123]. A field study of CLI agents in professional practice found a 24% lift in merged pull requests that persisted over four months [96]. The velocity gains are real. They are also paid for downstream: on a corpus of 3,000 repositories adopting cursor-style assistance, static analysis warnings rise 30% and cyclomatic complexity 41% within two months of adoption, while the velocity gain itself dissipates over the same period [58]. The speed arrives and leaves; the complexity it deposited stays. Meanwhile adoption is near universal—84% of professional developers report using AI tools—while trust in the accuracy of those tools fell from 40% to 29% in one year, and 46% now actively distrust what the tools produce, with experienced developers (ten or more years) the most sceptical [130]. Developers are producing code they do not trust, faster than they can account for it, and discarding it at a rate that suggests they know. The condition that made the agreement sound—that the developer holds a model of what changed and why—no longer holds reliably. Winograd and Flores called this kind of event a *breakdown*: an assumption so embedded in practice that it was invisible as an assumption, until the practice changed and the assumption became the thing that needs redesigning [155]. The assumption that the developer holds the decomposition was never stated as a design decision; it was the ground on which every other design decision rested. A developer who types three sentences in an hour and receives fourteen functions across five files cannot supply the decomposition a tool asks for, because they never held it. We argue that the design choice should be renegotiated.

Three consequences of this shift make the mismatch concrete. First, the unit that version control addresses, the line, is no longer stable enough to serve as an anchor. A model that generates a function in one response and regenerates it with different formatting or variable names in the next has changed every line while changing nothing a developer would name. Across 211 million changed lines from public repositories authored between 2020 and 2024, the share associated with refactoring—lines moved into a better position rather than written afresh—fell from a quarter to under a tenth [47]. In the same period, code revised within two weeks of being written roughly doubled. A line is now replaced faster than it settles into a position worth recording, and the developer wants to revisit a feature as it stood two requests ago—a unit the line-level record cannot express.

Second, the developer does not hold a detailed model of what the code says [117]. A natural language prompt specifies an outcome but not the logic that implements it, so the implementation contains decisions the model made and the developer has not reviewed. The evidence is consistent across methods: developers read generated code for shape rather than line by line [7], frequently cannot identify which code the model wrote [135], rank understanding generated code among their most common difficulties [82], and score 17% lower on assessments of code they produced with AI assistance than code they wrote by hand [125]. The consequence for version control is specific: a developer who searches their history for a prior state must describe it in terms they never acquired. Furnas et al. measured this vocabulary problem in information retrieval thirty years ago [45]; it now appears inside the repository itself. Nguyen and Glassman observed it directly in AI-assisted programming: developers and models systematically name the same behaviour differently, so the developer's search terms and the model's naming choices diverge even when both refer to the same function [100]. The developer searches with vocabulary they never learned and finds whatever happens to match instead of the version that served a stated purpose [28].

Third, the unit the developer thinks in has moved from a location in the code to a purpose the code serves. A prompt names what the software should do (rather than which file to edit), so the developer's mental model of their own work is organised by feature: the retry policy, the export endpoint, the rate limiter. This is the level the program comprehension literature has always found people work at: Biggerstaff et al. called recovering which functions implement a named concern the concept assignment problem [12]. The feature location field exists because answering it by reading code is expensive [35]. The commit is the closest unit a version control system offers, and it does not correspond to this level. Herzig and Zeller found changes serving several unrelated purposes throughout the histories of five open source Java projects [62]—what they called tangled commits, produced in 2013 by humans in a hurry. An agent session produces exactly that structure systematically, because a commit ends where the developer stopped typing a command and a request ends where the agent finished implementing a feature, and those two boundaries have no reason to align.

The response we describe is a version control system that records at the grain the developer thinks at. Instead of recording lines inside a commit, we parse each file into its functions and classes [15] and record one edit per changed function, together with what that edit was built on. We group those functions into features from how they change and how they refer to one another [143, 159, 165], and label each group with the words the developer and the agent used while the work was happening. A version of the code is then a set of these edits, and the files on disk are rebuilt from whichever set is included, so a feature can be taken out or put back by naming it rather than by naming the lines it produced.

None of the individual techniques is new. Change coupling mining, community detection, feature location, and set-based version control each have a literature. What is new is composing them into a single tool that a developer operates daily, because that composition introduces interactions none of the original designs anticipated:

- A function identity tracker that loses a rename breaks the dependency chain that reverts depend on.
- A grouping inference that absorbs a shared helper pulls unrelated work into a single feature.
- A layout record—the whitespace between functions—becomes individually addressable, and acquires failure modes the entity record does not have. More than three in four violations across ten thousand operations trace to a layout record, either targeted directly or dragged along by an entity operation whose own logic worked.

The contribution is making this composition survive daily use on a real codebase, measuring where it fails honestly, and reporting both the design's costs and the conditions under which it should not be adopted (Section 10.5).

For example, a service that answers price quotes for a trading dashboard has a handler, a database layer, and a set of row accessors. On Monday the developer prompts `cache the price lookup, it takes 400ms and the price barely moves`. The agent writes `cache.py::get_cached` and `cache.py::_ttl`, adds a parameter to `db.py::query` so the cache can pass a time-to-live through, and changes `api.py::handle` to consult the cache before the database—four edits to four functions in three files, all filed under the one request. Over the next two days two further prompts each change `api.py::handle`: one adds a rate limit, the other adds a retry policy. While adding the retry the agent also reworks two cache functions, so the price-cache feature now spans six edits. A colleague adds a metric for quote latency.

Three weeks later the developer finds the dashboard showing a stale price. They type `sgt revert "price cache"`. The tool prints the six edits that would come out, names `api.py::handle` as the one function where later work was built on something being removed, shows what each affected function would look like afterwards, and asks before touching anything. The rate limit and the retry policy stay, because their edits are not in the set coming out. The colleague's metric stays, because it was never in that set. The cache parameter leaves both `db.py::query` and `api.py::handle`, because both were in it. When the developer decides a week later that the cache was worth keeping after all, `sgt restore` returns the files byte for byte, because adding the six edits back into the set is the same arithmetic as removing them. Figure 1 draws these three requests arriving in a single sitting, under both records, and Section 5 follows this repository through five sittings.

Three prices are paid for this, all of them in the first hour of use rather than in some later edge case. Two edits to the same function made from the same starting point are recorded as competing versions that a person has to decide between, including the pair at opposite ends of a fifty line function that touch no common line and that git would have merged without comment. The grouping of edits into named features is inferred from coupling rather than from request boundaries, so it is systematically finer than what the developer asked for—one request typically becomes four to six features—and correcting that granularity costs the developer a merge, a rename, or a split, none of which moves a recorded edit. sgt reads Python, TypeScript, and TSX and versions every other file whole, at the grain git already uses, so a repository whose interesting logic is written in Go gets nothing from sgt today.

The paper makes four contributions. We characterise the mismatch between the unit a developer using an agent decides in and the unit a version control system records, one attempted operation at a time (Section 3), in the style of conceptual analysis Pérez De Rosso and Jackson applied to git itself [104, 105]. We describe sgt's design as four commitments, say what each one buys and what it costs, and run the whole set of operations on one repository (Sections 4 and 5). We state which observation would show each of our claims to be wrong (Section 6), and report what twelve developers found when they used the system on code they had never seen (Section 8). We report what fifty-seven days of sgt versioning its own construction taught us about where the design breaks (Section 9), and what three pilots taught us about the distance between a property that holds across 10,237 operations and a property a new user can rely on (Section 7). The evaluation found that the read operations—attribution, dependency display, consequence preview—survived first contact with every repository and every participant, and that the write operations did not. The paper therefore reports both the attempt and what it taught us about the order in which a tool of this kind should be built.

2 Related Work

2.1 Programming by request

The ways to program with foundation models have changed constantly over the last four years, along with the advance in what the models can do. At first a model was mostly a programming assistant offering inline code completion. Barke et al. watched 20 programmers and found them using it in two ways, finishing a line they had already planned, or asking for a starting point in code they did not yet know how to write [7]. Vaithilingam et al. found that 24 participants using Copilot did not complete tasks faster and still preferred to keep it on, because it gave them a draft to work from [146], and Sarkar et al. collected what the experience felt like from the developer's side [117]. In all of these the developer accepted a line or a block of code. Four years later SWE-bench gives a model a natural language issue report from a real repository and scores it on whether the repository's own tests pass afterwards [69], and the systems built to score well on it edit several files in a loop [158]. The unit a developer hands over has gone from a line to a paragraph of English, and the code that comes back is correspondingly larger.

All of this work studies the minutes around acceptance: whether the code is correct [106], whether the developer can tell that it is correct [147], whether they trust it [23, 82], and how much faster they finish [29, 94]. The trust question has since been answered at population scale: 84% of professional developers now report using AI tools while trust in the accuracy of those tools fell from 40% to 29% in one survey year, and experienced developers are the most sceptical [130]. Adoption and distrust are growing together, which means the code that enters the repository is code its own author does not fully trust. The natural organisational response—deploy a second model to review what the first model wrote—is structurally circular when no executable specification exists: Zietsman showed that the generating agent and the reviewing agent reason from the same artefact with correlated failure modes, so the review checks code against itself rather than against intent [164]. The circularity breaks only when an external reference independent of both agents enters the loop—a specification, a provenance record, or a developer who holds the decomposition the code was built from. Nguyen and Glassman studied what happens during acceptance itself and found a systematic vocabulary mismatch: beginning programmers and code models read each other's intent through different naming conventions, so the developer's description of what they asked for and the model's description of what it produced diverge even when the behaviour matches [100]. That mismatch compounds over time, because the developer who later searches their history must use the model's naming, not their own, and cannot tell when the two have drifted apart. The code that arrives also carries systematic defects: Tambon et al. taxonomised 333 bugs in LLM-generated code and found "misinterpretation"—the model produced something other than what the prompt asked for—to be among the most prevalent patterns [133]. A misinterpretation is code filed under the wrong request: it arrived during one prompt but implements something different, and the developer who later reverts that request will either miss the rogue code (if it was miscounted) or accidentally remove correct code that happened to be produced in the same response. Both failure modes are invisible without a record that tracks provenance at finer grain than the response that delivered it. Nanda et al. found misbehaviours in 30% of production agent trajectories across three categories—specification drift, reasoning problems, and tool call failures—and deployed Wink, a system that monitors trajectories and nudges the agent back on path [97]. What Wink corrects in real time, version control must recover after the fact, because the 70% of trajectories that completed without intervention still interleave edits across multiple requests in a single session, and a specification drift that Wink did not catch becomes a provenance error that only shows up when someone tries to undo one request without disturbing the others. Al Mujahid and Imran confirmed that developers know they are in this position: they found 81 code comments across public repositories in which a developer incorporates AI-generated code while explicitly stating they do not understand its behaviour or correctness—what the authors call GenAI-Induced Self-admitted Technical Debt [1]. The debt is not incurred by accident; it is incurred knowingly, because the alternative is to stop and rebuild the understanding the tool was hired to make unnecessary. None of this work asks what happens a week later, when the developer needs to recall what their codebase was, what has changed since, and why it is constructed the way it is. Parnin and Rugaber studied resumption after interruption in programming and found it takes a median of fifteen minutes

before a developer can resume productive work—even on code they were writing themselves minutes earlier [103]. The bottleneck is reconstructing the *mental simulation*: which functions are involved, what state they expect, and what the developer intended to do next. An agent session produces code the developer never held that simulation for, so resumption does not mean rebuilding a model that decayed; it means constructing a model that never existed, at a cost the fifteen-minute figure only lower-bounds. Xia et al. measured the steady-state version of the same cost by instrumenting 78 professional developers at work and found they spend roughly 58% of their time on program comprehension rather than on writing or modifying code [156]. That figure is the budget the whole shift is drawn against: if comprehension already consumes the majority of a developer's day, then code arriving faster than comprehension can keep up does not free time—it moves the work from writing into understanding, where it was already the larger half. Sillito et al. catalogued what that time is actually spent on, identifying 44 distinct questions developers ask while making a change and finding that the hardest of them concern relationships between pieces of code rather than the pieces themselves [128]. Their catalogue was compiled about code the developer's own team had written, and every question in it becomes harder when the author is a process that has exited. Bainbridge identified this structure in 1983: automation removes the need for a skill, the skill degrades through disuse, and when the automation fails the operator needs exactly the skill that has degraded [5]. The irony applies here directly. A developer who specifies in natural language no longer practises the decomposition that version control demands, so the ability to say which function served which request—the skill git's revert requires—decays in proportion to the success of the tool that made the decomposition unnecessary. Bastani et al. confirmed the mechanism experimentally: participants who used a generative model during practice solved faster in the moment but performed significantly worse on later assessments without access to it [9]. Risko and Gilbert's framework for *cognitive offloading* supplies the missing framing: people strategically delegate cognitive work to external resources (notes, tools, other agents) when the cost of internal processing exceeds its benefit, and the delegation is rational—but its side effect is that the offloaded information is never encoded in memory at all [112]. Bainbridge's irony assumes a skill that existed and degraded; cognitive offloading describes the stronger case where the skill was never exercised in the first place. A developer who tells an agent "add caching" has offloaded the typing and the decomposition together: which functions to create, which to modify, what state they share. That decomposition never entered the developer's working memory because the offloading succeeded, and it cannot decay from a state it never reached. The developer is not *forgetting* what the agent produced; they never knew it at the grain version control will later demand. A version control system built for this developer must therefore compensate for absent encoding, not for degraded recall—which is why the read layer (Section 4) provides attributions the developer can recognise without having to reconstruct from scratch.

Rozenblit and Keil named the metacognitive mechanism that makes this degradation invisible: people systematically overestimate the depth of their own understanding of causal systems—what they call the *illusion of explanatory depth*—and discover the gap only when asked to produce a step-by-step explanation [115]. A developer who stated "add caching" and received working code believes they understand the caching because they specified its purpose, but their understanding lives at the purpose level, not at the mechanism level (which functions call which, what state they share, what breaks when one is removed). The illusion persists until something forces a confrontation with the mechanism—a revert that breaks an unexpected caller, a rename that orphans a test—at which point the developer discovers that what they knew was the intent, not the implementation. The read layer is the forcing function: an attribution that surprises the developer is the step-by-step explanation Rozenblit and Keil showed dispels the illusion, delivered before the developer acts on their false confidence rather than after. Sergeyuk et al. confirmed the consequence at scale: a two-year longitudinal study of 800 developers showed that sustained AI assistant use produces substantially more code *and* deletes substantially more, while developers themselves perceive minimal change in anything but productivity [123]. Murphy-Hill et al. distinguished the form factor: CLI-based coding agents produce a sustained 24% lift in merged pull requests over four months, whereas He et al. found that cursor-style inline completion produces a velocity gain that dissipates within two months as cyclomatic complexity rises 41% [58, 96]. The difference is structural. An inline completion changes one expression inside a function the developer is already reading; a CLI agent session changes fourteen functions across five files while the developer watches a progress spinner, and what accumulates is not lines of code but interleaved edits whose provenance the developer never held. Vella and Blincoe tracked 95 matched developers over six months and identified the paradox this creates: productivity perceptions held stable at 84% while the share reporting worsened developer experience nearly doubled from 14% to 27%, with flow state and cognitive load both eroding over the period—a divergence they traced to a new category of work they called *supervisory engineering* (direction, evaluation, and correction of AI output) [149]. He et al. documented the consequence at organisational scale: when an enterprise mandated doubling per-engineer throughput via AI tooling, per-reviewer load roughly doubled and automated review overtook human review, while the throughput target was met [57]. The production side of the bottleneck widened; the verification side did not, and what absorbs the difference is review depth. Cynthia et al. confirmed the downstream consequence: across 54,791 agent-generated review comments on 342 repositories, the only reliable predictor of a comment being resolved was the presence of a concise inline code suggestion; lengthy or complex feedback went unacted-upon regardless of its correctness [30]. The churn is invisible to the developer who produces it, but it is exactly what version control must later untangle—more code written per unit time means more interleaved edits per commit, and more deletion means more opportunities for a removal to carry unrelated work with it. Wang et al. argued for the reorientation these findings demand: from autonomous coding agents evaluated on benchmark completion to human-centred coding agents evaluated on four interaction dimensions—task alignment, verifiability, steerability, and adaptability [154]. Those four dimensions are not properties of the agent itself but of the interface between agent and developer, and all four degrade when a developer cannot see which request produced which code. Verifiability requires a record the developer can check;

steerability requires units the developer can redirect or remove; adaptability requires knowing what happened before, not just what exists now. All three are version control questions answered in terms of commits that predate the interaction pattern they describe. Casserini et al. named the accumulation that results: *agentic entropy*, a systemic drift between agent actions and architectural intent that compounds over time and is invisible to traditional diff-based review because a diff captures what changed in one commit, not how the aggregate trajectory diverges from what was asked for [20]. Their proposed remedy is process-level telemetry—conformity seeding, reasoning monitoring, a causal graph of agent decisions—and the unit of observation is the agent's internal reasoning trace. sgt observes a different layer: not what the agent was thinking but what the developer asked for and what ended up in the repository, which is the layer a revert operates on and the one a developer can verify without access to the agent's internals. Treude analysed terms of service for widely-used AI coding tools and found a consistent pattern: providers shift responsibility for correctness, safety, and legal compliance onto the developer, while providing no mechanism for the developer to exercise that responsibility at the granularity the agent works [144]. The developer is accountable for code they did not write line by line, produced by a process they did not supervise step by step, and no infrastructure exists to help them trace which request produced which function when something breaks. The governance gap Treude identifies is what makes provenance tracking a requirement rather than a convenience: a developer who bears responsibility needs, at minimum, a record that says what they asked for and what they got. Lyu et al. interviewed 20 practitioners building SE agents and surveyed 80 more, and found that as implementation becomes cheaper, bottlenecks shift rather than disappear: what emerges is not a reduction in total difficulty but a migration toward evaluation, coordination, and what they call *comprehension debt*—code outpacing understanding [86]. Comprehension debt is the practitioner's name for the same phenomenon Bainbridge identified theoretically: the skill of holding the codebase's structure degrades through disuse, and the degraded skill is exactly what a developer needs when the automation fails. We observe programmers gradually losing the theory of their own codebase [98] over the time they use these tools, and Bainbridge's framework says the loss is structural rather than incidental: it is what automation does to the skill it replaces.

Three recent empirical studies quantify how long this debt persists. Liu et al. mined 302,600 verified AI-authored commits across 6,299 repositories and found that 22.7% of issues the AI introduced—code smells, correctness defects, security vulnerabilities—still survive at the latest version [85]. Nearly one in four problems becomes permanent. Rahman and Shihab tracked 200,000 code units across 201 projects and found the opposite of the disposable-code hypothesis: agent-authored code survives *longer* than human code (15.8 percentage-point lower modification rate), while carrying a modestly elevated corrective-change rate (26.3% vs. 23.0% for humans) [109]. Code that persists longer while needing more bug fixes is code whose provenance matters more over time, not less—each surviving function is a future undo that requires tracing back to the session that introduced it. Sawada et al. confirmed the maintenance asymmetry from the other direction: AI-generated files receive less frequent maintenance than human-authored code, and when maintenance occurs, human developers perform the large majority of it [119]. The pattern across all three is consistent: AI-generated code enters the repository, persists, accumulates corrections, and is maintained by people who did not write it—which is the exact scenario in which provenance attribution becomes load-bearing.

Storey proposed a framework that names these accumulations precisely [131]. Traditional technical debt resides in code; *cognitive debt* resides in people (the erosion of shared understanding across a team); *intent debt* resides in externalized knowledge (the absence of recorded rationale that developers and agents need to work safely with code). AI-generated code can reduce technical debt—a model that writes clean implementations produces fewer shortcuts—while simultaneously increasing the other two, because neither the team's understanding nor the system's externalized rationale keeps pace with the code the model produces. Catalan et al. confirmed the mechanism empirically: cognitive engagement with agent-generated output declines as tasks progress, and current agentic assistants provide limited affordances for reflection, verification, or meaning-making [21]. The developer stops reading before the session ends, and what they do not read becomes intent debt the moment the session closes—code whose purpose no one holds. sgt's read layer is an affordance for exactly the reflection Catalan et al. found missing: it externalizes the rationale (which request produced which function) so that the intent debt does not accrue silently, and it presents the result as a scannable attribution rather than a wall of code, so that a developer whose engagement is declining can still absorb the provenance without reading every line. Dhanorkar et al. interviewed 17 experienced developers and identified four forms of emergent oversight work: a priori control, co-planning, real-time monitoring, and post hoc review [32]. Their central finding is that oversight spans all four stages—developers invest preventative effort before agent execution and proactive effort during it, as well as reactive checking after—and that the tools available for each form are inadequate. "Difficulty reviewing agent-generated code" was a situated challenge participants named explicitly, and the heuristic they adopted (using test results as a proxy for correctness) is the same substitution that produces undetected errors in the study of Section 8: a developer who relies on tests alone misses the structural damage a passing suite cannot detect. sgt's read layer is infrastructure for the post hoc review form—it shows what was produced and by which request—and the consequence preview supports the co-planning form, because a developer who can see what a revert would touch before confirming it is planning jointly with the tool rather than delegating blindly.

2.2 What a version control system agrees to record

Git records a version as a snapshot of the whole working tree, and each commit names its parents plus a tree of file contents addressed by hash [141]. Verifying that a history has not been altered is then a matter of rehashing it, and a merge is computed on demand from a common ancestor, so nothing in the repository has to say what happened when two edits met. We keep the first of those properties and give up the second on purpose. sgt addresses its records by their contents, and when two edits to one function descend from

the same parent it records that they did rather than computing what the pair amounts to, for the reason Section 4.3 gives.

Pérez De Rosso and Jackson analysed git's concepts against the purposes users bring to them, found several places where a concept serves a purpose users do not have, and removed the staging area and detached `HEAD` in their redesign [104, 105]. The commit survives that redesign untouched, and it should, because it serves a purpose users do have. It is also the smallest thing git can address, so any operation over less than a commit works by having the developer point at lines, and `git add -p` and `git rebase -i` are the tools for pointing. Under git's original constraints this is the cheapest correct design available, because the person running `git add -p` in 2005 had typed the lines minutes earlier, and writing their decomposition into the repository would have meant maintaining a second structure that could disagree with the first.

The commit does carry one field meant to hold intent, and it is worth saying why we do not build on it. Tian et al. examined what makes a commit message good and found that most messages in practice document neither what changed nor why, supplying at best one of the two [140]. The message is optional prose written after the fact by someone under time pressure, so it degrades exactly when the history gets long enough to need it. A tool that reads commit messages as its provenance record inherits that variance, which is why sgt treats the request that preceded the code as the anchor and uses the message only to name what the coupling signal has already grouped. This also sets the ceiling on what sgt recovers from a colleague who never ran it (Section 9.7): their grain comes back from parsing, and their vocabulary comes back only as far as their commit messages carry it.

Darcs takes the other option and records a version as a patch that adds and removes lines in a stated context [114]. Patches that do not depend on each other commute, so a user can pull the third patch in a series without the first two, and the design has been formalised and reimplemented since on a data structure that makes the operations fast [67, 89, 91]. A Darcs user who pulls one patch onto another branch moves the patch itself, along with the dependencies it names. Git's cherry-pick, by contrast, reapplies a diff in a context it was not written against and produces a second commit with a second hash—so the same work acquires two identities. But Darcs works out what a patch depends on from the lines the patch touches, so a dependency that leaves no trace in the text goes unrecorded. When an agent adds a `Retry` class in a new file and edits `api.py::handle` to call it, the two patches share no line, and nothing in the record says that pulling the second without the first yields a file referring to a name that does not exist. Darcs is explicit about this and offers a way to declare such a dependency by hand, and we would have made the same call in 2003, because computing a dependency between two functions requires parsing the language and a system that parses works for some languages and not for others. Darcs declined that price and we pay it.

Jujutsu and Mercurial both add a record of history's own history. Jujutsu gives each change an identifier separate from its commit hash, so rewriting a commit produces a new commit while the change keeps its name, and a change with more than one visible commit is called divergent [150, 151]. Mercurial stores obsolescence markers apart from changeset data, keeping a history of how the history was edited alongside the history of how the files were [138]. Both designs let a developer say "the change I was working on yesterday" after five rebases and be understood, and both let two people rewrite the same region of history and have the disagreement recorded rather than silently resolved, which is the construction sgt uses for two competing versions of one function. We part company on what receives the durable identity. In both systems it is a commit, so the identity is exactly as coarse as the commit, and after a developer squashes a two-hour agent session into one commit, one change identifier covers all three requests and nothing smaller has a name. Figure 2 puts these records beside each other.

The granularity mismatch is not specific to code. Deng et al. analysed 170 forum threads about version control in computer-aided design and identified the same structural problem in a different material: CAD tools version entire assemblies while designers think in components, so the questions a designer asks ("what changed about the bracket?") have no answer in the record [31]. Their four challenge categories—management, continuity, scope, and distribution—map to specific failures in this paper: scope is the tangled commit of Section 3, continuity is the checkpoint story of Section 5, and management is the naming problem of Section 4.4. The convergence across domains suggests that the mismatch between a tool's recording grain and a practitioner's thinking grain is a general property of version control design, not a side effect of any particular language or medium, and that any tool which closes the gap must parse the artefact's internal structure rather than treating it as opaque content.

2.3 Version control that already knew about functions

Recording a version at the grain of a function has been built before, more than once, and the reason none of it displaced the commit is the same reason we argue it should now. Historage rewrites a Java repository so that every method sits in a file of its own, at which point git's existing machinery gives each method a history of its own [56], and FinerGit refines that format so that git's rename detection keeps working when a method moves [63]. Stellation and Coven made the fine-grained unit the thing the repository stores rather than a transformation applied to it, so a developer could check out a single method [24, 25]. MolhadoRef settled the problem Section 9.3 reports as our weakest link, by having the development environment record each refactoring as the developer performed it, so a rename entered the history as an operation instead of arriving as something a later tool had to guess at [34]. Mylyn recorded, for each task a developer worked on, the set of program elements they touched while working on it, and used that set to decide what the environment showed them [72]. All of this existed at least fifteen years before the first coding agent and all of it worked. Historage and FinerGit are in use today as instruments for mining software histories, which is research reading a repository from the outside, and a working programmer choosing what to run on Monday morning uses none of it.

We do not read that as a failure of engineering, because a finer grain buys a developer only what they cannot already supply for themselves, and until recently they could supply all of it. Whoever wanted the caching out of a 2010 repository had written the

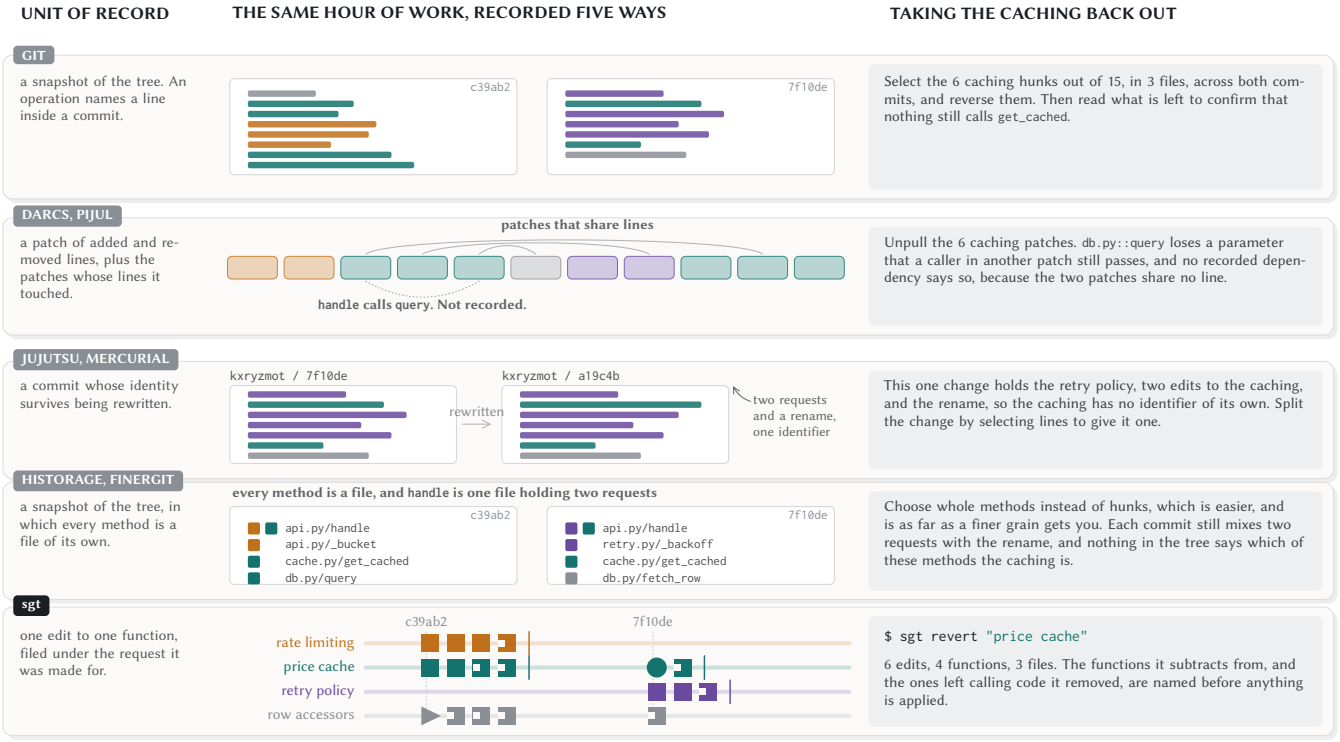

Figure 2: One hour of work, recorded five ways, with the same request taken back out of each. The fourth row already records at the grain of a method and still cannot say which of those methods the caching is, which is why we read what is missing as the name of the request and not the size of the unit.

caching, so a system that told them which methods it touched was performing work they had already done, at the price of a repository their colleagues could not clone with plain git. The value of a finer grain has not changed; what changed is the identity of the author. A record that was redundant while the developer wrote the lines is now the only surviving copy of the decomposition, because the developer has stopped writing them. Mylyn's task context and sgt's named group of edits are the same object built for two different situations, and the difference is that Mylyn watched a developer's attention move through code that developer was writing, whereas the attention sgt reconstructs belonged to a process that has already exited.

The merge question in this literature was approached from the end opposite to ours. Horwitz, Prins and Reps showed that two versions of a program can be integrated automatically when a semantic analysis proves the changes do not interfere [64], and Apel et al.'s semistructured merge parses a file far enough to see that two edits landed in different methods, resolving a large share of what a line-based merge reports as a conflict [2]. The design here rests directly on two further results from that area, that a merge of the recorded operations tells a developer more than a merge of the states those operations produced [83], and that a version is usefully treated as a set with a logic over it, which is the construction Section 4.2 uses [163]. sgt departs from Horwitz and from Apel deliberately. Where semistructured merge combines two edits to one method as soon as it can see they touch different lines, sgt stops and asks a person, and Section 9.8 concedes that a repository with one person in it is the wrong place to have tested that rule.

The strongest case against stopping comes from a literature built on the opposite commitment. Conflict-free replicated data types guarantee that concurrent edits converge to one state without asking anybody, by constraining the operations so that any order of application yields the same result [124], and the approach has been carried into collaborative text and rich-text editing [84]. A CRDT would eliminate the fork rule's entire cost: no queue, no interruption, no 18-fold amplification. We decline it because convergence and correctness are different guarantees, and Kleppmann et al. demonstrated the gap directly: text CRDTs that satisfy convergence still produce *interleaving anomalies*, where two users' concurrent insertions are merged into a sequence neither of them wrote and that means something neither intended [76]. Every replica agrees on the result, and the result is wrong. For prose an interleaved sentence is embarrassing; for code an interleaved function body is a program that compiles and does something nobody asked for, filed under a provenance record that says both requests succeeded. That failure is the silent success of Section 10 promoted to a guarantee: the system's own consistency model certifies the state as correct. A developer who did not write either version—the developer this paper is about—has no way to detect it. So the fork rule buys the one property convergence cannot supply, which is that every function

in the repository was chosen by a person, and Section 4.3 states the price.

2.4 Recovering a decomposition after the fact

Commits that address more than one concern distort the tools built from a history. Herzig and Zeller examined five open source Java projects, found such commits throughout them, and showed that separating those changes alters which files a defect prediction model identifies as the most likely to contain bugs [61, 62]. The untangling tools built after that finding recover the grouping from the diff itself, partitioning it by the definition and use relationships between the changed statements [8] or by a dependency graph across the versions a commit spans [108], and every one of them is evaluated on whether it reproduces a grouping a human labelled. Two anticipate the move we make: Dias et al. read the fine-grained edit events an IDE records while the developer is still typing [33], and Shen et al. hand the partition back to the developer to correct, which is the arrangement Section 4.4 adopts [126, 136, 152]. What none of them could assume is a statement of the boundary written down before the code existed, because in 2015 no such statement existed to be had, and the strength of their inference is a measure of how much they had to recover without one.

That is the loosened constraint, and it is worth being exact about what it buys, because sgt performs the same inference they do and performs it no better; we would expect it to be worse, since it has to finish while a developer waits. What has changed is what the inference is for. Their partition has to be right, because it is the answer. Ours has to be close enough that a name attaches to the right set, and the developer who reads a wrong name fixes it with a rename that moves no code.

sgt does depend on two results from this literature. Keeping a function's identity when it is renamed or moved is the problem that refactoring detection and tree differencing address [40, 44, 145], and it is the weakest link in our implementation—though not for the reason one would expect. A rename that goes undetected appears as one function removed and another added, but a rename that is detected still breaks the output path: the system records that the identity persisted but never records a path remap, so the old path becomes a zombie that composes nothing, and the new path receives content keyed to a location that no longer exists on disk. Grouping edits into features is a clustering over which functions change together and which call each other, both halves of which the co-change mining literature established [49, 75, 159, 165], resolved with a community detection algorithm [142, 143]. Both are inferences, and Section 4 states what a developer does when they are wrong.

2.5 Histories the editor keeps

A second tradition puts the record in the editor and asks nothing of the developer. Azurite stores a programmer's editing operations at a finer grain than any commit and lets them undo one past edit while keeping the edits made after it [160–162]. Variolite lets a data scientist wrap a region of code as a variant and switch between variants in place, after Kery et al. found that people doing exploratory work avoid version control for exactly that work [73], and the same tradition has held alternative versions side by side, replayed a document's editing history, versioned one experiment at a time, and cleaned a notebook up afterwards, in every case recording what the developer did as a byproduct of their doing it and never asking them to prepare it [52, 55, 59, 74, 90, 137]. That commitment sgt keeps. All of them also stop at the boundary of one machine: the history lives in the editor's own store, so a colleague, a build, and a code review never see it, and the unit is a single edit event, and a set of related edits still has no name a person can use. sgt crosses that boundary by writing its record into the git repository as files committed with the code, and Section 9.7 describes what that costs.

2.6 Layers over git built for the way people work now

The tools developers reach for today accept git's storage and add a layer above it. GitButler keeps several branches applied to one working directory at once, draws each as a vertical lane with its own staging area, and lets the developer drag a change into the lane it belongs to [46], on the correct reasoning that which branch a piece of work belongs to is easiest to decide once the work exists. GitButler assigns an individual file change to a lane, so it cannot express the case at the centre of Figure 1, where three requests all edited `api.py::handle` and no assignment of `api.py` to one lane is right. Graphite and GitHub's stacked pull requests take the ordered series as their unit, keeping a chain of changes consistent by rebasing and retargeting the higher layers when a lower one merges and landing every unmerged layer beneath the topmost ready one in a single operation [48, 50], which is the right answer to a reviewer who cannot read one large change with the care they can give to each of five small ones. Graphite's guide states the precondition plainly, that developers must understand how to break down their work into small, incremental changes, and every command the tool offers operates on a decomposition the developer has already performed. In an agent session that decomposition exists in the developer's request history and nowhere in the code. The precondition is not merely harder to meet; with CLI agents it is structurally unavailable, because the agent's changes span multiple files in a single response and the developer sees the result only after the decomposition would need to have been decided.

The closest system to ours is sem, which parses about 32 languages with tree-sitter and reports which functions, methods and classes a commit changed, resolving an entity's identity across versions in three passes, by exact identifier, by a structural hash that ignores whitespace and comments, and by token overlap above 80% [3]. sem keeps its answers in a derived SQLite cache outside the repository, and that placement makes sem safe to adopt and safe to abandon, gives entity-level answers about a history recorded years before sem existed, and requires agreement from nobody else on the team. It also bounds what sem can offer: a record any teammate's ordinary commit can silently invalidate cannot be the record an operation acts on, so every sem command reports and none of them changes state, and a developer who learns from `sem diff` that `get_cached` changed removes it with a line-level tool. sgt commits its record to the repository so that operations can act on it, and we pay for that in two places. A record that operations depend on has to be repaired when it is wrong, which Section 9.4

reports the current cost of, and everybody who pulls the repository now carries a record they did not ask for, which Section 9.7 works through from the point of view of the three people who never installed sgt. sem chose the safer placement and we chose the more useful one, and the honest form of that sentence is that we have taken on an obligation sem does not have. Kula and Treude proposed the shift at the level of the development environment itself: a graph-based IDE that abandons the static file as the unit of work and lets a developer prune and reshape AI-generated code the way a gardener shapes a tree [78]. Their diagnosis is the same as ours—"current static-file IDEs lack the mechanisms to track the provenance of AI-generated code"—and the response is complementary: they restructure the editing surface, while we restructure the versioning substrate beneath it. Velasco et al. address provenance at the model level: tracing generated code back through the prompt, the training data, and the model's internal representations [148]. That question—*why did the model write this line?*—is orthogonal to the one sgt answers—*which developer request caused this function to exist?*—and a developer may need both. The first explains a line's origin in the model's training distribution; the second explains its presence in the project's history. sgt provides the second without requiring access to model internals, using only the record of what was asked and what changed.

The literature above converges on a position no single paper states. The cognitive precondition for version control—that the developer holds a model of what changed and why—is eroding (Bainbridge, Shen and Tamkin, Risko and Gilbert). The debt that results has been named three times under three framings (comprehension debt, intent debt, agentic entropy). Tools that address it either stop at display (sem, Azurite, Mylyn) or stop at the editor boundary (Variolite, Kery et al.), or address a different grain (GitButler at the file, Darcs at the line, Jujutsu at the commit). The combination that does not yet exist is a tool that records at the grain of the function, groups those records by the request that produced them, writes the result into a repository colleagues can pull, and exposes operations (revert, restore, switch) over those groups—so that a developer who typed three sentences and received fourteen functions can act on one sentence without manually reconstructing which functions it covers. That combination is what Section 4 describes, what Section 5 demonstrates, and what the remainder of the paper evaluates. Section 10.4 returns to the five systems above once the evaluation data exists, and compares them on what each one's failures cost rather than on what each one promises.

3 Five Operations a Developer Cannot Name

The five scenarios below follow one developer through a single afternoon and the week after it. We use them to name specific operations—things a developer wants to do to a repository—that require knowing which functions served which request and that nothing in the existing record preserves.

The running example is a Python service that answers price quotes. In one sitting the developer asked a coding agent for three things: rate limiting, a price cache, and a retry policy. The agent implemented all three by editing the same two functions—`api.py::handle` (we write `file::function` throughout) and `db.py::query`—and renamed a helper while it worked. By 15:00 the developer had three features interleaved inside two functions and no record of which lines served which purpose (Figure 1).

What the developer sees on screen at 15:00 is not the problem. The file is correct, the tests pass, and three features work. What they see a week later is the problem. They open `api.py::handle` and find sixty lines: a bucket check at the top (rate limiting), a cache lookup before the database call (price cache), a retry loop around the database call (retry policy), and the cache write after it. They did not write these lines, and the agent that did write them no longer exists. They typed three sentences and they are looking at sixty lines, and the question each scenario below asks is what happens when they need to act on one of those sentences—take it out, move it, show it to a reviewer—and the only record that connects a sentence to the lines it produced is the one inside the developer's memory, which has been decaying since 15:00.

Every scenario below has a workaround in git. The workaround always requires the same labour: reading the code to figure out which functions belong to which request (Figure 2). That labour was once free, because the developer who typed the lines already knew. It is no longer free, because the developer typed three sentences and an agent produced the code.

Naur argued that programming builds a *theory*—a specific understanding of how each part of the solution maps onto the problem—and that this theory is what makes modification possible [98]. An agent produces the code without building the theory in the developer. The five scenarios are five faces of one cost: recovering a decomposition that nobody recorded.

3.1 "Commit this when it is done, and I do not know yet when that is"

A developer working through an agent still has to choose when to commit, and in this hour every moment available to them is wrong in a different way:

- Commit after each request: writes a version of `api.py::handle` that existed for twenty-nine minutes and that nobody tested. The retry commit also contains cache changes, because the agent reworked the cache while adding the retry—so even one-request-per-commit mixes two purposes.
- Commit once at the end: one commit holding all three requests and the rename.
- Split the difference (what this developer did): two commits, each mixing two requests with the rename (Figure 1).
- Stage hunk by hunk: asks the developer to decide, for each of fifteen hunks, which of four purposes it serves.

Green and Petre named these costs. The first three are *premature commitment*: a notation that forces a decision before the information it needs exists [13, 51]. The fourth is *viscosity*: the resistance a notation puts up when a decision turns out to be wrong. In a session driven by requests, however, the boundary itself is not missing—the developer stated it before any code existed. What is missing is the list of functions on either side of it, which only the agent ever held.

The operation. Record the boundary the developer already stated, without asking them to state it a second time in a different notation.

3.2 "Change the rate limiter I built yesterday"

The rate limiting from 14:02 used a fixed one-minute window, and three days later it needs a sliding one. The work lands in two of the four functions the original request produced, and the developer's first question is what the rate limiting looked like before they touched it.

Git can follow one function at a time: `git log -L :handle:api.py` matches a function by name pattern; `git log -S` finds commits where a string appeared or disappeared. The developer runs the first command four times—once per function they believe was involved—and assembles the answers manually. Because the set of functions belonging to "rate limiting" has no name in git, nothing checks whether four is the right count or whether a fifth function was missed. The developer's confidence in their answer is invisible to the tool, and so is their error.

The question being answered—"why is this code this way?"—is among the hardest developers face [79], and understanding prior rationale is the most common reason developers examine history at all [28]. Here the rationale was one sentence of English, typed three days ago into a chat window that is now closed.

The operation. Name the work one request produced, see its current state and its previous state, and change it as a unit.

3.3 "Take the caching back out and leave the rest alone"

The caching turned out to serve a stale price for up to sixty seconds, and it has to go. Its six edits are spread over three files and two commits, mixed with nine hunks belonging to three other purposes:

`db.py::query`	added a `ttl` parameter
`api.py::handle`	passes that parameter at the call site
`cache.py::get_cached`	the helper itself (three call sites)

`git revert` takes back a whole commit, so reverting either of these two removes work the developer is keeping. The developer therefore checks out both commits, selects the six hunks by hand, reverses them, and confirms the result compiles. The confirmation is what takes the time: nothing in the record says that the `db.py::query` signature change and the `handle` call-site change were made for the same reason, so the developer must re-derive that relationship by reading the code—the same decomposition recovery that costs nothing when the person paying it wrote the code.

The operation. Remove a named set of edits, leave every other edit to the same files in place, and name the code that still depends on the removed set before anything is applied.

3.4 "Send just the export work to the release branch"

Two days later the same developer adds a fourth request on the same branch: an export endpoint that serialises the quotes for a downstream consumer. The branch now carries the export work and a half-finished retry policy that shares a helper with it. The export has to ship today; the retry does not. The developer needs to move just the export work to the release branch. Git's mechanism for moving individual commits (`git cherry-pick`) operates on whole commits, and the export work shares a commit with part of the retry policy, so extracting it means splitting the commit manually—a step that must be repeated every time new export work arrives. Darcs solves the part of this problem that is about lines: a Darcs patch carries its line-level dependencies, so moving it requires no manual reapplication (Section 2). The export code, however, calls a helper that the rate limiter introduced—a function-level dependency that no line-level record captures. The developer discovers this only when the release branch fails to build, because the failure is a missing definition rather than a merge conflict, and nothing in the cherry-pick warned them.

The operation. Carry a named set of edits and its dependencies to another branch, and be told before the operation which dependencies would come along.

3.5 "Let someone review the rate limiter while I keep building on it"

Back on the original afternoon: the rate limiter was finished at 14:30 and the caching that followed calls one of its functions (`api.py::_bucket`), so the two form a stack: the rate limiter underneath, the cache on top. Submitting the rate limiter for review while continuing to build the cache requires the two to live on separate branches—a decision the developer would have had to make at 14:30, before they knew the cache would depend on the limiter. Stacked pull requests address this: review works best on small changes arriving often [111], and a reviewer's main difficulty is understanding the change in front of them [4]. Graphite and GitHub automate the mechanics—rebasing the upper layer when the lower one changes (Section 2)—but the decision of what constitutes a layer must still be made before the work exists. In an agent session the layering becomes visible only after both layers are written. Green and Petre call this a hidden dependency: a relationship the system relies on that the notation does not show [51]. Church et al. traced the same effect through version control [26]: a branch says nothing about which functions call which.

The operation. Discover the layering after the work exists, and submit one layer while continuing to work on the layer above it.

3.6 What the five have in common

Every one of these operations takes a set of edits as its argument, and the developer identifies that set by the purpose the edits served. Git expresses a set of lines, a set of files, and a set of commits, and a developer can build all five operations out of those three primitives. The price is recovering the decomposition that nobody recorded.

That recovery was affordable when the person paying it wrote the code—they held the answer already, having typed the lines minutes earlier. A developer who typed "add a retry policy" instead does not hold the answer. The longer the session ran, the less they remember of the code it produced, until the recovery costs more than the operation it serves.

Beaudouin-Lafon named the design move for this situation *reification*: turning something that exists only in a user's description of their work into a first-class object the system can display and they can manipulate [10]. Here the concept to reify is "the set of edits that served one request." Once that set is an object with a name, each of the five operations above becomes a one-argument

command instead of a manual reconstruction. Section 4 describes how we build that object.

4 What sgt Commits To

A version control system's unit of record decides which questions a developer can ask of their own history. The unit should match the grain a developer thinks at—because those are the questions they will actually ask. sgt records one edit per changed function and files it under the request the edit was made for. This section states that commitment and the three that follow from it, each with the price it carries. Figure 3 shows the first two of them at work on the sitting from Figure 1: what one save writes down, and what a version is.

The design philosophy is that a tool should refuse rather than guess wrong, and should report rather than act silently. A differently-minded designer would merge concurrent edits to one function the way git merges concurrent edits to one file, on the grounds that most such merges produce working code. We tried that first and abandoned it because when the merge fails the developer has lost work they did not know was at risk, and the failure appears somewhere other than where the merge happened. Another reasonable design would ask the developer at save time which request each function belongs to. We tried that too, and the developer cannot answer it without reading the diff, which is the labour the tool was supposed to eliminate. The four commitments below are what survived: each one is the design we arrived at after building the alternative and finding its cost.

4.1 One edit to one function, filed under the request

A save reads the working files, compares them against the last recorded state, and writes one record per changed function, naming the function, the version of it the edit started from, the version it produced, and those other functions the new code refers to that sgt can identify without guessing: a reference is recorded when the name it uses resolves to exactly one function in the codebase, and left unrecorded when the same short name is defined in several places, because a reference attributed to the wrong definition is worse than a missing one. The developer supplies one sentence, `sgt save -m "retry the upstream call when it returns a 5xx"`, and sgt reads everything else out of the files, so a record can be no more wrong than the code it was read from.

We chose the function as the unit because the three alternatives each fail on the same ordinary case. The price cache in Figure 3 added a parameter to `db.py::query` and passed a matching argument from `api.py::handle`, and those two changes share no line and no file, so a record that bottoms out at a line has nowhere to write down that they were made for one reason, and the developer who removes the cache a week later learns about the second one when the import fails. A record whose unit is the file cannot say that two changes to `api.py` served different purposes, which is the default case in Figure 1 where three requests all edit the same handler. GitButler uses this unit and solves the problem it can solve, which is the case where one feature per file suffices [46]. A record whose unit is the commit holds whatever was pending when the developer typed the command, which in Figure 1 is three requests and a rename the agent did on its own initiative, divided across two commits by the clock. A function is the smallest thing that both the developer and the agent name out loud while the work is happening, and the feature location literature works at that same unit because the code serving one concern is a set of functions scattered across files [12, 35].

sgt finds functions in Python, TypeScript, and TSX with tree-sitter [15] and records every other file as its whole content, at the grain git already offers, so the rest of this section applies only to files sgt can parse. Within the three parsed languages a function can be smaller than a purpose. The rename in Figure 1 moved `db.py::fetch` to `fetch_row` and updated four call sites, so sgt writes five records for what the developer thinks of as one action. Because those five records share one name, the mismatch is invisible until somebody reads the list underneath it. A function can also be larger than a purpose, which happens when two requests change opposite ends of one fifty line function, and Section 4.3 describes what sgt does then.

4.2 A version is a set of edits, and the files are rebuilt from it

Two rules decide which sets are versions. No edit is included without the edits it was built on, and each function has one included version. sgt rebuilds each file by writing out the included version of every function it contains and keeping the text between functions exactly as it was, which produces the bytes that are checked out. That interstitial text (whitespace, comments between definitions) is itself recorded as a layout symbol so that its position is deterministic under set operations. (Section 9.5 reports the cost of that choice.) The middle panel of Figure 3 is one such set before and after a revert, and the right panel is the file each set produces.

The alternative is to store patches and replay them, which is what Darcs and Pijul do at the line level [89, 114]. We chose sets over replay because replay introduces ordering: reverting a feature means computing an inverse patch and applying it to the current state, and the result depends on what the current state is. A set-based revert is idempotent (removing what is already absent changes nothing) and its inverse is exact (restoring a removed set reproduces the original bytes with no drift). Those two properties make the preview trustworthy, because the files the preview shows are the files the operation will produce, regardless of what else has been recorded since. The cost is that every function must have exactly one included version at any moment, which is the constraint the fork rule (Section 4.3) enforces.

Because a version is a set, `revert`, `restore`, `diff`, and `switch` all take a set and return a set, and two properties follow that a developer can check on their own repository. Reverting the price cache and then restoring it produces a file byte-identical to the one that was there before either command ran, because both operations are set arithmetic over the same records and neither reapplies a patch. Each of the four can also be shown in full before it happens, since working out the resulting files takes no more than rebuilding them from the proposed set: `sgt revert "price cache"` draws the twelve edits that would remain, names the functions whose text would change, and asks `apply this revert? [y/N]`. Run with no terminal attached it prints the same preview and declines, so

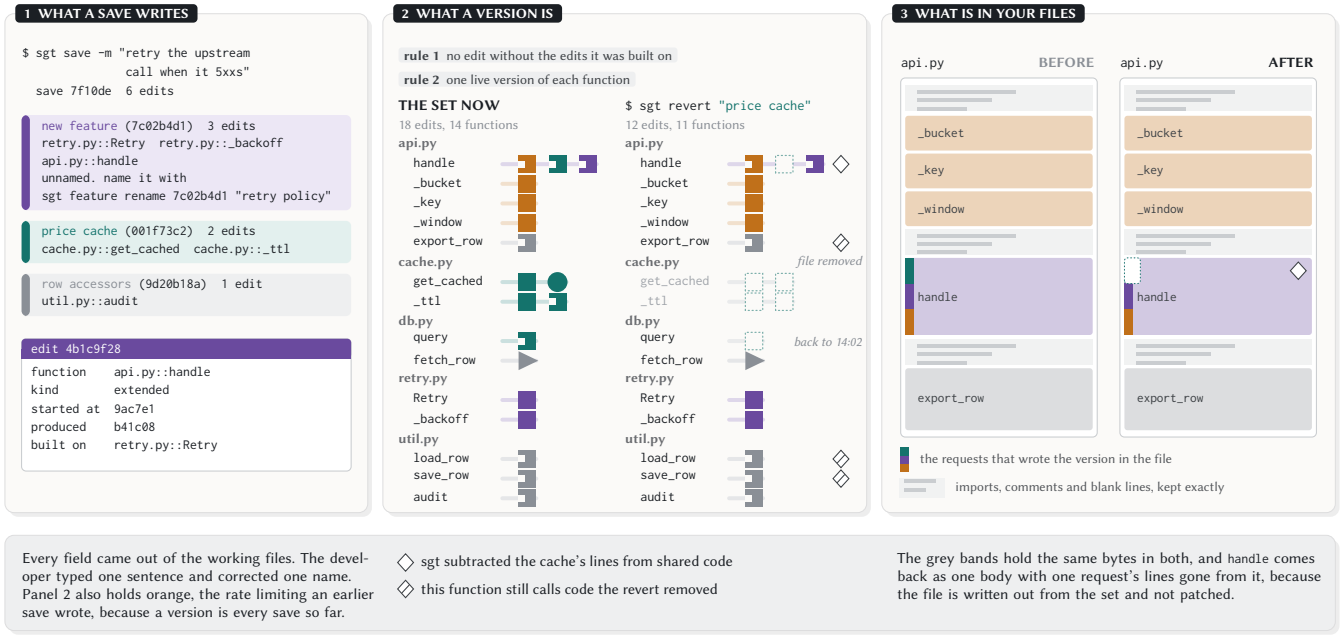

Figure 3: What one save writes down, what a version is, and where the bytes in a file come from. Every operation in this paper is arithmetic on the set in the middle panel, and the right panel is what that arithmetic does to a file the developer has open.

a revert nobody watched does not happen. The safety property (no mutation without confirmation) held throughout the evaluation; the *information* property did not. During the evaluation the machine-readable preview omitted the subtraction report entirely, and its one-line summary asserted "nothing depends on it" while its own payload carried a non-zero dependent count. An agent that checked the preview to decide whether to confirm was therefore given less information than a developer reading the terminal.

Three requests wrote `api.py::handle`, and in Figure 3 it carries an edit from the rate limiter, one from the cache, and one from the retry policy, in that order. The retry policy was built on the cache's edit (it starts from the version the cache left behind), so the dependency rule chains them: removing the cache alone would orphan the retry. We shipped that strict interpretation first—a revert of the cache also removed the retry—and it emptied a test repository, because on any history where features interleave inside a shared function, removing an early edit takes every later edit with it. The default now keeps the later edits and subtracts the reverted feature's contribution from the current text of the shared function with a three-way merge, the same operation git performs when it merges a text file. The version of `api.py::handle` that Figure 3 leaves in the file is produced that way, and the hollow diamond beside it is how sgt says so. The old behaviour remains available as `revert --take-dependents`, and its help text says what it removes.

That three-way merge is the only place sgt changes text that no person wrote, so sgt names every function it changes that way and every function it decided to leave alone. When the subtraction overlaps a line a later edit changed, sgt leaves the function exactly as it is and prints `kept unchanged (the removal overlaps later edits — needs your edit)` with the function named, and when a function sgt is keeping still calls a function the revert removed, it prints `still references removed code (fix or revert separately)`. Both are reports and neither stops the revert, because the alternative is a command that refuses to finish over a consequence the developer may already intend. The second warning depends on the reference field described in Section 4.1, which requires a globally unique function name, so on repositories where short names recur (monorepos, projects with many modules) the warning is structurally absent and the developer's test suite is the only oracle. Across the evaluation corpus, 6.7% of records carry any reference edge (the per-repository range is 0.0–20.5%). The dependency-based refusal in Section 5's Wednesday scenario therefore represents a capability most records cannot trigger: a repository whose function names are not globally unique sees none of it. The cost of the default is that after the merge the shared function is text the developer has not read, and their test suite decides whether the revert was correct. Naming the function before the revert runs is as far as we are willing to go on the developer's behalf.

4.3 Two versions of one function, and a person decides

When two edits change the same function from the same starting version, sgt records both, includes neither, and asks the developer. This is the commitment that costs the most goodwill in the first week, so we want to be precise about whose behaviour we are departing from and why theirs is right.

Git merges text and knows nothing about what a function is. A system that has to work on every text file in every language anyone will ever write can afford to know nothing else, which is why `git merge` is useful on a Makefile. In the case Figure 4 draws,

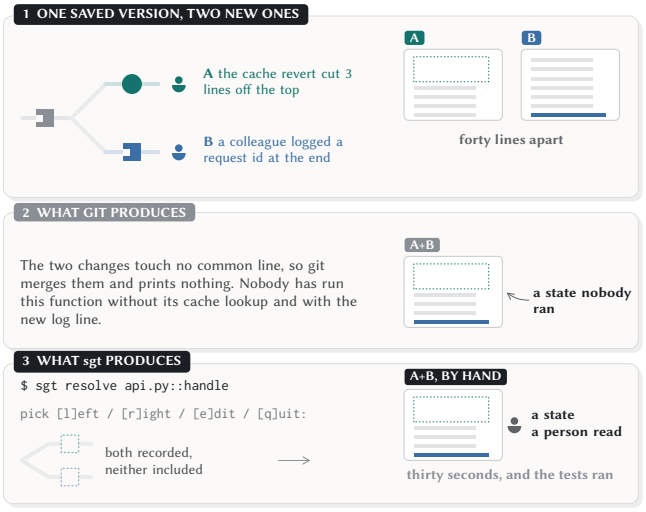

Figure 4: Two edits to one function from the same saved version, forty lines apart. Git produces a state nobody ran and sgt produces a question, and of everything sgt refuses this is the only refusal that can stop work which would have been correct.

two people had last saved the same version of `api.py::handle`, one of them reverted a caching feature, which took the cache lookup out of the first three lines, and the other added a request identifier to the logging block forty lines away, with no line in common. Git merges the two without comment, and the function that comes out has never been run in that state by anybody. sgt sees one function with two versions from one parent, names it, and stops. `sgt resolve api.py::handle` prints the two versions beside each other and asks `pick [l]eft / [r]ight / [e]dit / [q]uit`, where the third choice opens the file in the developer's editor and the fourth leaves the fork open, and whatever they write has to pass the project's test command before sgt will include it. Where no test command is configured the developer supplies the verdict with `--override pass` and a `--reason`, and the reason and their name are recorded beside the resolution, so the next person to read that function finds out why it looks the way it does.

A developer pays for this by being stopped in cases where the merge would have been fine. In one of those cases sgt stops for nothing at all: deleting a function and adding it back with byte-identical content in two places produces two versions from one parent, and sgt treats content that agrees exactly as a disagreement. The design is a fail-stop commitment in Schlichting and Schneider's sense [121]: the system converts a potentially-wrong state (a textual merge whose correctness nobody has verified) into a detectable halt (the developer is asked). Fail-stop costs availability—the tool is unavailable until the developer answers—in exchange for a guarantee about everything it does let through: if the system did not stop, the included version is one a person chose, not one a program composed. The read layer inherits this property: every attribution, dependency display, and consequence preview rests on versions that entered through a human decision, so the reports are as trustworthy as the decisions they trace back to. A silent textual merge would break that chain, because the function it produces has never been chosen by anybody, and the next revert that touches it would be operating on text whose provenance is a program's guess rather than a person's act.

We hold the commitment loosely, because a developer who merges thirty branches a day would reasonably weigh correctness against availability the other way, and for them the current design is worse. The 18× amplification Section 9.1 reports is the measured cost of choosing correctness over availability at this grain, and a tool builder facing the same choice should treat that number as a lower bound on what fail-stop costs at the function level.

The deeper reason for accepting that cost is the context in which the tool operates. A developer who typed the competing versions themselves would have read both, would hold a model of what each does, and could evaluate a textual merge in seconds. A developer whose agent produced one version while a colleague's agent produced the other has read neither, holds no model of the interaction between them, and would accept a textual merge not because they verified it but because the tool offered no reason to look. Shen and Tamkin measured this directly: developers who relied on AI assistance scored 17% lower on assessments of the code they had just produced, with debugging—recognising when code is wrong—showing the largest gap [125]. The fork rule is designed for the developer in that degraded state: it forces a decision point that requires looking at the two versions, which is the cognitive engagement their earlier delegation removed. A tool that silently merges code its user has not read, in a context where the user's capacity to read critically has already declined, is compounding the problem it was built to address.

4.4 The names are inferred, and correcting one moves labels only

Every figure in this paper labels a group of edits with a name like *price cache*, and those names are the part of sgt that guesses. Functions that changed in the same saves and that refer to one another are grouped together, using the change coupling signal established by Zimmermann et al. and Ying et al. [159, 165] and a community detection method over the resulting graph [143]. The reason a graph algorithm finds real features rather than arbitrary cuts is that codebases exhibit what Simon called *near-decomposability*: interactions within a subsystem are strong and frequent while interactions between subsystems are weak and infrequent [129]. Community detection recovers that structure because it was designed to, and the known exception—a hub function called from everywhere—is the same entity that breaks feature location in the literature and that Section 10 reports as the main source of grouping errors here: it pulls otherwise separate requests into one community because the coupling signal cannot tell structural proximity from shared purpose.

A language model then reads the saves in a group and writes the label, which is the descendant of a task attempted before language models existed, when Buse and Weimer generated a description of a change out of the change itself [17]. The label draws from the developer's words rather than from the code's identifier names. Nguyen and Glassman showed that the naming a model uses in

its output and the naming a developer uses to describe the same behaviour systematically mismatch [100]. A label derived from identifier names would therefore speak the model's language rather than the developer's, and the developer who later searches for "the caching" would find nothing under whatever the model called its cache class. Membership is computed from the code and the label is written from the words, and those two steps fail in different ways, so we treat them separately. The grouping is also systematically finer than the request: a community detection algorithm cuts where the coupling is weak, not where the developer stopped typing, so one request that touches several disjoint parts of the code becomes several features. Section 10 reports a median of 4–6 features per request across four repositories. The result is a sub-intent decomposition rather than an intent recovery—the developer gets a vocabulary for the structural neighbourhoods their request produced, not a map back to what they asked for. The finer grain has a cost and a benefit. The cost is that the developer cannot name what they asked for in one selection; they must merge or select several. The benefit is that operations work at a finer level than the request: the developer who asked for caching and later wants only the TTL helper removed can name that piece without reverting the whole request. The mismatch is in level as well as in number: the developer thinks at the level of "the price cache" (one purpose, one name) while the tool presents four structural communities inside it, a difference Section 10 traces to Rosch's distinction between basic-level and subordinate categories [113].

We chose to operate at the subordinate level rather than the basic level for a reason that becomes clear only when the developer acts on the grouping. A request boundary is unstable: it depends on what the developer typed, which changes between "implement caching" (one request) and "add a TTL helper, then wire it into the handler" (two requests for the same code). A structural boundary is stable: it depends on which functions call which others, which does not change when the developer rephrases. A revert, a restore, or a dependency warning all need a boundary that means the same thing regardless of when the developer asks—and a boundary defined by coupling has that property while a boundary defined by request phrasing does not. The over-decomposition is therefore not an error in the clustering but a consequence of choosing stability over correspondence: the tool gives four addressable pieces where the developer expected one, because those four pieces are the ones whose membership will not shift when the next request arrives. The cost (merging to act at the request level) is paid once; the benefit (stable addresses for operations) pays on every subsequent use.

We considered asking the developer to confirm the grouping at save time and decided against it, because answering that question takes the same knowledge commit hygiene takes, and a developer who could reliably answer it could also have staged the hunks. Shipman and Marshall showed that systems which ask users to formalise their own work get abandoned [127]; the design they arrived at has the system propose a structure the user may accept, modify, reject, or leave alone, which is the arrangement here. Our conditions are easier than theirs: the structure their systems tried to elicit was tacit, so a user had first to work out what they thought. The structure sgt recovers was spoken in a request before any of the code existed—though not at the grain the tool produces, since one request typically splits into several structural clusters. The reason it needs recovering at all is that nothing wrote it down. What matters is therefore not whether the guess is right but what a wrong one costs, and correcting one moves labels and touches no recorded edit. `sgt feature rename 001f73c2 "price cache"` renames a group whose identifier stays what it was, `sgt feature regroup` moves functions between groups, and `sgt save --as "price cache"` files a new save into a group that already exists. Because identifiers survive all three, a revert written down in a code review comment last month still removes the same edits after somebody renames the group today.

The deeper function of inferred names is coordination. Clark's analysis of language use shows that people who need to refer to the same thing build a shared vocabulary through proposal and acceptance: one party offers a term, the other either adopts it or repairs it, and after a few exchanges the term is common ground [27]. A version control system that infers feature names is proposing one half of that exchange—offering "price cache" as the term both the developer and the tool will use to pick out a set of edits. If the developer types `sgt revert "price cache"` the proposal has been accepted and the name is grounded; if they rename it, the repair has completed the same grounding through a different path. Either way, the tool and developer now share a vocabulary for the codebase, and that vocabulary enters the developer's prompts to the AI assistant, which is why the "prompt specificity" measure in Section 8.3 tests whether the tool's names reached the developer's language. A wrong label costs a rename (one command, no data moved); a missing shared vocabulary costs every subsequent conversation about the code, because the developer and the tool have no common referent for the set of edits they both need to name.

Two failures show up often enough that a reader will hit them. A function that everything calls, an argument parser or a logging helper, changes in nearly every save and pulls otherwise unrelated work into one group, and the fix is a regroup the developer has to notice they need. A save with no message gives the language model less to read, and sgt says so at the time, printing `· no words captured` under the save it just wrote, because the label summarises what the developer said about the work and a developer who says nothing gets a name derived from the code alone.

4.5 What sgt will not do

sgt writes no code of its own. Every byte in the working files came from the developer or their agent, apart from the three-way merge splice in Section 4.2, which reports every case it cannot perform cleanly. We have been asked more than once during this work why sgt does not hand the shared function to a language model and have it redrafted after a revert. We considered it seriously and rejected it for one reason: a revert whose output depends on a model's judgement produces a file whose correctness the developer can verify only by reading every line the model touched, which is the labour Section 3 argued this tool should eliminate. A revert whose output is computed from the records the developer already validated produces a file whose correctness follows from the records, and the developer's test suite decides whether the composition is sound. The cost is that a subtraction which overlaps later work requires a hand edit; the benefit is that the developer knows what they are reading. Bansal et al. found that AI explanations increase acceptance

whether or not the underlying recommendation is correct [6]; a model-in-the-loop revert would face the same problem, because the developer would have to judge whether the model's redrafted function is correct, and the tool's report that it passed tests is exactly the kind of explanation that increases acceptance without increasing accuracy. The refusal has a deeper justification. A version control system is infrastructure: it sits beneath every other decision, and a revert that is wrong invalidates every test, every review, and every deployment that followed it. An AI assistant is a collaborator: its suggestions are proposals that a developer accepts, modifies, or rejects one at a time, and a wrong suggestion costs one undo. The failure modes are not symmetric. A wrong line completion costs the developer a delete; a wrong revert costs them their codebase state, and the difference in blast radius is the difference between a tool that should reach for a model (the assistant) and one that should not (the substrate). The trend toward model-in-the-loop for every operation conflates the two roles, and the conflation is invisible as long as the model is correct. The moment it is wrong, the developer discovers that the substrate contained a guess they never verified, and the cost of discovering it late is what Section 9 measures.

sgt also does not stand between a developer and git. The records are files under `.sgt/` committed alongside the code, `sgt git <args>` passes through to git untouched, a colleague who has never installed sgt clones an ordinary repository with an ordinary history, and there is no server and no background process. Two commands refuse to run: `switch` stops when there are unsaved edits, because sgt has nowhere to put them, and names the files and prints `record them with 'sgt save', or commit / 'git restore' those files, then re-run`; `restore` stops when putting a set back would leave two versions of one function live, and there its message names the edits by identifier while the function whose two versions caused the refusal goes unnamed, which is the rough edge Section 5 reports us fixing in the fork warning and not here. Everything else sgt finds wrong it reports, and then it proceeds.

4.6 From the five operations to the commands a developer types

Figure 5 sets out the nine operations that act on the record, each with its cost beside it, and the five operations Section 3 ended on are four commands and one thing that needs no command. Recording a boundary without declaring it is `sgt save -m`, run whenever the developer feels like running it, since the grouping is recovered afterwards from the code. Seeing the work one request produced is `sgt show "rate limiting"`, or `sgt show "rate limiting@1"` for one stretch of work on it alone. Removing a named set is `sgt revert "price cache"`, and carrying one to another branch is `sgt propose land --subset "export"`, which declines a subset that omits a group another chosen group depends on and names the group that is missing. Each of those four takes as its argument a phrase somebody used while the work was happening, so the manual recovery Section 3 described is replaced by a named command—on repositories where the store reproduces the committed bytes. Where it does not (27 of 28 outside repositories), the command refuses rather than producing a wrong file, and the developer is back to the manual recovery with no help from the tool (Section 9.1).

Discovering the layering needs no command at all wherever the references resolved, because the cache edits record that they were built on `api.py::_bucket` and therefore on the rate limiter, so the developer has at any later moment the fact they needed at 14:30 and did not have. Where they did not resolve, the layering is simply absent rather than wrong, and the developer is back to reading the code, which is the position the ambiguity rule above trades down to on purpose. On 35 corpus repositories, 93% of ops carry no reference at all, because resolution requires a globally unique function name—so the absence is the common case, not a rare failure. The 7% that do resolve tend to be cross-module calls (a handler calling a cache function in another file, a test importing a fixture), which are precisely the dependencies whose absence would break the code silently—so the coverage is thin but concentrated on the cases where a human would not have noticed the dependency either. The revert output's warning ("still references removed code") can only fire when resolution succeeded, which on most repositories is never. The implication for adoption is blunt: a developer whose repository has few cross-module calls will never see a dependency warning, and will discover that the tool offers no protection there only when a revert breaks something silently.

A reader should treat the dependency display as a bonus that fires occasionally rather than a guarantee the design rests on; the design rests on set arithmetic over edits, and the dependency enriches a revert without being required for one. The rest of what a developer types is housekeeping around those nine, `sgt init` to begin, `sgt log` and `sgt sync` to see where things stand, and the `sgt feature rename` of Section 4.4 to correct a name, none of which changes which edits a version includes. Section 5 runs these in order on one repository.

5 One Repository, Five Sittings

The repository here is a five thousand line Python service that answers price quotes for a trading dashboard, and the first sitting is the hour Figure 1 draws. We assembled the sequence from sessions we ran while building sgt so that one repository contains the five operations Section 3 asked for, in that order, followed by the one case where sgt stops and asks. Every command below runs and every block of output came out of sgt as it is today—lines too wide for this column are wrapped at · separators, two displays too tall are replaced by bracketed descriptions, and the rest is unchanged. The sequence is arranged to show which decisions the developer is left to make and when in the work each one arrives. This is the case the design was built for: a repository recorded live, by one developer, save by save. Section 9 reports what happens on repositories the tool did not record live and under randomised sequences that exercise combinations no developer would hit in a single sitting.

5.1 Monday, 14:02 to 15:20: two saves and four names

At 14:33, with the rate limiting finished and the price cache first drafted, the developer saved.

```
$ sgt save -m "rate limiting and a price cache on quotes"
```

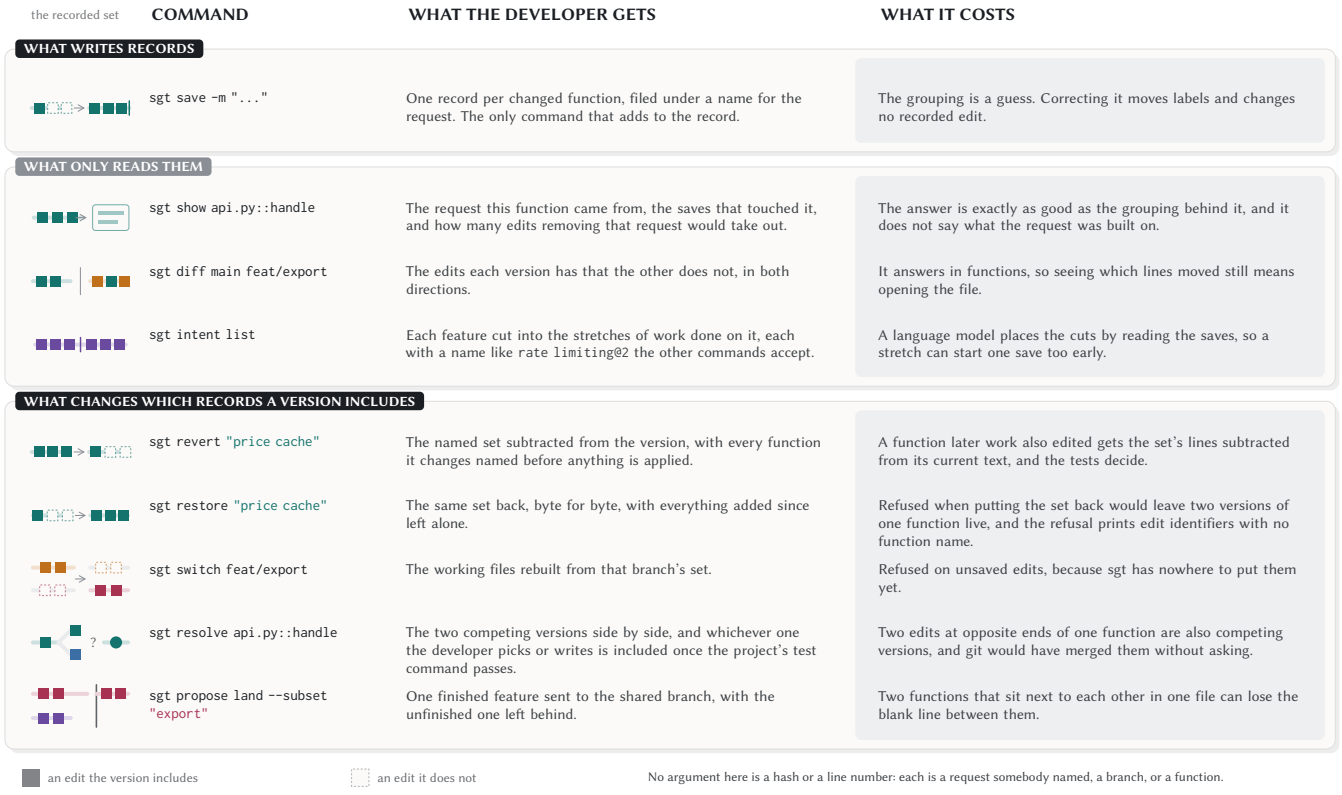

Figure 5: The nine operations that act on the record, grouped by what each one does to it. One command writes records, three only read them, and five change which of them a version includes.

```
✓ save c39ab2f "rate limiting and a price cache on quotes"
├─○ new feature (4a1c9e02) — unnamed; name it:
│  sgt feature rename 4a1c9e02 "<label>"
│  api.py::_bucket, api.py::_key, api.py::_window +1 more
├─○ new feature (001f73c2) — unnamed; name it: . . .
│  cache.py::get_cached, cache.py::_ttl, db.py::query +1 more
└─○ new feature (9d20b18a) — unnamed; name it: . . .
db.py::fetch_row, api.py::export_row, util.py::load_row +1 more
△ one save touched 3 features — deliberate? `sgt log --map'
shows them; `sgt feature regroup move' re-files work
↶ reverse this save: sgt undo
```

The developer typed one sentence and sgt wrote twelve records. The three groups have no names yet, because sgt writes records when the developer saves and works out the names when the developer next reads, and a name costs a language model call that has no business running while somebody is mid-session. The warning is the one line here that resembles what git asks for, and sgt printed it after the save was already written—a deliberate ordering, because blocking the save to ask "are you sure this is one thing?" is the formalization tax Shipman and Marshall showed drives people away from structured tools [127]. The developer read it, agreed the sitting had been mixed, and carried on, which is the case Section 3 opened with.

The second save at 15:20 covered the retry policy and a rework of the cache the agent made while adding the retry. By then the first save's groups had names, so its echo listed • `Price Cache (001f73c2)` for the group that already existed and ○ `new feature (7c02b4d1)` for the retry work. `sgt log --tree` afterwards printed the four groups the hour produced.

```
Rate Limiting (4a1c9e02) · 4 symbol(s)
Price Cache (001f73c2) · 4 symbol(s)
Retry Policy (7c02b4d1) · 3 symbol(s)
Row Accessors (9d20b18a) · 5 symbol(s)
```

Three of the four names are what the developer would have written, and the fourth is the agent's unrequested rename, named from the code because the developer never said anything about that part of the work, which is the failure Section 4.4 predicts. It is also the only one of the four whose name nobody needs, so nobody corrected it.

5.2 Thursday: changing the rate limiter

The fixed one minute window has to become a sliding one, and the developer starts from what the rate limiting is now.

```
$ sgt show "Rate Limiting"
feature 4a1c9e02 "Rate Limiting"
 4 edits · 4 symbols in 1 file · last touched 3d ago
 symbols api.py::_bucket, api.py::_key, api.py::_window,
 api.py::handle
 saves c39ab2f rate limiting and a price cache on quotes
 reverting this removes 4 edits
```

The four functions are the answer to the question that took four runs of `git log -L` in Section 3, and nothing here required the developer to guess that four was the right number of places to look. What this

output does not say is that the price cache calls `api.py::_bucket` and therefore sits on top of the rate limiter, so a sliding window is a change underneath work that already depends on it. The record holds that fact, because the cache's edits name the functions they refer to, and `show` declines to print it, so the developer rewrites `_bucket` without being told who reads it. `propose land` refuses on this same relation on Wednesday, which is what makes the omission here a decision about what to report and not a gap in the record.

The agent rewrites `api.py::_window` and `api.py::_bucket` and leaves the other two functions alone, and the developer saves, naming the group this time because they already know which one it is.

```
$ sgt save --as "Rate Limiting" -m "sliding window, 60s of buckets"
✓ save 1f0aee3 "sliding window, 60s of buckets"
  └• Rate Limiting (4a1c9e02) api.py::_bucket, api.py::_window
  ✓ named "Rate Limiting"
```

`--as` is the one place a developer states a grouping by hand, and it changed nothing here, because both functions were already in that group and the edits would have been filed there without it; they typed it because they knew the answer and had no reason to let sgt work it out. The rate limiting now has two stretches of work on it, and `sgt log --focus 4a1c9e02` names both.

```
• Rate Limiting 4a1c9e02 · 2 checkpoint(s)
[0■■■] 4a1c9e02@0 :rate-limiting Rate Limiting
[1■■ ] 4a1c9e02@1 :sliding-window Sliding Window
operate:
sgt revert 4a1c9e02:sliding-window (one checkpoint, by name)
sgt revert 4a1c9e02 (whole feature)
sgt revert 4a1c9e02@1 (by index)
```

The developer never asked for these two stretches. sgt cut the work where their words changed, which in this repository is where one request ended and the next began. Either half is addressable on its own: `sgt revert "Rate Limiting:sliding-window"` takes Thursday's work off and leaves Monday's four edits in place. The bar beside each name puts that checkpoint's edits in their place in the history, read left to right, with a middle dot where a span of the history holds none of them. The terminal spends four characters on the encoding Figure 1 spends a panel on. The three forms under `operate` are printed because people on our own team found that a stretch of work had a name and then stopped, not knowing what would accept it.

5.3 The following Monday: taking the cache out

The cache serves a stale price for up to sixty seconds and has to go.

```
$ sgt revert "Price Cache"
 [the log region redrawn, with the 6 removed edits marked]
 subtracted from shared code (later work kept): api.py::handle
 removed going forward: cache.py::get_cached, cache.py::_ttl,
 db.py::query
 △ still references removed code (fix or revert separately):
 api.py::export_row, util.py::load_row, util.py::save_row
apply this revert? [y/N]
```

The retry policy edited `api.py::handle` after the cache had edited it, so that function keeps the retry work and loses the cache's three lines, and a request made a week ago comes out from under a change made yesterday. Two functions existed only for the cache and come out with it, which empties `cache.py` and removes the file, while `db.py::query` loses the parameter the cache added and goes back to the text it had at 14:02. The three functions in the last line are row accessors the agent rewrote during its rename. The agent pointed all three at `get_cached` while working in that code, so their edits belong to a request the developer is keeping yet name a function the revert deletes. sgt names them and proceeds, because whether the repair is deleting three call sites or reverting the rename as well is a question about the developer's intent, which the recorded edits cannot answer.

The developer answered `y` and ran the test suite, which failed in the three places the report named and nowhere else, and the repair was deleting the cache lookup from each of them. sgt builds every one of these operations as set arithmetic so that a revert can be taken back exactly, and the developer tested that claim before trusting it.

```
$ sgt restore "Price Cache"
 ✓ restore applied — 0 edit(s) removed, 6 added.
 (`sgt undo' reverses this.)
```

Putting the cache back returned `cache.py`, `db.py::query`, and `api.py::handle` to the bytes they held before the revert, and it left the three call site repairs the developer had made in between alone, because a restore adds records to the set and rebuilds the files from the result without reapplying a patch. They had wanted the cache gone, so they reverted it a second time, and that run printed the same three consequence lines as the first. Both times the report fired because the revert was a removal: six records left the set entirely, and three functions lost the definition they called. Rolling a checkpoint back—`sgt revert "Rate Limiting:sliding-window"`, had the developer needed it—would leave every function present in some version, so nothing the remaining code references would disappear and the report would be empty.

5.4 Wednesday: shipping the export fix alone

The export endpoint from Section 3 has to ship to the release branch today, and the branch it sits on also carries the half finished retry policy. The export code goes into a new `export.py` and applies the rate limit before streaming, so `export.py::write_csv` calls `api.py::_bucket`, and nobody thought about that call as a dependency while writing it.

```
$ sgt propose create --base release
✓ proposal 3f9a2c81b04d (base release, +17 op(s), 4 feature(s))
$ sgt propose land 3f9a2c81b04d --subset "Quote Export"
× 'Quote Export' requires Rate Limiting -- include it in
 --subset too
```

The developer asked to move one group and sgt named the group that has to come with it, which is the fourth operation Section 3 asked for. The record of the call to `api.py::_bucket` was written at the moment the export code was saved, and sgt reads it now to answer a question nobody was thinking about then, where git would have taken the cherry-pick and the release branch would have failed to import.

```
$ sgt propose land 3f9a2c81b04d --subset "Rate Limiting" "Quote
Export"
✓ propose land release: 8b21c0d4e7fa (+8 op(s))
```

Two groups went to the release branch in the order the records put them in, and the half finished retry policy and the accessor rename stayed behind. Those two groups are the stack from Section 3, assembled without the branch that had to exist before the work did.

5.5 Thursday: two versions of one function

A colleague on this repository had been editing `api.py::handle` as well, adding a request identifier to every outbound log line, and neither of them knew about the other's edit until the developer pulled.

```
$ sgt sync
△ sync origin/main: merged 4f2a1c8b93de
 +7 op(s)
 △ 1 OPEN FORK(S) — forked symbol(s) sit at the common
 ancestor until resolved:
 api.py::handle → sgt resolve api.py::handle
```

The remedy is printed as a function name because that is what the developer knows about the situation. For months the fork warning printed `sgt advanced merge-op 3c91ab02 8ee14d77` instead, which names the two records in conflict exactly and leaves the developer to work out which function they are in, and it is the clearest instance in this repository of a report that was accurate and useless. The two versions of `api.py::handle` are the one Monday's revert produced and the colleague's, which changed no line the revert had touched, so this is the case Figure 4 draws, reaching the developer as a consequence of an operation they ran earlier the same week. sgt prints them beside each other and asks.

```
— fork on api.py::handle — (◄ left = tip A ► right = tip B)
 [the two versions of the function, side by side]
pick [l]eft / [r]ight / [e]dit / [q]uit:
```

The developer picked `e`, merged the two by hand in about thirty seconds, and sgt ran the project's test command before including the result. We are not going to claim the textual merge would have been wrong here, because it would most likely have been right, and sgt charged thirty seconds and one interruption to establish that. We designed for the case where the merge is wrong—one edit adding an early return and the other edit's new code sitting below it, unreachable. The test run that would have caught it happens here, on a branch nobody has pushed yet. A developer who merges often and reads carefully pays thirty seconds for a certainty they already had; the design bets that when the developer did not write either version of the function, they do not read carefully enough for the certainty to be free, and the evidence in Section 7.5 is that even careful developers missed consequences the preview named.

5.6 What this cost

Across five sittings and one fork the developer made three decisions sgt could not make for them. They decided the export endpoint should ship without the retry policy, they decided how the two versions of `api.py::handle` should read, and they decided that the three functions still calling the cache should lose that call and keep the rest of the agent's rename. All three are decisions about the code itself, and none of the three is a question about which lines belong in which commit.

The developer paid for those three decisions in three places. One group of five edits carries a name derived from the code, because nobody said anything about that part of the work; the revert left `api.py::handle` holding text nobody had read, so their test suite decided whether the revert had worked; and the fork stopped them on a merge that was probably fine. The second and third are direct consequences of Section 4.2 and Section 4.3, and on a repository this size we would choose both again, because both are costs a test suite and thirty seconds can settle. The costs git would have charged instead are the ones Section 3 describes: reading the diff to find which hunks belong to which request, rebuilding a decomposition the developer never held, and discovering a consequence nobody named when the release branch fails to import.

6 What Would Show This Is Wrong

Three claims in this paper fail independently of one another, and each fails in a way that a different kind of evidence catches:

(1) *The record reproduces the bytes.* A question about a program, settled by running it over histories that already exist.
(2) *The grouping matches what was asked for.* A question about an inference, settled by comparing it against transcripts where someone wrote the requests down.
(3) *A developer gets more operations right with sgt than without it.* A question about people, settled by experiment.

The first has a procedure and a number (Section 9). The second is evaluated against agent transcripts where the request boundaries are known. The third is a within-subjects laboratory study whose results appear in Section 8. Underneath all three, sgt has been the version control for its own repository since the day it first ran—which is where every number in Section 9 comes from, and why the two prices Section 4 states most flatly are the two that cost us something before they cost anybody else.

We state the measures before presenting the data so that a reader can evaluate whether the result matches the prediction or was chosen after the fact to make the design look good. The pilots in Section 7 tested the tasks, the tools, and the rubric; the full study followed the same protocol with twelve new participants.

6.1 The rebuild has to be exact

If sgt reassembles a file into bytes that differ from what the developer left there, then every operation in Section 4 is set arithmetic over a fiction, and nothing else in this paper is worth reading. This is the one measurement whose failure would end the work rather than qualify it, so it goes first and it is the only one we insist on before adoption.

The procedure follows the shape of the claim. We walk a repository's history one commit at a time, mine each commit into records, rebuild every file from the set of records that commit's state implies, and compare the result against the blob git holds, byte for byte. A repository passes at a commit only if every file matches, so a single differing byte in a single file is a failure at that commit. We report the fraction of files that rebuild rather than an average over repositories, because a developer cares whether their file comes back and not whether most files do. The corpus is open source Python and TypeScript repositories chosen for length of history and nothing else, and we are still assembling it, so the only number this procedure has produced comes from our own repository, which is the hardest case available to us because records written by five different versions of our miner sit in it.

That number is fifty-six files that no valid set of records reproduces, one in seven of the files whose records name functions, and Section 9.4 gives that denominator and says what is in the fifty-six. What we do not know is whether fifty-six is the cost of our own

format changes or the cost of the design, and since five miner versions in fifty-seven days is a rate no adopter will ever match, the corpus run is how we intend to separate the two: a repository sgt has never touched cannot have our migrations in it.

6.2 Whether the grouping matches what was asked for

The names in every figure in this paper are inferred, and a developer who types `sgt revert "price cache"` is trusting that inference with the contents of their files. The untangling literature evaluates clustering against a partition somebody drew, for the good reason that its authors had nothing better to compare against (Section 2). We ask a different question: whether the set that comes back when a developer names one request is the set of edits that request produced.

That question needs a corpus where the requests were recorded, which agent sessions now supply as a matter of course, since the transcript of a session names each request and the files the agent touched while carrying it out. We mine each repository without reading any transcript, derive a reference grouping from the transcripts alone, and then ask two things of every request in the session: how much of what that request produced comes back when the developer names it, and how much else comes with it. Those are the two ways a revert disappoints the person who ran it: the edit it missed and left sitting in the file, and the edit belonging to a different request that it carried off. They fail asymmetrically, and the second is worse. A developer who is handed too little notices when the code still runs the way it did. A developer who is handed too much has already lost work they meant to keep.

We expect the shared helper to be where this measurement goes wrong, because Section 4.4 predicts it and fifty-seven days of daily use have not stopped it happening. An argument parser or a logging function that changes in nearly every save is coupled to everything by the signal the clustering reads, and it draws otherwise unrelated requests into one group. If most of the disagreement between our grouping and the transcripts turns out to sit on functions of that kind, then the fix is to weight a function's coupling down as the number of things it couples to grows, and that is a change to one term rather than to the design. If the disagreement is spread evenly across ordinary functions instead, then the coupling signal is not carrying the grouping and the developer should be asked at save time, which is the design we argued against in Section 4.4 and would have to argue for.

6.3 Three requests, a transcript, and code nobody in the room wrote

The study puts a developer in the position the paper is about: a repository whose code they did not write and the three sentences that caused it. Each participant receives a five-thousand-line Python service, the transcript of the session that produced its most recent hour of work, and three tasks that name one request from that transcript and ask for it to be moved or removed without disturbing the other two. This is the state of the developer in Section 3 a week later, and it is also the state of a developer who picks up a colleague's branch on a Monday. The arrangement matters because it gives every participant the same starting knowledge. If participants directed the agent themselves, each would remember a different amount of what came back, and that uncontrolled remembering is the variable whose absence the paper claims matters.

Participants and counterbalancing. We recruit twelve developers who use a coding agent as part of their ordinary work. Each works through two repositories, one with git and one with sgt, in a counterbalanced within-subjects order (four groups of three, fully crossing condition order with project assignment). Within-subjects is the forced choice at this sample size: the variance between developers in git fluency, agent familiarity, and code comprehension speed is larger than the effect we hope to see, so each participant must serve as their own control. The two repositories hold different sets of interleaving requests so that a participant who learned the shape of the first task cannot apply it to the second.

What differs between conditions. Both conditions include the same AI coding assistant and the same editor. They differ only in how history is recorded and queried. The git condition includes GitLens, because comparing a graphical extension against a bare terminal would measure the presence of a view rather than the representation underneath it. Before the timed requests, each condition opens with ten minutes on a throwaway practice project where participants work through attribution, revert, and restore in that condition's vocabulary and may ask any question. The sgt practice sheet says explicitly that ten minutes will not produce fluency; what it controls is that no participant meets a command for the first time with the clock running.

Tasks and scoring. The three requests are: identify what a tangled commit introduced and when (provenance, five minutes, three closed questions scored against a key); remove one named feature without breaking the other two (entangled removal, fifteen minutes shared with the third request); and correct a dependency that the removal broke (a repair under time pressure that tests whether the participant noticed the collateral). With twelve participants we can detect large effects and nothing else, and the paper will say so in its limitations rather than obscure the sample size behind a p-value.

Request 1 is scored by key: three closed questions, each with one correct answer and an "I could not tell" option that is real data rather than a wrong guess. Requests 2 and 3 are scored by script, and the script asks three questions. The tests belonging to the removed request fail, the tests belonging to the two kept requests pass, and no function outside the intended set differs from the reference repository. That third check catches the failure this paper is about: a participant who removes the waitlist and quietly takes half the enrollment logic with it has produced a repository that passes its own tests—the enrollment tests left with the enrollment code, so the remaining suite gives a clean run over a broken system.

Reach prediction: does the representation change what the developer expects? Between provenance and removal, each participant completes two reach-prediction trials. A piece of work is named in product terms, and the participant sees a fixed list of twelve behaviours of the application—each written as something a person does with it—and answers one question: *which of these behaviours run through the code this piece of work added?* The answer is given in two stages. First, blind: from the representation alone (the feature list in sgt, the commit log in git) without opening the code. Then,

Table 1: The seven measures of the study in Section 6.3, each with the result that would count against the design. The first two rows are what we would report first; the fourth row is the one a reader would expect us to report first.

What we measure	What we expect	What would count against us
Reach prediction gain (`checked − blind` F1 against measured ground truth).	Positive gain under sgt: seeing the feature map moves the participant's prediction toward truth. Smaller or zero gain under git.	Gain at or below zero under sgt. The representation would then add nothing. Worse: high blind confidence with low blind accuracy—the inferred label confidently misleads.
Submissions the participant believed correct and that were not.	The gap here is wider than the gap in time. Removing a request touches code the participant never read, and in git nothing tells them so.	Equal rates in both conditions. The recovery we call expensive would then be something developers already do reliably, and Section 3 is a story about us.
Submissions that pass the check.	More of them with sgt, and the ones that fail fail in the places `revert` named out loud.	A failure in sgt that the preview did not name. A report that misses a consequence is worse than no report.
Time to the moment the participant says they are done.	Shorter with sgt, and we would not publish this alone.	Shorter with sgt while the first row is flat. sgt would then be a convenience, and we have argued for it as something else.
Lines of diff the participant opened before submitting.	Far fewer with sgt, since no command in Section 4 asks anyone to look at a hunk.	Participants reading the diff anyway. They would be checking sgt's report rather than using it, and after twenty minutes of that they are doing the git task with extra steps.
Forks sgt raised, and how many the participant judged pointless.	A small number, mostly judged pointless, all cheap.	Any participant switching the rule off, or a fork resolved by picking a side whose result then failed the check. Being stopped would have bought nothing.
Labels the participant corrected, and what the correction cost.	One or two renames, no regroups.	A regroup on the critical path of an operation. The inference would then be spending the developer's attention rather than saving it.

checked: using whatever the condition offers—history, code, the assistant—before submitting a final set. Confidence is rated at both stages.

Three numbers per trial, all objective:

- **blind**: F1 of the stage-one set against measured ground truth. Whether the representation is legible at a glance.
- **checked**: the same, after checking. Whether the developer can arrive at the truth at all.
- **gain**: `checked − blind`. What the representation bought them—the within-participant movement toward ground truth.

Ground truth is measured, not asserted: each behaviour maps to a CLI command handler, and a behaviour reaches the work when its handler transitively calls one of the work's functions. The two targets point in opposite directions on purpose (one reaches broadly, one narrowly), so no fixed number of ticks does well on both and the score is F1 rather than agreement. The result that would falsify us is `gain` at or below zero under sgt: the representation would then add nothing a developer could not already see. The result that would expose our weakest link is high `blind` confidence paired with low `blind` accuracy under sgt—the signature of an inferred label that confidently misleads rather than one that merely fails to help.

No published evaluation of a history tool reports whether a consequence preview moves a developer's expectation toward the truth. Every prior evaluation (Gitless, Azurite, Replay) reports completion time, success, and preference. We measure `gain` because it is what the tool is for: the claim is not that the tool saves time but that it changes what the developer knows before acting, and a wrong label can make us lose rather than tie.

The second row of Table 1 is what matters on the mutation side. A developer who spends twenty extra minutes untangling a commit by hand has lost twenty minutes. A developer who submits a repository they believe is clean—and that still calls a function they deleted—has lost something they will not find until the release branch fails to import. Hand untangling fails silently, and a design whose whole argument is that the developer no longer holds the decomposition has to be measured against silent failure rather than against the clock. We collect time because a reader would want it and because a tool that is correct and unbearably slow is not adopted, and we would not publish a time difference as the finding.

The last two rows of Table 1 measure the prices we stated in Section 4 rather than the benefits, and they are there because a study that measures only what a design buys is a study that cannot be wrong. A fork stops the developer and a wrong label makes them type a rename, both of which are countable inside the session, and both have a threshold past which we would change the design rather than defend it.

Three pilots (Section 7) validated this design before the full study: one in each condition tested the tasks, and a third exercised the study apparatus end to end. Both task conditions completed all six requests, confirming that the primary measure is error rate and cost rather than task completion. The pilots found twenty-one tool defects that would each have cost a real participant their data point, four defects in the study design that would have made a result uninterpretable, and one validity threat (a confound favouring our

condition) that we disclose in Section 7.4. The tool participants receive is a different version from the one the pilots ran.

6.4 What none of this would tell us

Only the first of the three claims improves by running more of the same: adding repositories to Section 6.1 sharpens that number. The other two hit ceilings that more data cannot move.

A recorded request is a sentence, and a sentence is less than an intention. A developer who types *make the price lookup faster* holds a view of what should change that the sentence does not contain, and the agent's response is a guess at the remainder. The alignment we measure against a transcript is therefore alignment with what was said, and the transcript sets a ceiling on it that better clustering alone cannot exceed. We think this is the right measurement anyway, on the ground that what was said is what the developer will type when they come back to remove it, but a reader who believes intention and utterance come apart more often than we do should read that number as an upper bound on something smaller.

Two hours with a transcript understates a week of forgetting. The cost we describe in Section 3 grows as a developer's memory of the session fades, and our design substitutes reading somebody else's transcript for that fading. A participant who has just read three sentences knows more about the session than the developer who typed them a week ago, so the git condition in our study is easier than the situation it stands for, and the difference we measure understates the one that matters. We prefer the understatement to a study nobody can run.

Finally, every task in the study has an answer a script can check, and we chose those three tasks for that reason. The operations a developer wants most may be the ones with no checkable answer, and deciding whether a stack of two reviews was the right shape is one of them. We can say what sgt makes possible there and we cannot measure it, and Section 9 keeps that distinction.

7 Pilot Study

We ran three pilots before the full study: one in each condition (sgt and git, both on the same task set) and a third exercising the study apparatus end to end. All three used AI coding agents as participants rather than human developers, because the purpose was to stress-test the tool and the protocol (can the tasks be completed? do the verbs work? does the scoring script produce a result?) rather than to measure human behaviour. The first two were designed to find defects in the tool that would cost a real participant their data point, and defects in the study design that would make a result uninterpretable. The third tested the machinery between the participant's machine and the facilitator's console—exactly the seam nobody had exercised—and found a validity threat that neither task pilot could surface. We report all three because the defects they found are consequences of design commitments rather than accidents of implementation: a technical evaluation establishes that a property holds across 10,237 operations; it takes a new user to show that a property that holds still costs too much to use.

Request	sgt condition	git condition
R1: Provenance	Correct, 4 cmds	Correct, 6 cmds
R2: Removal	Done; tool corrupted file, repaired by hand	Done cleanly
R3: Selective restore	`restore` crashed; recovered via git	Done cleanly
R4: Regression repair	Correct fix + better test	Correct fix + better test
R5: Plan & fork	Done in git (declined sgt)	Done cleanly
R6: History edit	Half done (no rename verb)	Done (scripted rebase)

Table 2: Per-request outcomes across both pilot conditions on coursecraft (same tasks, same rubric). The sgt condition ran on version 0.1.0; twelve defects fixed afterwards. R4 and R5 produced equivalent results in both conditions.

7.1 What the pilots tested

Six requests, drawn from the scenarios of Section 3: find which commit introduced two unrelated changes (provenance), remove a named feature while keeping the rest (removal), selectively restore a single function (selective restore), locate and repair a regression whose test still passes (regression repair), plan work on a branch and resolve a conflict (plan and fork), and split a tangled commit so each half has a clear name (history edit). Both conditions worked on the same five-thousand-line Python project and were scored with the same rubric and the same automated check. Table 2 summarises the outcomes.

7.2 What the sgt condition found

The sgt pilot completed four of six tasks and reached the correct answer on a fifth. Three of the eight verbs in the daily interface failed during the session: `revert` produced syntactically invalid Python, `restore` crashed with a raw traceback, and `save` refused to record a legitimate edit. The participant fell back to git for the repairs and finished the substantive work outside the tool.

Twelve defects were fixed from this one session. We describe three because they are consequences of design commitments this paper defends.

Semantic subtraction glued lines together. Reverting the wait-list feature left one file with three function headers on one line, `find_student` relocated to end of file, and thirteen collection errors in pytest—under a green checkmark. The cause is the round-trip design of Section 4.2: the rebuild synthesises zero separator bytes, so a removal that takes the layout chain of an entity it keeps produces definitions with no whitespace between them. The fix re-grounds the layout of every kept entity after a subtraction.

A preview flag mutated state. The tutorial teaches that every destructive verb shows a preview first and acts only with `--yes`. Adding `--keep-dependents` to a revert silently voided that contract, writing intermediate state into the store with no preview and no confirmation. The participant's verdict: "If a flag can turn a preview into an action, I can't reason about which of these commands are safe." The flag now requires `--yes`.

The map showed husk features. `sgt log --map` displayed a feature called *Section Waitlist* that contained zero symbols in zero files—a cluster whose only records were bookkeeping sentinels—while the feature that actually held the waitlist code was named after a deleted experiment and did not appear in the view at all. The participant followed the wrong name for the entire removal task. Later validation found the problem was systemic: 9 of 24 leaf features in each study repository owned no behavioural code, including the three features the study's own removal requests name. The mechanism was that the clustering graph includes positional gap-bytes as nodes (they carry within-file cohesion), and Leiden readily forms a community made entirely of them. The fix absorbs any leaf whose members are all non-behavioural into the leaf that already owns the entities those members hang off, reducing coursecraft from 24 leaves to 14 with none empty.

The comprehension operations (naming what a request produced, seeing what a revert would remove) held up under a user who had never seen the tool before; the participant credited "sgt's attribution and its revert preview, which git genuinely cannot do." The mutation operations did not survive contact, and the data-loss path that one of them opened—a `fulfill` command that overwrote uncommitted work in five files while printing a green checkmark—was the worst defect we found in the system during this evaluation. The participant's conclusion: "From here I treat `sgt revert` as 'a fast, accurate first pass that leaves me a punch list', not as an operation that completes." When asked at exit whether they would adopt the tool: "For reading history, yes, starting today. For anything that writes, no."

7.3 What the git condition found

The git pilot completed all six tasks inside the time budgets, including request 6 (history edit), which the protocol had expected to be out of reach in the git condition. The participant scripted the interactive rebase, split the tangled commit in two, and verified that both halves pass independently by comparing tree hashes. That result is better than the sgt condition managed on the same task: sgt had already split the same commit at mining time, which is the more impressive half, but the participant could not give the two halves clear names because no verb renames a checkpoint.

Four defects in the study design surfaced:

(1) The git-condition copies carried commits written by sgt (`sgt revert f-10462e17@2`, `sgt undo: restore prior ideal`). A participant in the baseline should not be able to tell that another tool was involved.
(2) A request ticket described a change as "showing times differently" when no earlier format existed. Both pilots spent time on our wording rather than on the task.
(3) Request 6 needs a rubric for both conditions rather than an assumption that the git condition cannot finish.
(4) One ticket referenced a duplicate commit and an undo authored by the researcher.

All four are fixed.

The git participant also described, unprompted, what they wished the history could answer: "Ask history a question about a symbol, not a file. 'What happened to `overlaps`, who calls it, and when did that change?' would have turned request 4 from a hunch into a lookup." A baseline participant independently describing the treatment condition's premise is evidence that the problem this paper addresses exists independently of the tool we built. We added a question to the exit interview to collect this evidence deliberately.

7.4 What the apparatus pilot found

The third pilot ran the full study end to end—real bundles, real terminal, real git and sgt—with the sole purpose of exercising the machinery between the participant's machine and the facilitator's console. It found nine defects, all in the seam between systems. Three generalise.

The primary read verb rejected its own output. `sgt show` failed 6 of the 10 times it was used, and it is the verb the practice sheet tells participants to lean on. It rejected save identifiers printed by `sgt log`, full git commit hashes, and feature names typed exactly as displayed. The cause was narrower than "identifiers are inconsistent": the id column of `sgt log` prints the 7-character git commit hash, and `show` did not accept that format as a selection kind. The participant fell back to `git blame` and `git show --stat`—in the sgt condition—and answered the question that way. A tool that pushes someone back to the baseline mid-task is not a measurement problem; it is the measurement. The defect is fixed: `show` now resolves commit-shaped tokens as saves.

A validity threat: the sgt condition degrades its own baseline. Every commit in the sgt-condition repositories carries its operation identifiers as commit-message trailers. On a representative commit whose actual message is the single line `add course search`, we measured 174 `Sgt-Op:` trailer lines. So `git show`, `git log`, and `git log -p` all bury the real content under a screenful of hex. The git-condition repositories are stripped of these by design, so a participant in the baseline sees clean history while a participant in the treatment sees this. The direction of the bias is the problem: it makes git look worse exactly where sgt is being measured against it, and it does so in the treatment arm—the arm whose participant reaches for `git blame` as a fallback whenever sgt does not have the answer. We disclose this because it favours our condition: a win for sgt on any task where the participant consulted git history is weaker than it appears, and we cannot retrospectively subtract the effect. The pre-registered analysis acknowledges it as a confound.

The instrument broke the thing it measured. The recording tool ran `git add -A --intent-to-add` on the participant's repository to fingerprint the current state. That stages every untracked file—including sgt's own `.sgt/` directory—as index entries pointing at git's empty blob. Where that blob is absent from the object store, every subsequent `git write-tree` fails, and sgt calls `write-tree` whenever it snapshots the repository. Confirmed live: a measurement tool was capable of silently disabling `sgt save` and `sgt revert` for the remainder of a session, and the facilitator's console showed no indication that anything was wrong. The fix includes a regression test that asserts the tool under study can still snapshot the repository after the recorder has run. The structure is the same one the evaluation journal found independently (Section 8): a measurement that modifies its own object—here by staging files that break the tool, there by rebuilding the store that changes its labels—and

a guard on the participant path that catches a state the evaluation path walks into silently.

Carry-over between isomorphic tasks. The two test repositories are isomorphic by design (same episode script, nouns swapped) so nobody sees the same project twice. The pilot participant reported that the resemblance is strong: "by half two I was pattern-matching against an already-solved puzzle rather than reading fresh." The design counterbalances order and models it as a factor; this finding confirms that modelling order is essential rather than a formality, and that the carry-over effect is likely large relative to the condition effect on a twelve-person sample.

7.5 Thematic analysis

We coded all three pilots' observations against the design commitments of Section 4, interpreting each through three lenses from the automation trust literature. Parasuraman et al.'s four-stage model asks which stage failed [102]. Lee and See's trust calibration framework asks what the failure taught the user about the system's reliability [80]. Norman's gulfs of evaluation and execution ask whether the user could perceive the state and act on it [101]. Six themes emerged that generalise past the individual defects.

Comprehension held; mutation did not. The comprehension operations—attribution, revert preview, dependency display, feature listing—produced correct answers on first contact in the task pilot. The sgt participant reached the right answer on request 1 in four commands, using a path the git participant could not take (symbol-level attribution across files). Every write operation—`revert`, `restore`, `save`, `fulfill`—either failed, crashed, corrupted output, or refused to run. The apparatus pilot qualifies this split: the *interface* to the comprehension layer failed at the same rate as the mutation layer (`show` rejected 6 of 10 queries), so a user who could not navigate to the right screen never reached the comprehension that screen provides. The distinction matters because it separates two failure modes with different fixes: the underlying analysis was correct (the attribution existed and could be displayed), but the path to it was blocked by an identifier mismatch between two surfaces of the same tool.

Norman calls these two directions the gulf of evaluation and the gulf of execution: the first asks whether a user can tell what state the system is in, the second whether they can act to change it [101]. Parasuraman, Sheridan, and Wickens decompose automation into four stages—information acquisition, information analysis, decision selection, and action implementation—and argue that automating the earlier stages while leaving the later ones to the human is both safer and more robust than automating uniformly across all four [102]. sgt automates stages one and two heavily (parsing, clustering, dependency tracking, consequence display) and leaves stage three to the developer (which feature to revert, how to resolve a fork). The pilot failure sits squarely at stage four: the action implementation that follows a correct analysis either crashed or corrupted output. The design implication is that the comprehension layer is independently valuable even when the action layer is not yet trustworthy, but only if the developer can reach it—an interface failure at the front door costs the same as a corruption behind it, because in both cases the developer falls back to git.

Silent wrong answers erode trust faster than crashes. The participant encountered both an omission error (`restore`: raw traceback, no data loss, the system failed to act) and a commission error (`revert`: green checkmark, three function headers glued onto one line, the system acted wrongly while reporting success). Mishler and Chen showed that commission errors erode trust in automation faster than omission errors, because a system that does the wrong thing while claiming to have done the right thing attacks the operator's ability to distinguish reliable reports from unreliable ones [92]. That asymmetry appeared here in full: the crash cost one command; the corruption cost twenty minutes of repair work, left the participant unwilling to trust checkmarks, and propagated forward: when the session feature later reported "86 new ops" on a workspace with zero edits, the participant declined to use it. The number was explainable, but the tool had spent its credibility on an earlier false positive.

Lee and See's model predicts exactly this: a crash provides correct calibration information (the system failed and said so), whereas a false positive undermines the calibration mechanism itself, because the user can no longer distinguish a true positive from the next false one [80]. Wang et al. found the same asymmetry in developers using AI code generation: one bad recommendation erases the credibility of three good ones, and a single negative experience with a category of output causes developers to stop reading that category entirely [153]. Madhavan and Wiegmann explain why the recovery never came: people hold automation to a higher initial standard than they hold humans, and a single failure triggers categorical attribution—the system is *broken*—with no intermediate state between working perfectly and being untrustworthy [87]. Sabouri et al. measured the downstream consequence: developers retained only 52% of initially accepted AI suggestions on revisitation, and the retention rate dropped further after encountering a single incorrect suggestion they had initially accepted [116]. Trust is not per-operation but cumulative, and one self-reporting failure costs more than the task it belongs to.

Eight such failures appeared across the full evaluation period, four of which we had not found after eight months of daily use and found only by auditing the evaluation's own results—the newest was a revert that reported "removes 0 edits" and then rewrote two files, a command whose entire pitch is that it shows the developer what will change before changing anything. The eight share a structural pattern: in each case, a named command succeeded (exit code zero, affirmative message) while performing either nothing or something other than what was named. Three were empty successes (a revert that removed nothing, a save that saved nothing, a sync that synced nothing), and five were misdirected successes (a revert that targeted the wrong records, a merge resolution that dropped a fix, a reference warning that fired on the wrong commit). The empty-success variant is harder to detect because the developer has no reason to inspect an operation whose output confirms their expectation; the misdirected variant is harder to recover from because the developer's next actions build on a state they believe they verified.

The pattern is not specific to sgt: any tool whose output is a self-report rather than an independently verifiable artefact is vulnerable. The robustness harness's design (Section 9.1) addresses this class directly, by checking every operation's outcome against a reference

that does not depend on the tool's own state. Without such a probe, the silent-success failure can persist indefinitely, because no test fails, no user complains, and the tool's own diagnostics confirm that the system is working. The phase-one evaluation found two instances that had persisted through eight months of daily use by the developer who wrote the warning paths: the revert-warning code was unreachable (a boolean gate was always false), and reference warnings fired on 7 of 33 operations that should have triggered them (the coverage window stopped at 10 seconds of history on side branches). Both passed every test, because the test suite verified that the warning *code* ran rather than that the warning *fired when it should*.

The tool's value was in the punch list. The participant reframed the revert as "a fast, accurate first pass that leaves me a punch list" rather than an operation that completes on its own. Every item on the removal's consequence report came true: the three functions that still referenced removed code broke, and nothing else did. The preview was the product; the execution was the problem. The exit debrief sharpened this: "It indexes your history by function instead of by file, so you can ask 'what changed this symbol, and what else came along for the ride' and get a real answer in one command—that part is excellent. Use it to find things, then make the change yourself with git."

Ferdowsi et al. found that continuously displaying runtime values mitigates both over-reliance and under-reliance on AI-generated code, by lowering the cost of validation through execution [42]. The consequence preview serves the same function: it shows what a revert would break before executing it. The pilot confirms both halves of Ferdowsi's finding. Where the preview was correct, the participant relied on the tool appropriately. Where it was incorrect (the green checkmark over corruption), the lowered validation cost produced worse outcomes than no tool at all, because the participant trusted the preview and did not verify independently. Cynthia et al. found the same asymmetry at scale in 54,791 agent-generated code review comments: the strongest predictor of a comment being acted on was a concise inline suggestion, while lengthy explanations went unresolved regardless of correctness [30]. The consequence preview is the inline suggestion of version control—short, checkable, and wrong in a way the developer can see—and its effectiveness depends on staying a list rather than becoming a paragraph.

Bansal et al. found the mechanism that explains both halves: AI explanations increase human acceptance of the system's recommendation regardless of whether the recommendation is correct [6]. The explanation does not help the human distinguish right from wrong; it makes the recommendation *feel* more trustworthy whether or not it is. A consequence preview is an explanation in Bansal et al.'s sense—it provides a reason to accept the revert—and the pilot shows the predicted failure mode: the participant accepted the revert *because* the preview named consequences, without independently checking whether the named consequences were complete. The distinction between a preview that lists what will break (which the developer can verify cheaply by scanning the list) and a preview that claims nothing else will break (which the developer can verify only by reading the entire diff) is the distinction between an explanation that supports appropriate reliance and one that manufactures inappropriate reliance. The pilot's corruption case was the second kind: the checkmark asserted completeness, and the participant had no path to discovering the assertion was wrong without doing the full inspection the tool promised to save them.

Kaur et al. found the same structure in a closer domain: data scientists using interpretability tools over-trusted and misused them, and few could accurately describe what the visualizations actually showed [70]. The tools were correct; the failure was that users could not distinguish what the output *claimed* from what it *demonstrated*. A SHAP plot shows feature attributions for one prediction; users read it as showing feature importance globally. A consequence preview shows which functions will change; a user reads it as showing which functions *could* change. The gap between the claim and the reading is where over-reliance enters, and it enters through expertise rather than around it—Kaur's participants were practicing data scientists, not novices.

A partially-working tool can deliver value through its read layer alone—it tells the developer what to change, and they change it with the tools they already trust—provided the reports list what they found without asserting completeness. The punch-list framing the participant independently arrived at ("leaves me a punch list") is the correct interface contract: the tool reports what it knows, the developer treats the report as a lower bound, and the absence of an item from the list means "not found" rather than "verified absent." The accuracy condition circles back to the previous theme: a tool whose punch lists are wrong on one task stops being consulted on the next.

Vocabulary mismatch across surfaces. Four counting units—edits, ops, symbols, saves—appeared in different outputs with no definitions, and the participant stopped reading the numbers halfway through: "the edit counts have been meaningless all session." Two views of the same feature reported different symbol counts (the tree counted cluster members; `show` counted footprint symbols). Selector namespaces varied per verb, with no cross-lookup. Each of these is a consistency defect, not a missing feature, and each one makes the tool harder to learn because the participant cannot build a stable model of what the numbers mean. The fix was mechanical: one counting unit (symbols) and one namespace across all verbs. Carroll and Rosson named the mechanism that makes this costly: active users learn by generalising from one interaction to the next, and inconsistent surfaces punish generalisation by teaching a rule that fails on the second screen [19]. Jackson frames the same requirement as a property of the concepts a system exposes: a concept whose state looks different depending on which operation queries it has no stable operational principle [66]. The problem compounds in a tool whose labels are already inferred: Nguyen and Glassman showed that vocabulary mismatch between a developer and a model is the norm rather than the exception [100], so a developer already arrives at the tool uncertain of the code's naming; a tool that then disagrees with itself across its own screens removes the one stable reference point (the tool's own output) that could have resolved that uncertainty. The lesson is that internal consistency is a learnability precondition (not merely cosmetic), and a tool whose surfaces disagree teaches the user to ignore its numbers.

Baseline sufficiency reframes the study. The git condition completed every task, including the one we had expected to be out of reach (splitting a tangled commit via scripted interactive rebase). The study therefore measures cost and confidence rather than raw capability: both conditions can finish, and the question is how many commands, how many errors, and how much the developer trusts the result. That is a stronger design than one where the baseline cannot finish, because it makes both directions of the comparison informative—the git condition can win on tasks where the tool's mutation failures cost more time than the workaround saves. The reframing also constrains what counts as evidence for the design. A tool that enables tasks the baseline cannot complete proves its necessity by existence; a tool that does the same tasks faster or with fewer errors proves its value by degree, which requires both more participants and a more precise measure. The first row of Table 1 (undetected errors) is the measure that survives this reframing, because an error the developer did not notice is a cost neither condition can address through more effort—it is a structural gap in what the developer saw, and the gap either widens or narrows when the tool changes what is visible. The pilot's git participant independently described wanting exactly what the sgt condition provides ("ask history a question about a symbol, not a file"), which confirms that the operation is desired and the question is only whether the implementation delivers it at acceptable cost.

The treatment degrades its own fallback. The apparatus pilot found that the sgt condition makes plain git—the tool a participant reaches for when sgt fails—measurably worse than in the git condition. Every sgt-authored commit carries operation identifiers as commit-message trailers (174 lines on a one-line commit), so `git show` and `git log` bury the content a developer needs under hex they cannot use. The git-condition repositories are stripped of these by design, so the baseline sees clean history while the treatment does not. This creates a confound that favours our condition: any comprehension task where the participant consulted git history in the sgt arm was harder than the identical task in the git arm, for a reason unrelated to sgt's attribution. The measurement lesson is that a tool layered on top of a host system (here, git) must account for how it changes that host's usability, because a participant evaluating the treatment is also evaluating a degraded version of the control. We acknowledge this as a threat and note that the pre-registered analysis cannot correct for it post hoc.

The six themes, read together, describe a tool whose analytical core is sound and whose interface to the developer is not yet stable enough to carry the analysis through to action. The first three themes (comprehension/mutation split, trust erosion, punch-list reframing) all resolve in the same direction: the developer can use this tool as a comprehension instrument today, and should not trust it as an execution instrument yet. The fourth (vocabulary mismatch) explains why even the comprehension layer requires polish before study: a developer who cannot navigate to the right screen never reaches the analysis that screen provides, and inconsistency between screens teaches them to stop reading. The fifth (baseline sufficiency) constrains the study design: both conditions finish, so the measurement must be about confidence and error rate rather than completion. The sixth (treatment degrades control) constrains the interpretation: any advantage we observe in the treatment arm is weakened by a confound we cannot remove.

A seventh observation sits across the six themes rather than within any of them: within each pilot session the participant's willingness to issue comprehension commands stayed flat or grew, while their willingness to issue mutation commands declined monotonically after the first silent failure. The two curves are independent because the comprehension layer never committed an error that the participant saw—its failures were refusals (a query syntax that was rejected) or confusions (which count unit does this report use?), both of which teach the developer to rephrase rather than to stop asking. Mutation failures, by contrast, violated an already-verified expectation: a preview said one thing and the execution did another, or a report said nothing would happen and something did. Each such violation moved the participant further from the tool's write path and closer to the fallback ("I'll do it in git"), without moving them further from the tool's read path. The implication for sequential deployment is that the two layers can be released on different schedules: a team that ships the read layer first builds familiarity, and a write layer released later enters a context where the developer already knows what the tool claims and can tell when the claim is wrong. Releasing both simultaneously means a mutation failure on day one poisons a comprehension layer the developer would have found trustworthy on its own, because the developer attributes the failure to the system rather than to its action subsystem.

Reflecting back through the four design commitments of Section 4, the pilots identify which commitment caused each class of failure. The comprehension successes trace to commitment 1 (the function as unit) and commitment 4 (inferred names): parsing succeeded, grouping was reachable, and the vocabulary was the developer's. The mutation failures trace to commitment 2 (set-based versions, whose layout seam produced the corruption) and commitment 3 (the fork rule, whose amplification blocked the write guards). The counterfactual is sharp: had we shipped with a weaker fork rule—merging two edits to one function textually whenever they touch no common line, the way Apel et al.'s semistructured merge does [2]—the write layer would have worked on repositories it currently refuses. The tradeoff would then be silent merge errors instead of refusal-to-act, which is Cavalcanti et al.'s finding stated as a design question: semi-structured merge reduces false positives (spurious conflicts) but introduces false negatives (missed integration problems) [22]. The fork rule is the maximally conservative position on that spectrum, and the pilot shows the cost of conservatism: the developer who is stopped every time eventually stops using the tool. The study is designed to answer whether the alternative—a textual merge inside a function that sometimes produces wrong code nobody notices—is actually the worse outcome, because that is the claim this paper makes and the pilot cannot settle.

The twenty-one fixes and four design corrections together mean that the study differs substantially from the study we would have run two weeks earlier. Three consequences follow for its design. First, the primary dependent variable is now time and error count per task rather than task completion, since both conditions finish. Second, the exit interview asks whether the participant trusts the tool's reports, because the pilot showed that credibility, once lost

to a false positive, does not recover within a session. Third, the sgt condition's tasks are scored on the comprehension half (did the participant identify the right target) separately from the mutation half (did the tool execute correctly), so that a mutation failure in the tool does not mask a comprehension success in the participant. The result that would count against the design is error-rate parity between the two conditions. If both conditions produce equal rates of undetected errors, then the recovery this paper describes is something developers already do reliably without the tool, and Section 3 overstates the difficulty. We report the pilots rather than only the findings, because we want a reader to know which version of the tool the full study runs against and why the study measures what it measures.

7.6 Study repository validation

We built two 5,000-line Python projects (coursecraft and confplan) using scripted agent sessions—28 and 26 commits, 23 episodes each—so that the ground truth for every task is known from the build log. Running the derivability census on both repositories before the pilot produced one pass and one failure: confplan's central removal task ("take the promotion system out") was not answerable from the record, because the promotion subsystem sat inside a 40-symbol feature that also held the entire CLI. The same episode script, the same 23 episodes, the same task—answerable on one repository and not the other.

Tracing the cause exposed a single defect with four visible consequences: the largest feature was named *init repo* (a label derived from one seed-commit vote in a 71-op leaf), 77–84% of commits had edits filed under that seed feature, eight features carried file-path labels, and confplan's promotion subsystem was absorbed into one cluster. The mechanism was provenance blindness on the save path: sgt's two clustering signals read each operation's provenance field to learn "these symbols were written in the same sitting," and every operation saved through `sgt save` carried no provenance—the commit reference lives in trailers that three call sites did not resolve. 366 of 370 operations, unattributed. Co-change without temporal separation is co-occurrence, which produces one cluster.

The finding that generalises beyond our system: the blindness appeared only in histories built *through* sgt. Every unit test, every dogfood run on the development repository, and every retroactively-mined fixture had provenance, because all of those derive their temporal signal from git commits that already exist. The save path—the one workflow we ask participants to use—was the least-tested path in the codebase, and a tool evaluated only against mined history has been evaluated on its strongest arm.

After three fixes—provenance resolution, label prompt filtering, and an aliasing-rename crash—both repositories pass an 8-of-8 derivability gate: the answer to each of the study's four requests is reachable through the taught verb set (`log`, `show`) in at most three commands. We score this as "8/8 by the author's reading": the pre-registered primary metric could not be computed as written, so a referee is entitled to treat the replacement as weaker evidence.

Four tangle cases—commits the build log placed two unrelated changes into—are all separated correctly: in coursecraft, `cli.py::cmd_search` lands in one feature while `slots.py::parse_slot` and its test land in another, and the two concerns the commit message conflates are two features in the record. Reconstruction is exact on both repositories (0 drifted paths), so nothing above came at the cost of fidelity.

One asymmetry remains: coursecraft's waitlist feature contains five enrollment symbols (a hub function's edit chain pulls them in), so a feature-level revert would violate the request to "keep enrolling working." The member list is visible before applying, which makes this a legibility cost rather than a silent failure, and it is the cost we report to participants in the task handout rather than one we hide. On mirrored histories the clustering diverges—the same episode script produces 2 separable features in coursecraft and 3 in confplan—which is itself a finding about the method's stability under near-identical inputs.

8 What the Study Found

Twelve developers completed both conditions. This section reports what they did, what they got right, and what they said—in that order, because the primary measure (Section 6.3) is errors a participant does not notice, and everything else is context for interpreting that number. A tool that helps a developer finish faster while leaving hidden damage is worse than one that slows them down and catches it. The developer who submits with confidence has spent their verification budget and will not look again.

Mozannar et al. modelled this budget formally: the dominant cost in AI-assisted programming is not generation but verification—a developer spends more time evaluating whether generated code is correct than they saved by not writing it [94]. The cost scales with the developer's existing comprehension of the surrounding code, which is exactly the variable our two conditions manipulate. In the git condition, the developer's comprehension of which functions serve which request must be constructed from reading diffs. In the sgt condition, the tool's attribution supplies that comprehension directly, so the verification cost of a revert preview should be lower—if the attribution is correct. If it is wrong, the developer inherits a false model and the verification cost rises above the git baseline, because they must now correct both the code and their understanding of it. Parasuraman et al.'s model predicts this asymmetry—automating information analysis (stage 2) without adequate information acquisition (stage 1) produces exactly the undetected error [102]—and the study's primary measure is designed to catch it: an error that survives the participant's verification is evidence that the budget was spent on the wrong model.

We pre-registered the measures and the analysis before the first session; the predictions stated in Table 1 were fixed before the data existed, and the reader can check each one against the result below.

All confidence intervals are 95% studentized bootstrap over participants (10,000 resamples), because at $n = 12$ the sampling distribution of the mean is not guaranteed to be normal and a t-interval would require that assumption. We report paired estimates (sgt minus git) throughout, so each participant serves as their own control and individual variation cancels.

8.1 Participants

We recruited twelve developers who use a coding agent as part of their ordinary work. [Background: mean years coding, mean years git,

git expertise composite (0–24), agent tools used, AI share of shipped code.] Four groups of three, fully crossing condition order with project assignment (coursecraft and confplan). No participant had used sgt before the session.

8.2 Task outcomes

Figure 6 shows the three primary outcome measures, paired by participant. The measure the paper committed to first is collateral damage: tests broken outside the set the participant was asked to remove.

Provenance (request 1). Three closed questions, scored against a key. [Git mean, sgt mean, paired difference, CI, d_z, n.] The confidence calibration (stated confidence minus proportion correct) was [git calibration, sgt calibration, difference] — positive is overconfident. [Interpret: did sgt participants know more, or merely believe they knew more? If calibration is flat while score rises, the tool helped them be right. If calibration also shifts, they were also more sure of what they got right — or more sure of what they got wrong.]

Reach prediction (Foresee block). Two trials, each scored as F1 against measured ground truth (call-graph reach). [Per-trial blind, checked, gain means by condition. Paired gain difference (sgt minus git), CI. Confidence calibration at both stages.]
Trial F1 (agenda export, narrow reach): [blind, checked, gain per condition.] Trial F2 (slot-comparison normalisation, broad reach): [blind, checked, gain per condition.]

[Interpret: (1) Is gain positive under sgt? If so, the representation moved the participant's prediction toward truth. (2) Is gain larger under sgt than under git? That is the between-condition test. (3) Did any participant show high blind confidence paired with low blind accuracy under sgt? That is the inferred-label failure mode—confidently wrong rather than helpfully wrong—and if it appears on F2 (the pre-registered risk case), it is reported as a U1 failure. (4) Does the narrow-reach trial (F1) produce different gain than the broad-reach trial (F2)? If gain is asymmetric, the representation helps in one direction (pulling predictions down or pushing them out) but not the other, which says something specific about which kind of provenance question the tool serves.]

Removal and correction (requests 2 and 3). Task score (rubric points): [git mean, sgt mean, difference, CI]. Collateral damage (failing tests outside the target): [git mean, sgt mean, difference, CI]. [Interpret: the critical question is whether the sgt condition's consequence preview let participants avoid the collateral that the git condition could not see coming. If collateral is equal, recovery is something developers already do reliably and Section 3 overstates the difficulty.]

Time and cap pressure. [Proportion hitting the cap per condition. Mean active time. Manipulation check: felt time pressure response distribution per condition.] We report time because a reader would want it, and we would not publish a time difference alone: a tool that is correct and unbearably slow is not adopted, but a tool that is fast and wrong is worse.

8.3 How the work was done

Outcome measures say whether a tool helped; process measures say how. Two conditions can produce the same error rate through different strategies—one because the developer checked carefully, the other because the tool caught things the developer missed—and the distinction matters for design. The process telemetry also grounds the preference data: a developer who says "I trusted it" while the telemetry shows frequent verification is better calibrated than one whose telemetry confirms the trust.

The process telemetry recorded every command, prompt, and edit action, categorised into nine kinds (orient, inspect, search, prompt, agent edit, manual edit, history operation, verify, recover). The first three are navigation (the developer is looking for something); the next three are transformation (the developer or assistant is changing something); the last three are quality assurance (the developer is checking or undoing). Figure 7a shows how those three phases distribute over normalised request time.

Verification ratio. [Git mean, sgt mean, difference, CI.] The verification ratio is the proportion of actions in the "verify" category (running tests, reading diffs, inspecting output) relative to all actions within a request. The direction of this measure is ambiguous by design. A higher ratio in sgt could mean the preview gave participants something concrete to check (appropriate reliance), or it could mean the tool's unfamiliarity forced more checking (friction). A lower ratio could mean participants trusted the consequence display and skipped verification—efficient trust, or dangerous complacency. The self-report item q14 (accepted unreviewed work, reversed; Section 8.4) disambiguates: if verification drops AND q14 rises, participants skipped checks they felt they should have made. [Report the numbers and which disambiguation holds.]

Prompt specificity. [Git mean (0–3), sgt mean, difference, CI.] Each prompt to the AI assistant was coded on a four-point specificity scale: 0 (no target named) through 3 (names a file, a function, and what to do). The coding was applied to the raw prompt text by the taxonomy of Section 6.3, not by a human judge, so inter-rater reliability is not a concern and every prompt across both conditions received the same treatment. The measure tests whether the tool's vocabulary—feature names, checkpoint labels, symbol identifiers visible in the map—entered the developer's prompts. Two opposing mechanisms are plausible. If sgt's names gave participants concrete referents they otherwise lacked, specificity should rise: a developer who can see that the waitlist is feature `f-7ab21c` writes "revert f-7ab21c" (level 3) instead of "undo the waitlist stuff" (level 1). If the tool substituted for the developer's own precision—they delegated navigation to sgt and therefore never acquired the vocabulary themselves—specificity should fall, which is the scaffolding-removal concern Kazemitabaar et al. measured in learners [71]. The distinction matters for the design's long-term effect. A tool that raises specificity is augmenting the developer's coordination with the assistant. A tool that lowers it is creating a dependency the developer cannot sustain when the tool is absent. [Report which direction holds and connect to the vocabulary theme of Section 8.6.]

Wrong turns. [Git sum, sgt sum, per-condition.] A wrong turn is a history operation followed by a recovery action (undo, restore, or manual repair) within two minutes—the participant tried something, saw it was wrong, and backed out. The direction of this measure is the sharpest test of whether the pilot's mutation failures persist in the fixed version. If wrong turns are higher in sgt, participants reached the write layer and found it still unreliable:

[Figure: PairedEstimation with panels R1 provenance (0–3), R2+R3 removal (0–4 rubric points), and collateral damage (inverted axis). Lines connect each participant's git and sgt scores. Thick bars are condition means. Below each panel, the paired mean difference with its bootstrap distribution and 95% interval.]

Figure 6: Task outcomes by participant and condition. Every participant appears twice, joined by a line. Below each panel, the paired mean difference (sgt minus git) with its bootstrap distribution and 95% studentized interval. Collateral damage is plotted with the axis inverted, so up is better in all three panels.

[Figure: ProcessSignature with (a) share of action categories across normalized request time, pooled per condition, with one strip per participant beneath; (b) action bigrams that most distinguish the conditions, by weighted log-odds with informative Dirichlet prior.]

Figure 7: (a) Share of action categories across normalised request time, pooled per condition, with one strip per participant beneath. (b) Action bigrams that most distinguish the conditions (weighted log-odds, informative Dirichlet prior; positive leans sgt).

the tool stopped them (good) but stopping is work, and enough stops constitute a wrong-turn tax that erodes the time the comprehension layer saved. If wrong turns are lower in sgt, participants either avoided the write layer entirely (using sgt only to read, then acting in git) or the fixes actually stabilised it. The telemetry distinguishes these two explanations: an absent write layer shows up as zero history_op events of the mutating kind, while a stable write layer shows up as history_op events without recovery following them. [Report numbers and which explanation holds.] In Ferino et al.'s reliance-control framework [43], a wrong turn followed by recovery is a developer exercising control at the appropriate moment—they detected a problem and corrected it. A wrong turn followed by abandonment (switching to git for the remainder) is control escalation: the developer decided the tool's level of reliability does not merit continued engagement. The telemetry distinguishes these, and the second pattern is the one that would count against the design, because a tool that drives developers away from its write layer on first contact has failed to build the reliance its read layer earned.

Distinguishing sequences. [Top 3–5 bigrams that lean sgt, top 3–5 that lean git, with log-odds and count. Interpret each: what does the sequence mean as behaviour? For example, if "history_op → verify" is strongly sgt, participants checked what the tool did. If "search → search" is strongly git, participants were hunting through files.]

The bigram comparison uses weighted log-odds with an informative Dirichlet prior (Monroe et al. [93]), which controls for frequency: a rare bigram that appears only in one condition produces a high ratio but a low weight, preventing one participant's idiosyncratic strategy from dominating the comparison. A positive z-score leans sgt; negative leans git. We report bigrams above a minimum corpus frequency of five, and state how many rare sequences were dropped, because silent truncation reads as "we looked at everything" when we did not.

In information foraging terms [107], the two conditions differ in how rich each information "patch" is. Feature names and consequence previews are scent: proximal cues that predict what an operation will reveal before the developer commits to it. The n-gram analysis tests this directly. If the sgt condition shows fewer search→search cycles and more history_op→verify sequences, the tool enriched the history record as a foraging patch: developers spend less time in between-patch navigation and more in within-patch exploitation. Ko et al. measured that navigation accounts for 35% of maintenance time [77], so reducing it is not a minor convenience but a structural change in how the developer spends their session.

Two further patterns would be informative. First, inspect→history_op in sgt would mean that seeing the record's structure gives participants enough confidence to act without further search—the attribution serves as Pirolli and Card's "information scent," a proximal cue of distal content. Second, history_op→recover in sgt would mean write operations failed often enough that participants had to undo them—the mutation layer's residual instability showing up as a behavioural signature. If both conditions show the same cycling, feature names carry no more scent than commit messages and the vocabulary is not reaching the developer's search behaviour.

Letovsky's comprehension model offers a finer prediction [81]. Programmers reading unfamiliar code generate questions ("why is this here?", "what does this call?") and conjectures ("I think this handles retries"), and the questions drive their navigation. In the git condition, the provenance question—*which request produced this function?*—has no answer in the record, so the developer must generate it as a conjecture, test it against the code, and revise. In the sgt condition, the attribution answers it directly: the developer reads the feature label, confirms or renames, and moves to the next question. The observable difference is a shift from *generative* inquiry (conjectures, with their risk of wrong turns) to *verificatory* inquiry (checking a provided answer). The telemetry can distinguish these: generative inquiry shows up as search→search→inspect sequences where the developer builds understanding incrementally; verificatory inquiry shows up as inspect→history_op, where a single inspection provides enough confidence to act. If the sgt condition shifts the balance toward verification while maintaining accuracy, the tool's attribution is doing what Letovsky's model says a programmer's own memory does when reading code they wrote themselves: supplying answers to provenance questions before the search has to generate them.

8.4 What the two conditions felt like

The HLAC battery (History Legibility and Agent Collaboration) asked fourteen 7-point items after each half, grouped into four blocks. No validated scale exists for history comprehension in

[Figure: LikertDiverging showing 14 HLAC items grouped in 4 blocks (Finding your way around, Changing things, What you came away with, Working with the assistant), one panel per condition, with paired mean differences and CIs as dots and bars.]

Figure 8: Perceived experience on fourteen 7-point items (HLAC battery), grouped into four ad-hoc blocks. Reverse-coded items (marked ⟷) are recoded so agreement always means better. Dots are paired mean differences (sgt minus git); bars are 95% bootstrap intervals.

AI-assisted development. General trust-in-automation instruments [68] measure disposition toward automated systems but cannot distinguish provenance legibility from consequence awareness from retained mental model, which are the three mechanisms this paper claims. We therefore wrote items that map one-to-one to specific design claims, report them as ad-hoc Likert-type items rather than construct scores, and ground interpretation in the exit interview where participants state what they experienced in their own terms. Figure 8 shows the full response distribution and the paired difference per item.

Finding your way around (C1). Four items: found when it changed, found why it changed, saw the whole piece of work, guessed at names (reversed). [Item-level means and paired differences. Interpret: these map directly to claim C1. If sgt participants found provenance more legible, these items should shift. The reversed item (guessing at names) tests whether inferred labels helped or confused.]

Changing things (C2). Four items: knew what else it would touch, removed a feature safely, recovered from mistakes, surprised by the result (reversed). These four map to the consequence-preview mechanism of Section 4: the tool shows, before applying a revert, which other functions still depend on something being removed. That display is the bridge between comprehension (seeing what would break) and action (deciding to proceed). If participants in the sgt condition rated "knew what else it would touch" higher, the preview reached them—they read the consequence list and it calibrated their expectation. The reversed item (q12, "surprised by the result") tests whether that calibration held through execution: a participant can report knowing what would happen and still be surprised if the tool's execution differed from its preview. [Item-level means and paired differences. If collateral is lower AND q4 (knew consequences) shifts, participants both did better and knew they would—the preview set accurate expectations AND the execution honoured them. If q4 shifts but collateral is flat, participants felt more informed without producing fewer errors, which is Bansal et al.'s finding restated: explanations increase confidence without increasing accuracy [6].]

What you came away with (C3). Two items: clear picture of the project (q7), would get back up to speed (q13). This is the weakest block in the battery and we state why rather than papering over it. Two items cannot establish a construct. They are here because the claim they probe—that the tool leaves the developer with a transferable mental model of the codebase—is central to the paper's argument but too diffuse to measure with four items in a two-hour session. A developer's "picture of the project" accumulates across days, and what two hours can measure is at best whether the tool planted the seed of one. The study design compensates by grounding whatever signal these items produce in the exit interview, where a participant can say what they came away with in their own terms rather than rating an abstraction. [Item-level means and paired differences. If both items shift toward sgt, the tool's attribution gave participants more than task-local comprehension—it left them with a model of the project's structure they believe they could return to. If neither shifts, the twenty-minute window was too short for the seed to take root, and a longitudinal deployment study would be needed to test C3. Report honestly either way: a null on two items in twenty minutes does not refute the claim, but neither does it support it, and the reader should know that.]

Working with the assistant (Q4). Four items: directed the assistant precisely, checked the assistant's work, accepted unreviewed work (reversed), fought the tool (reversed). The two reversed items serve as honesty valves: a participant who straight-lines "agree" on every positive item is caught by the negative ones. But they also measure real phenomena. "Accepted unreviewed work" (q14) is the over-reliance signal: a developer who trusts the tool's consequence preview and skips independent verification has delegated the check to the tool, and the tool's guarantee is only as strong as its rebuild fidelity (Section 6.1). If sgt wins on comprehension items (C1, C2) while q14 also rises in sgt, the tool improved understanding *and* reduced the verification that keeps understanding honest—which is Bansal et al.'s finding that explanations increase acceptance regardless of correctness [6]. "Fought the tool" (q10) is the interface-friction signal. The pilot's vocabulary-mismatch theme predicted that even a fixed tool would impose a learning cost—the concepts are unfamiliar, the operations are new, and twenty minutes is not enough to build fluency. If q10 is higher in sgt while outcome measures also favour sgt, the tool is useful but abrasive, and a developer who adopted it would pay a front-loaded cost the study's two-hour window cannot observe amortising. [Report item-level means and paired differences. State which pattern holds.]

Workload and usability. NASA-TLX (raw, unweighted): [git mean, sgt mean, difference, CI.] UMUX-Lite (0–100): [git mean, sgt mean, difference, CI.] TLX measures what the task cost; UMUX-Lite measures whether the developer thought the setup met their requirements. The two instruments answer different questions and can point in opposite directions. A tool that is useful but demanding (higher UMUX-Lite, equal or higher TLX) is a tool worth learning: it delivers capability that justifies its cognitive cost, and the cost may amortise with fluency. A tool that is easy but insufficient (lower TLX, lower UMUX-Lite) failed to provide what the developer needed, regardless of how little it asked. The combination the pilot would predict—based on the vocabulary-mismatch finding and the mutation failures—is higher TLX in sgt (the unfamiliar interface costs effort) with higher UMUX-Lite in sgt (the consequence preview and attribution answer questions git cannot). If TLX is flat while UMUX-Lite rises, the tool did not impose measurable additional load—which would mean the learning cost the pilot predicted did not materialise in a twenty-minute window, either because the fixes removed the friction or because the task was short enough to stay

below the threshold where unfamiliarity binds. [Report numbers and which pattern holds.]

8.5 Preference

After both halves, each participant compared the two setups on seven job-level items (five-point scale, symmetric, neither tool named) and two hypothetical scenarios. The seven items span a gradient from low-stakes contexts (a throwaway prototype) to high-stakes ones (the participant's own production codebase), so a participant who differentiates across contexts—preferring sgt for their real code but indifferent on a throwaway—is telling us that the tool's value scales with the cost of getting things wrong, which is the claim Section 3 makes. A participant who straight-lines the same answer across all seven items is either genuinely undifferentiated (both tools felt the same) or not engaging with the gradient, and the two hypothetical scenarios ("a codebase you inherited from a colleague" and "a fresh project you are starting alone") separate the two: the inherited case is the paper's scenario and the solo case is the control where the developer holds the decomposition themselves. [Report per item: distribution, recoded mean. Do participants differentiate across the gradient? If so, where does the preference cross zero—at what level of stakes does the tool's cost (learning, friction) start being justified by its value (comprehension, fewer errors)?]

The "what made the difference" multi-select gives the reasons in the participants' own language. The "what would put you off" multi-select collects the price in the same terms. These two together answer a question no Likert item can: whether the developer's stated reason for preferring or rejecting the tool matches the mechanism the design predicted. If the reasons cluster around comprehension ("I could see what belonged together," "the consequence preview helped") the read layer is carrying the value. If they cluster around mutation ("I could take things out cleanly") the write layer is carrying it despite the pilot's predictions. If the costs cluster around trust ("not sure what it had done") the pilot's strongest finding survived the fixes. [Report both distributions. Map each selected reason back to the design commitment it reflects.]

The preference data answers a question the outcome data cannot: whether a tool that helps is also a tool a developer would install. Lee and See's framework predicts that the two can diverge—a tool whose occasional failures are dramatic enough to remember can be distrusted even when its aggregate performance is positive, because the availability heuristic weights vivid single events over base rates [80]. Two items in the cost multi-select test this directly: "not being sure what it had done" (trust erosion surviving from the pilot's strongest finding) and "it grouped or named things in ways I disagreed with" (commitment 4, inferred names, costing more than the pilots predicted). If either appears at high frequency while the outcome measures favour sgt, the tool is useful and unwanted. Buçinca et al. found the same pattern for the most effective cognitive forcing functions [16]: the intervention that helps most is also the one developers resist, because its cost is experienced while its benefit is invisible.

Exit interview: thematic analysis. After the preference items, each participant answered three open questions: what they noticed about how the two setups differed, what they would want changed in the setup they preferred less, and whether they thought about the code differently after the session than before it. The analysis follows Braun and Clarke's reflexive thematic analysis [14]: transcripts are coded inductively, codes are grouped into themes, and themes are checked against the design commitments to see whether the participant's account of their experience maps onto the mechanism the design predicted or onto a mechanism the design did not anticipate.

Four a priori themes derive from the four commitments: (1) *granularity mismatch*, where the participant describes the tool's units as too fine, too coarse, or misaligned with what they wanted to name; (2) *consequence awareness*, where the participant describes knowing (or failing to know) what an operation would affect before it ran; (3) *trust trajectory*, where the participant describes a moment that changed how much they believed the tool's reports; and (4) *vocabulary adoption*, where the participant uses the tool's names (feature labels, checkpoint names, symbol identifiers) in their own description of the codebase rather than speaking in file or function terms. A fifth theme is reserved for mechanisms the design did not predict: any coherent account of value or cost that does not map onto the four commitments is coded separately and reported as a finding about the design space rather than a finding about this system. [Report: which themes appeared, how many participants generated each, and the one or two quotations that most sharply illustrate the mechanism. Map each theme back to its design commitment. If theme 5 appears with more than two participants, it is the finding that matters most, because it means the developer's experience of the tool is shaped by something the designer did not control.]

8.6 Reflecting on the design commitments

The pilots predicted a specific shape: comprehension holds, mutation fails (Section 7.5). Twenty-one fixes and four design corrections separated the pilot version from the version participants used. The question this subsection answers is whether those fixes changed the shape or merely moved it.

Did the read/write split survive? The pilot participant's exit verdict was "for reading history, yes, starting today; for anything that writes, no." The study's R1 (provenance) is a pure read task and R2+R3 (removal/correction) require writing. A split that survives twelve participants is a finding about the design rather than about one person's session: it says that the representation (edits filed under requests) is independently valuable even when the operations over it (revert, restore) are not yet trustworthy, because the representation answers questions git cannot express—*which request produced this function, and what else came with it*—without requiring the developer to act on the answer through any particular tool. A split that closes (R2+R3 also improved) is a different finding: the pilot's mutation failures were implementation defects rather than design costs, and fixing them unlocked the operations the representation makes possible. [If R1 shows a clear advantage and R2+R3 does not, the split survived the fixes. If R2+R3 also improved, the fixes changed the story — the mutation layer's defects were the bottleneck, not the design. If neither improved, the pilot's read benefit was an artefact of one participant rather than a property of the representation.]

Did trust recover? The pilot's strongest finding was that a single false positive (green checkmark over corruption) eroded trust for the remainder of the session, and the trust did not recover within

that session even after correct operations followed. Lee and See's framework explains why: trust calibration requires multiple consistent observations to revise, and a single session provides too few trials after a failure for recalibration to occur [80]. The version participants used fixed all twelve defects the pilot found, so the question shifts from "can trust recover within a session" to "does a tool that never produces the failure build trust from scratch in the time available." The HLAC item "surprised by the result" (q12, reversed) and the preference item "not being sure what it had done" are the signals. These two measure different things: q12 asks whether the tool's behaviour matched expectations (a property of the current session), while the cost item asks whether the developer would carry uncertainty forward (a property of anticipated future sessions). A tool can score well on q12 (no surprises today) while still triggering the cost item (I would not trust it tomorrow), and that combination would mean the pilot's trust damage left a residue in the design's reputation that the fixes did not fully erase. [If q12 is flat, participants in the fixed version were not surprised. If "not being sure" still appears frequently in the cost multi-select, trust did not fully recover despite the fixes — meaning the tool's legibility gap is structural (commitment 4, inferred names) rather than a bug in the mutation layer.]

Did the vocabulary reach the developer? The pilot's vocabulary-mismatch theme (four counting units, selector namespaces varying per verb) predicted that even the comprehension layer would be hard to reach. The study version unified to one counting unit (symbols) and one selector namespace. The HLAC item "guessed at names" (q11, reversed) and the process measure "prompt specificity" are the signals.

This question matters beyond usability. A version control system that infers feature names is proposing a shared vocabulary for a codebase—the same function Furnas et al. showed designers cannot predict for their users [45]. If a developer adopts the tool's names into their prompts, those names become the vocabulary through which they and the assistant coordinate about the code. The tool has then changed how the developer talks about what they are doing, not merely how fast they do it.

[If q11 improved and prompt specificity is higher in sgt, the tool's vocabulary entered the developer's prompts — they used feature names to direct the assistant. If specificity is lower, the tool substituted for the developer's precision rather than augmenting it, which is the over-reliance concern of Bansal et al. [6].]

The vocabulary question also tests a specific design bet about the over-decomposition. The tool presents four to six structural communities where the developer expects one named request, and the design bets that developers will adopt the finer vocabulary for the same reason they adopt file names: a structural boundary that stays stable across sessions is more useful as a referent than a purpose boundary that shifts with rephrasing. If the vocabulary enters prompts, developers found the subordinate-level labels ("Rate Limiting," "Price Cache," "Row Accessors") useful as handles for directing the assistant, even though none of them names the request they typed. If it does not, the over-decomposition is finer than the developer wants to *talk about*, and a vocabulary nobody adopts fails Clark's grounding test [27], regardless of whether it is structurally correct. The pilot's git participant provides the sharpest test: they asked for "a question about a symbol, not a file," which is the structural level, not the request level. If study participants in the sgt condition adopt feature names at comparable frequency, the design bet is confirmed: developers prefer structural referents when they are available, and the mismatch between tool grain and intent grain matters less than whether the tool's vocabulary is learnable.

What the preparation found. Building the study repositories and running three pilots produced thirty-five defects, and the pattern across them is informative independent of their individual fixes. Twenty-two entered through the write path (revert, restore, save, fulfill) and zero through the read path (log, show, map, find). Twelve of the twenty-two shared a structural cause: a mutation that removes or rearranges records leaves metadata (layout chains, anchor positions, residue sentinels) pointing at records that are no longer present, and the rebuild produces invalid output from valid arithmetic. This is commitment 2 (set-based versions) failing at the seam it was designed to hide: files are rebuilt from sets of edits, and the rebuild is exact only when every piece of metadata that governs placement is consistent with the set it reads. The remaining ten write failures shared a different cause, one worth naming as a class: the silent success. A command reports that it finished, or a view displays a confident summary, and neither did what it claimed. A revert printed "removes 0 edits" and then rewrote two files. A feature named `Waitlist Queue` held the CLI scaffold and no waitlist code, sitting one row above the real waitlist. A save recorded 366 of 370 operations with no provenance because the clustering signal read a field that the save path never populated. In each case the quality degraded on the workflow the study asks participants to use, and the tool's own output gave no sign that anything was wrong. Eight of those ten are silent successes in the strict sense—a command whose report was affirmative and whose effect was not what the report named—and they are the eight this paper counts as the self-reporting class throughout. All eight were recoverable through `undo`, and all eight told the developer something false about what to do next. The common mechanism is that a tool whose entire pitch is legible consequences can produce illegible consequences from correct internal arithmetic. Every number was computed honestly, and the sentence those numbers were placed into was wrong.

The silent-success class is architecturally distinct from a crash. A crash teaches the developer something true about the system's limits: this operation failed, and the developer can reason about which operations share its structure and might fail the same way. A silent success teaches something false: this operation worked, which the developer generalises to other operations of the same kind. Tang et al. found that across 20,574 real-world coding-agent sessions, inaccurate self-reporting—the agent claiming success on work it did not complete correctly—accounts for 22.6% of all developer–agent misalignment episodes, and its share is *rising* even as other failure classes decline [134]. A tool that adds its own self-reporting failures to a context already saturated with them is compounding the ecosystem's dominant residual failure mode rather than compensating for it. Shen and Tamkin found that the skill most degraded by AI assistance is debugging—the ability to recognise when code is wrong and why [125]. A tool that undermines that same skill by confirming a wrong state as correct is compounding the deficit it was built to address. A version control system that exists because the developer can no longer hold a model of what the agent wrote

must not itself produce states the developer cannot distinguish from correct ones. The developer's degraded capacity to detect wrongness is the tool's entire reason for existing, and the silent success exploits that same capacity. The gap between the twelve metadata failures (which produce visibly wrong output: code that does not parse) and the eight silent-success failures (which produce invisibly wrong output: code that parses but does not do what the report claimed) is therefore not a severity spectrum but a category boundary: the first class fails safely, the second fails in the specific way the tool was designed to prevent.

If the study's process telemetry shows history_op events followed by recovery in the sgt condition, these residual write-path failures are the likely mechanism. If it shows history_op events followed by verify (and no recovery), the twenty-one fixes closed the path and participants reached the stable operations that survived the preparation intact.

What the robustness harness measured. The harness ran 289 randomised sequences totalling 10,237 operations across nineteen repository shapes (fourteen synthetic fixtures, five open-source repositories sgt had never recorded). Each sequence applies a random interleaving of saves, reverts, restores, undos, merges, and renames, then checks every file against the byte-identical reference git holds. A sequence passes only if every operation leaves every file matching; a single differing byte on a single file is a violation.

Of 289 sequences, 145 (50%) contained at least one violation on a user-issuable operation—meaning a developer working through the same sequence would have encountered a file whose contents silently differed from what the tool's report described. At the per-operation level the rate is lower: 340 of 10,237 (3.3%), concentrated on entity-target operations (8.1% of 2,447) and layout-target operations (4.1% of 2,308), with non-entity, non-layout operations at 0.8% (46 of 5,482). No sequence produced a traceback—the system never crashed—and no sequence lost data (the append-only store held throughout). What it produced instead was the failure mode the preparation identified: operations that succeed while leaving the files in a state the report does not describe.

The 50% per-session rate is the number a developer meets. A developer who uses sgt for a morning of interleaved saves and reverts has a coinflip chance of producing a file that silently disagrees with the tool's last report. The 3.3% per-operation rate describes a different experience: most commands work, and the developer has no way of predicting which one will not without running the verification the tool was built to replace. The gap between the two numbers is the gap between "it usually works" and "I cannot trust any single result without checking." Adoption depends on the second framing, not the first. Four of the five real (non-fixture) repositories produced at least one violation, so the rate is not an artefact of adversarial synthetic inputs.

What the transcript evaluation measured. Coding pairwise agreement between sgt's grouping and four agent transcripts found that the dominant disagreement was not a clustering error but a ground-truth problem. In CodeNav (30 coded pairs), 15 were "consecutive turns of one feature"—the transcript's turn boundary fell inside a continuous piece of work, so neither system was wrong; they disagreed about where a boundary should be. In eico (30 coded pairs), 15 were "continuation tokens, not intents": the transcript carried requests like "resume," "move on," and "ok sure," whose footprint is everything an autonomous agent then did over a long uninterrupted run. A pairwise F1 computed on a history whose ground truth is 50% continuation tokens measures the grain of the ground truth rather than the quality of the clustering. The share of contentless turns varies from 0% to 88% across the four repositories, and any aggregate metric that pools them without reporting the contentless fraction hides a confound larger than the signal.

The one mechanism that *is* an error in the clustering—intent sharded across leaves (coded F32)—appeared in both repositories and has a consistent shape: two leaves carry the same label and the same intent, separated only because their symbols live in different directories, with no interior node uniting them. In eico, F32 accounts for 11 of 30 coded disagreements. The fix is structural: a merge pass that detects identically-labelled sibling leaves would eliminate it without changing any clustering parameter. That it was not implemented before the evaluation matters: the evaluation found a defect the developer's own use did not surface, which is the role the transcript corpus was designed to play.

The thematic pattern across both repositories reflects back on a design choice Section 4.4 defends. The tool's grouping disagrees with the ground truth for two structurally different reasons: the ground truth is at the wrong grain (30 of 60 coded pairs across both repositories), or the tool shards one intent across leaves that share a label but not a directory (14 of 60). The first is not fixable by improving the clustering—it is a property of the ground truth, which counts "resume" as a boundary—and the second is fixable by a single post-processing pass. No coded disagreement traced to the coupling signal itself producing a wrong boundary between genuinely different intents. The implication is that commitment 4 (inferred names from structural coupling) holds as a grouping mechanism, but the grain it recovers is finer than what a developer would call "one thing I asked for," which is the over-decomposition the conclusion reports. A tool builder choosing between coupling-based grouping and request-boundary grouping should know that coupling gives stable structural communities whose boundaries reflect code architecture, while request boundaries give unstable units whose meaning depends on whether the developer typed "implement caching" or "resume."

Symbol coverage across the four repositories ranged from 49% (semipy-package, 83 ground-truth symbols, 41 known) to 79% (eico, 357 symbols, 282 known). The gap represents symbols the tool literally cannot answer questions about: functions that exist in the ground truth but appear in no sgt record, because the commit that introduced them predates the mining or because the miner's tier filter excluded their file. A developer who asks "what produced this function?" about a symbol in the gap receives silence rather than a wrong answer, which is the correct failure mode (honest ignorance rather than confident error), but it means the tool's coverage is a ceiling on the fraction of provenance questions it can answer rather than an approximation.

What the evaluation taught about measuring itself. The evaluation journal's most consequential finding was not about the tool but about the measurement. On August 15, a derivability check that had passed eight times was re-run after a fix, and the result

improved so sharply that the evaluator stopped trusting it. Comparing the trees revealed that the fixture shipped at signals version 3 while the census copies in `/tmp` had been rebuilt at version 2. The rebuilt copy contained 34 leaves where the fixture had 21, every feature label the check sequences typed existed only in the rebuilt copy, and not one of the twelve recorded command sequences was capable of running against the shipped artefact. The census, every derivability check, and the stop-gate result all described an artefact that existed in `/tmp` and nowhere a participant would encounter.

The failure has a specific mechanism: the analysis tool (`sgt refresh`) quietly rebuilds the store when it detects a version mismatch, the evaluation script copied the fixture before running the analysis, and the copy was what got rebuilt. The measurement was correct (the numbers described the copy faithfully), and the measured object was wrong (the copy was not what participants would see). The study's own participant-path guard already refused to start a session in this state, with a one-sentence explanation of why; no equivalent guard existed on the evaluation path, so the evaluation walked into the state the study was protected from and reported it as the study's result.

Three instances of the same shape—a number computed correctly from an object that was not the object it claimed to be—appeared in one day of evaluation. The only checks that caught them were pre-declared falsification rules written before the measurement: "the census must rediscover the blemishes the build log already confesses to." That rule found 1 of 4 known blemishes on its first run, and the failure of the check (not the failure of the tool) was what exposed the version mismatch. The lesson is methodological: in a system where the analysis tool can modify the artefact being analysed, the evaluation must verify the identity of the measured object after each run, from evidence the tool cannot have produced. A number that passes every internal consistency check is still wrong if the thing it describes is not the thing it claims to describe, and the only safeguard is a rule written before the measurement that asks "is this the same artefact?" rather than "is this number consistent?"

For a CHI audience the finding connects to Sedlmair et al.'s methodological reflection on evaluation in visualization research [122]: the hardest threats to validity are not the ones that make a result look bad but the ones that make it look good by accident. No incentive exists to investigate a passing check. Every one of the evaluation journal's withdrawn results was a result that passed.

What the pilots could not test. Three things only the study with real people under time pressure can show, all of which the pilots explicitly flagged as untestable by an AI agent:

(1) Whether the time cap binds differently per condition (the manipulation check "how much time pressure did you feel" is the direct signal).
(2) Whether carry-over between isomorphic projects is large enough to dominate the condition effect (order as a factor in the model).
(3) Whether a person gives up — the AI pilot participants never did, but a person who spends five minutes with no progress is fundamentally different from an agent that retries.

[Report: order effect size vs. condition effect size. If order dominates, the design lesson is that counterbalancing controls for it statistically but not experientially, and a future study should use non-isomorphic projects despite the difficulty of equating them.]

8.7 Scope and limitations

Twelve participants can detect large effects and nothing else. A confidence interval that includes zero at this sample size does not mean the effect is absent; it means we cannot tell. We designed for this deliberately (Section 6.3) and report intervals rather than p-values so that a reader can see the precision rather than a binary gate. The practical consequence is that a small-to-medium benefit of the kind sgt's read layer probably provides would appear as a wide interval tilted in the expected direction, not as a clean separation, and the reader should treat the qualitative pattern (which measures move, in which direction) as more informative than any individual comparison.

Two hours with a transcript is not a week of forgetting. The study's tasks substitute "code you have never seen" for "code you wrote through an agent and then forgot," and the substitution understates the difficulty of the condition Section 3 describes, because a participant who just read a transcript knows more than the developer who typed those words a week ago. Any advantage sgt shows in this study is therefore a lower bound on the advantage in the scenario that motivated it.

The two study repositories share the same eighteen-marker vocabulary and the same task structure. Counterbalancing controls for which project goes first, but a participant who solves the removal task in one repository has seen the shape of the answer before they open the other. The order effect size ([report]) bounds how much of what we measured is learning the task rather than learning the tool.

The feature labels that make the sgt condition legible depend on a language model call. During preparation we discovered that without a credential, a rebuild produces nine features labelled with raw symbol names (`build_parser main Section`) in precisely the features the tasks ask about. The shipped fixtures pre-cache labels so that participants see domain vocabulary regardless of network state, verified byte-identical across rebuilds with zero model calls. A view whose legibility depends on a network call and a credential is a view that can silently stop being the thing being evaluated; we state this as a limitation of the design rather than of the study, because a user who installs sgt without a model key faces the same degradation and receives no warning.

The limitation that matters most is the one we cannot bound. The study measures whether a tool helps a developer do three specific things to code they have never seen. It cannot measure whether the same tool changes how the developer works over weeks—whether the names enter their vocabulary, whether the attribution shifts how they prompt the agent, whether the consequence preview changes what they are willing to attempt. Those are the effects that would make the tool worth its learning cost, and a two-hour session can detect none of them. What a two-hour session can detect is whether the tool's mechanisms reach the developer at all on first contact. The pilot showed that they can: one participant adopted the attribution immediately and rephrased the revert as a punch list

within twenty minutes. Whether that adoption persists, deepens, or disappears once the session ends and the researcher leaves the room is a question only a longitudinal deployment can answer.

9 Where the Design Breaks

Every number in this section comes from one run of the command a user would run, on the repository that contains sgt, at a single commit. We took them together rather than as we needed them, because the repository is still being worked on and a set of counts gathered over a week is a set of counts of nothing. That repository holds fifty-seven days and three hundred and forty-six commits of daily use by the person who wrote the system. That is long enough to have found the limits below and not long enough to have found the ones that need a year, and we say which is which where we can tell. What emerges from the numbers is a distinction among three properties a developer needs from any tool that rewrites files on their behalf: that nothing is removed (the store is durable), that the tool operates on what the developer named (the report is honest), and that new work can still be recorded after the operation finishes. Section 10 traces each failure class to the property it violates; the numbers below are what those failures look like from inside a session.

9.1 The rate a developer meets

A robustness harness exercised every write verb and every selector format across 289 randomised operation sequences totalling 10,237 applied operations. Of those, 340 left the repository in a state that violated one of three invariants (a file that does not rebuild from its records, a record that names a function no file contains, or a composed file whose bytes differ from disk). The per-operation violation rate is 3.3%, but this number is the one we would choose if we wanted the design to look good, because a developer does not run one operation. A developer runs a session of twenty to fifty operations, and the question they experience is whether that session contains at least one violation.

Of 289 sequences, 145 contain at least one violation targeting an entity the developer would name—a per-session rate of 50%. The violations cluster rather than spreading evenly: 68% of flagged operations target entities (functions the developer named in the command), and of those entity-targeted violations, 157 of 232 trace to a layout record being dragged along by the entity operation rather than to the entity logic itself. The entity logic worked; the seam between it and the surrounding whitespace did not. Splitting the pool by target kind, operations aimed at entities the developer would name (entity plus feature-level and filename-level selectors) carry a 3.1% flag rate; operations aimed at layout records alone carry 4.1%. The seam is not noisier than the core, but it produces a different kind of failure: a layout violation leaves the developer's code correct and the tool's rebuild unable to reproduce it, which is a durability failure that silently weakens the next revert rather than a corruption the developer sees now.

The 50% per-session rate sounds incompatible with the 3.3% per-operation rate, but the two numbers measure different properties and the gap between them is itself a finding. Individual operations in isolation are reliable: a single revert fails at 1.1%, a single save at 1.5%, a single undo at 0.7%. What fails is composition. The round-trip probe—revert an operation, then restore everything the revert removed, and check whether the bytes returned—fails at 44% (175 of 399 trials). A developer who runs one revert almost always gets the right files. A developer who runs a revert, works for an hour, then decides they want it back discovers that the restore does not return to the starting point nearly half the time. The mechanism is that a revert's upward closure removes records whose dependents shift the composed image, and a restore brings back the originals without the shifts, so the two operations compose to a state neither one would produce alone. This is the set-theoretic version of a well-known property in concurrent systems: individual operations can be correct while their composition violates an invariant the individual specifications do not constrain. The design claims that revert followed by restore is an identity, and the harness measures how far from identity the composition actually lands.

On the five real repositories (cloned from GitHub, never touched by sgt before the harness), four of five produced at least one violation, confirming that the rate is a property of the design rather than an artefact of the study repository's migration history. The one clean real repository (asyncer, 50 operations) was also the smallest, so its survival is consistent with the hypothesis that violation probability scales with store size rather than being a fixed defect rate.

The fixture arm and the real-repository arm measured different systems in a precise sense. Fixtures are built to round-trip, so the write guard that fires when the store does not reproduce the committed bytes—the guard responsible for 70–79% refusal on real repositories—was structurally unreachable in all 284 fixture sequences and fired 82 times in 137 real-repository operations. Fixtures also write every caller and callee in one commit, so the cross-commit dependency the `--keep-dependents` flag exists to preserve was absent from all 878 fixture operations that tested it. A behaviour that is absent from the corpus does not produce a zero rate—it produces no rate at all, and a zero in the table reads as "tested and clean" when the honest reading is "not tested." The fixture arm is an excellent bug finder (nine defect classes, all in the layout seam); it is not a deployment estimate, and the 284-of-289 proportion in the pooled table would have hidden exactly that distinction.

The practical consequence is blunt: a developer who uses the write layer daily will encounter a violation within their first two sessions on average. Muir's model of trust development explains why this timing is catastrophic [95]. Trust in automation passes through three qualitatively different stages: predictability (can I anticipate what the system will do?), dependability (does it do what it says it will?), and faith (will it handle situations I have not yet seen?). A developer in their first two sessions is still at the predictability stage—they are learning what the tool's operations mean, not yet relying on them. A failure at this stage does not merely subtract one positive experience from a base of many; it prevents the stage transition entirely, because the developer cannot progress to depending on a system whose behaviour they cannot yet predict. Lee and See's calibration model explains why recovery is slow once trust is damaged [80]; Muir's developmental model explains why damage during formation is qualitatively different from damage during maintenance—it is not a setback within a

stage but a barrier to reaching the next one. The timing is worse than it would have been a year ago. Across the broader ecosystem, trust in AI-generated output is actively declining—from 40% to 29% in one survey cycle [130]—while adoption is near universal. A developer arriving at sgt in 2026 already expects automation to be unreliable, which means Muir's predictability stage requires more confirmations before the transition than it did when the baseline expectation was neutrality rather than scepticism. The threshold for earning trust rises as the ecosystem trains developers to withhold it.

A concrete instance from this paper's own construction illustrates the pattern the harness measures in the abstract. During development, a merge conflict resolution took the in-progress copy of six files whole, and the copies predated several fixes that had already landed. Git reported the merge as a success; the working tree was clean; six functions were silently removed, including one that the agent interface calls on every restore operation. Two of the six had no failing test, because the same resolution had removed their tests along with their code. A suite that passes over reduced coverage reads the same as a suite that passes over full coverage, and a green checkmark after a merge is the same silent-success pattern the pilot participant encountered at the interface level. Finding the removal required comparing name-by-name coverage across thirty files against a reference commit—exactly the operation sgt is designed to perform, and one we did not think to run on our own merge because we had no reason to doubt the result.

9.2 Most of a repository is not functions

`sgt log --summary` prints `396 file(s), 6821 symbol(s), 154 feature(s), 25% entity coverage`, and we printed that last figure for months before checking it against the records it claims to summarise. It divides the files that still hold a live function-level record at the current commit by every path any record mentions, including files deleted long ago, so its two halves are drawn from different populations and the ratio understates the coverage by a factor of nearly three. The count below is the one to read.

Counting the records themselves says something different. This store holds records for 534 files, more than the 396 on disk today because a record outlives the file it was read from, and of those 534, 381 carry function-level records and 153 are recorded whole. The whole-file set is 82 Markdown files, 19 JSON, 15 LaTeX, 13 HTML, and a scattering of CSS, SVG, and configuration, and it contains no Python and no TypeScript, so every file in a language sgt parses has a function-level record. The design's coverage of code is complete, and the files left over, nearly three in ten of everything the store has a record for, hold no function for anything to be filed under.

So the limit is the repository's composition. Calling it a parser problem let us describe a fixed property of software as though it were a defect we could fix. Go and Rust have functions. A tree-sitter grammar plus a decision about which node kinds count as one is a day of work per language, and we would take both tomorrow. It would not move that share at all, because there is no Go and no Rust in this repository. What is in it is prose, configuration, and generated files, in a proportion no repository of this kind escapes. A change to four lines of a deployment manifest has no function to file the developer's request under, no matter how many grammars we ship. Every operation in Section 4 still runs on those files, at git's grain, and none of them gets sharper than `git revert` would be. A reader deciding whether to adopt sgt should ask how much of the work they need to *take apart* happens inside functions, which is a different question from how much of their repository is code. In the repository this paper studies, 70% of files are code and 100% of the tangling problems the scenarios describe live in functions, because a deployment manifest does not entangle with a retry policy inside a shared definition.

9.3 Where a function loses its history

The design assumes a function's record survives when its source is renamed or moved, and that assumption rests on a matcher with a threshold in it. sgt links a function across a commit first by its name, then by identical bytes, then by identical structure with formatting and comments ignored, and last by token overlap above 0.80, with a guard that rejects any pair whose token counts differ by more than a factor of two. The 0.80 comes from sem, which uses the same figure for the same job [3]. It scores the overlap between the two bodies' sets of distinct tokens, which puts the breaking point lower than the number sounds: replacing more than about one token in nine drops a pair below it, and the function is then recorded as one removed and a different one added.

The developer then sees a function whose recorded history begins at that commit, with its earlier edits still filed under a group the function is no longer part of, and an agent asked to clean up a module produces exactly this, renaming six functions and rewriting their bodies in one sitting. The detectors that get a function's origin right compare full syntax trees across the two versions [40, 49, 75, 145], where sgt uses the token count because it runs on every save and has to finish while the developer is waiting. Lowering the threshold would link functions that merely resemble each other, and a wrong link is the worse failure, because it files one function's edits under another function's name and gives the developer no reason to doubt the answer. `sgt advanced identity join` states the link by hand, and using it requires the developer to notice that a function's history restarted, which is the kind of thing they installed sgt in order not to have to notice.

9.4 Fifty-six files we cannot rebuild

The same command reports 56 files whose bytes on disk no valid set of records can produce. The denominator is the 381 files whose records name functions and not the 534 the store holds, because a file recorded whole holds its own bytes and cannot fail to come back, so one function-level file in seven does not rebuild. sgt keeps every one of them, prints the count, names the first five, and points at `sgt advanced fsck --tree`, so nothing was deleted and nothing was quietly wrong (Section 4.2). What the developer loses is the record for those files: a revert of a group that touched one of them leaves the file as it is, and `sgt show` on a function inside it answers from an incomplete chain.

We have not tracked down what put all 56 in that state, and since this store holds records written by five different versions of our miner in fifty-seven days, some of them are the residue of our own format changes. A developer adopting sgt on a repository with two

years of history should expect a number of this kind on the first run, and should read it as work the design has not finished.

A deeper instance of the same incompleteness surfaced during the evaluation. sgt reads a repository's history in ten-second chunks and records a single commit as its stopping point. A single commit cannot describe position in a branching history, so when a chunk boundary falls between a merge and its second parent, commits on the side branch are stepped over and never read. On one repository mined twice, the second reading was missing 49 operations from one commit, and 8 commits carrying roughly 30 files were absent from both readings. Both readings reported themselves complete. The same mechanism means that a function renamed across a chunk boundary is recorded as one name removed and a different name added, producing 108 differing identifiers between two readings of the same commit on another repository. This is a self-reporting failure of the same class as the ones the pilot found at the interface level: the tool claims completeness while the claim is false, and nothing in the tool's own output gives the developer reason to doubt.

A deeper problem compounds the chunk-boundary effects: the budget is wall-clock time, so the store's contents depend on machine load at the moment of mining. Five mines of one repository produced 150, 168, 168, 237, and 237 operations from the same commit, and the two large stores reconstructed fewer files than the small ones, because extra operations introduced by a longer chunk sometimes compose to the wrong bytes. The practical consequence is that two developers onboarding the same repository on different machines can end up with different stores, different reconstruction rates, and different sets of refused operations, with no indication that anything distinguishes them. A design that records *what happened* should not depend on *when it was read*, and this coupling is a bug rather than a tradeoff.

The consequence on external repositories is severe. On the first repository outside the author's own projects that the evaluation pointed sgt at (746 commits, 118 merges, two package renames), reconstruction reproduces 35% of function-level files. This is not noise around a successful result; it is a total failure of the write layer, because the guard that protects composition refuses every write verb when the composed image disagrees with disk. A developer who clones a mid-sized library, runs `sgt init`, and waits the six unattended minutes it takes for onboarding to reach genesis gets a tool that can attribute but cannot act—the read layer works, and every write verb refuses. The reconstruction rate on the author's own repository (7 of 8 function-level files rebuild) is not evidence that the system works; it is evidence that the author's habits (linear history, no deletions, no package renames) happen to avoid the three inputs that break composition.

The rebuild also fails in the opposite direction: on 28 of 29 corpus repositories, the composed image materialises files that do not exist at `HEAD`—1,864 paths across the corpus, produced by operations whose targets were deleted or renamed and whose records were never pruned. A metric that checks only whether a file at `HEAD` can be reproduced from its records cannot see this failure, because the invented file is not at `HEAD` and therefore not in the denominator. Requiring the rebuild to produce `HEAD`'s file set *and nothing more* moves the corpus median rebuild rate from 0.33 to 0.24. The two numbers answer different questions: the first asks how much of the working tree the record can explain, and the second asks how close the record comes to reproducing the working tree as a whole. Both are needed, because the write guard compares the entire composed image against disk, and a surplus file disagrees just as a missing function does. On the five real repositories, surplus paths account for a third of refusals—the write verb refuses because the rebuild contains files that no longer exist, not because it cannot reproduce the ones that do.

The onboarding path has a separate failure that compounds the completeness problem. sgt reads history in ten-second chunks, and between chunks nothing tells the developer their history is partial: no command drives the walk to completion, the incompleteness flag exists only in the JSON output humans do not read, and the summary prints "13 file(s) on disk differ from the recorded state—`sgt save` absorbs them." Following that instruction would record 746 commits of other people's work as a single save by whoever just typed it, destroying the provenance the tool exists to produce. A confident instruction pointing at a destructive action, firing on every repository large enough to matter, is a stronger instance of the self-reporting pattern than any single defect the robustness harness found. The refusal messages themselves misdirect: in two cases on the recovery path, the tool names a wrong cause ("operation does not exist" for one it holds; "set `OPENAI_API_KEY`" for a failure unrelated to any credential). A system whose failure messages are wrong has the honesty property the design claims for it in the code paths that were tested, and not as a structural guarantee.

9.5 The layout seam

Order and whitespace are encoded as symbols but are not symbols under the kernel's relations. A blank line between two functions is recorded as a layout record, carries no callers and no dependencies, participates in no structural community, and cannot fork because nothing competes for it. Yet composition treats it identically to a function body: including a feature includes its layout records, reverting one removes them, and the rebuild-from-records guard checks the composed bytes against disk with no distinction between a wrong function body and a wrong separator. This encoding preserves insertion order deterministically—without it, recomposed files would lose their formatting—and it is the single most productive source of defects across the entire evaluation.

The evidence arrives from three directions at once. In the robustness harness, 157 of 232 entity-targeted violations trace to a layout record being dragged along by the entity operation rather than to any failure in the entity logic itself. On real repositories at rest—before any operation runs—19, 48, and 89 orphaned layout records appear across three of five clones, growing to hundreds after the mine completes, because the fork rule withdraws the entity records that anchor them and leaves the layout records pointing at a gap. And the evaluation's only two recoverability failures—a 33-byte file emptied while the tool reported it had done nothing—both targeted layout records. No entity record lost a byte in 10,237 operations. Every byte that was lost came through the seam.

The mechanism is not a bug in any individual component. The kernel's relations (calls, co-change, structural containment) ground entity records in a dependency structure that composition can reason about: if a function depends on another, including one includes

the other. Layout records have no such grounding. They float between the entities they separate, and when one of those entities is withdrawn by the fork rule the layout record stays behind, orphaned, pointing at a position that no longer exists in the composed image. The composed file then disagrees with disk by the bytes of that orphan, the rebuild guard fires, and every write verb that touches that file refuses. The developer sees "file does not rebuild from its records" and has no way to know that the record in question is a blank line between two functions that the fork rule withdrew.

A designer facing this choice has three options. Record order as metadata attached to each entity (loses the ability to revert a formatting change independently). Record order as a relation between adjacent entities rather than a symbol in its own right (the approach Darcs takes with its sequencing theory, at the cost of a different composition algebra). Or accept the seam as a maintenance cost and fix the orphans it produces. The third is what we have been paying, and the evaluation's finding is that the cost is higher than we estimated: not a cosmetic imprecision in the rebuild, but the mechanism behind half the addressable failure surface and the only path to data loss the harness found.

9.6 Findable is not decomposable

On the two study repositories, every information request is answerable from the sgt record in at most three commands: eight derivability checks, run three times across two rebuilds, pass every time. A reader might stop there. The checks are too easy. They ask whether the answer is *reachable*—whether some sequence of `sgt log`, `sgt show`, and `sgt find` can surface the relevant symbols—and at function granularity almost anything is reachable. The operations the design is built on are not retrieval but *decomposition*: naming a coherent piece of work as one unit, and reverting that unit without side effects. Measured as decomposition, the same trees fail. One feature in each repository participates in 18 of 22 episodes (courscraft's `CLI Scaffold`) or 20 of 22 (confplan's `Schedule Grid`), so reverting that feature removes work from nearly every sitting. Five of 34 features carry a label that describes under a third of their symbols. A 5-symbol removal reverses 13 edits, because the structural cut crosses the episode boundary wherever a hub function participates.

The three numbers are not three findings. Every episode splitting across 3–6 features, one feature spanning 90% of episodes, and reverting 5 symbols costing 13 edits are one fact seen from three sides: the clustering cuts along the code (files, coupling) and the work ran along features (sessions, requests), so the cuts land crosswise. A paper that reports them as a list of weaknesses would pad the count and hide that a single design choice produces all of them. The choice is commitment 1: the function as the unit of recording. A function belongs to one feature at a time. A feature is any set of functions. When a hub function participates in every episode, every episode and every feature overlap in that hub, and the overlap is the mechanism behind every number above.

The corpus evaluation confirms the same mismatch at a larger scale. On four real repositories (two external, two this project's own transcripts), one developer request becomes a median of 4–6 sgt features and up to 12 on the longest session. No repository reaches 1:1. The finding kills a framing the paper once used: features are not recovered intents. At this granularity they are recovered *sub*-intents—structural communities that correspond to what a developer did at the code level, not to what they asked for at the request level. Suchman's distinction between plans and situated actions explains why no refinement of the clustering will close this gap [132]: a developer's request is a plan in her sense, a resource that oriented the agent's work rather than a specification that determined it, and one orienting statement produces several independent implementation decisions the statement never mentioned. The gap between request and feature is the gap between planning and doing. The distinction matters because the paper's scenarios ask "which request produced this function," and the feature map answers a different question: "which other functions were modified in the same structural neighbourhood." That answer is useful—it is the cross-file grouping git cannot provide—but it is a weaker claim than intent recovery, and the paper must state it as the claim actually supported rather than letting the reader infer the stronger one.

The practical consequence for the developer in Section 3 is that `sgt revert "price cache"` does not do what the scenario promises. The price cache was one request. The tool split it into three features: one holding the cache helper, one holding the TTL logic because it co-changed with an unrelated timer, and one holding the parameter that threads the TTL through `db.py::query` because that function's coupling graph places it in the database cluster. The developer who typed one sentence must now name three features to remove one request, and deciding which three requires reading the feature map—the same comprehension labour the tool was supposed to eliminate. The labour is smaller (read a feature tree instead of reading the code), and it is faster (a `sgt log --map` call instead of a `git log -L` call per function), but it is not zero, and a design whose selling point is "name it and remove it" must say honestly that "name it" sometimes means "find the three pieces it became." Clark's grounding theory explains the gap: shared vocabulary requires proposal and acceptance between two parties [27], and the tool proposes its structural categories without any acceptance step from the developer. The merge command exists to repair this, but a developer who does not know the pieces exist cannot know they need merging. Surfacing the mismatch—showing, at save time, which features the tool proposes and asking the developer to confirm—is the fix this design has not yet built. Building it requires interrupting the save flow, which is the forcing-function cost the fork rule already pays. That fix also contradicts Section 4.4's own premise: Shipman and Marshall showed that systems requiring users to formalise their work get abandoned [127], and a confirmation prompt at every save is a formalisation request at the highest-frequency operation. The design space is therefore narrower than Clark's model alone suggests: the acceptance step has to be cheap enough that developers do not skip it, and intermittent enough that it does not become the same cognitive tax the tool was built to eliminate. A preview that appears only when the grouping changed since the last save, or a deferred confirmation that accumulates until the developer acts, might thread that needle. We have not built either, and until we do the grounding gap stays open.

9.7 What a colleague sees

Because the records are files in the repository, everyone who pulls the repository gets them whether they asked for them or not, and this section works through what that means for three people who are not the developer running sgt: a colleague who has never installed it, a colleague who installed a different version of it, and the reviewer of a pull request.

The one we worried about most turned out to be the easy case. A colleague who has never heard of sgt commits ordinary code, and the next `sgt sync` mines their commits into function-level records, the same way sgt mines pre-existing history. Their edits arrive at the right grain and group with the work they belong to. What does not arrive is the request sentence, because there was none to capture: the name for their group is drawn from commit subjects instead. A colleague who writes `fix tests` and `wip` gets names derived from the code, the same failure Section 4.4 describes for a save with no message. The grain sgt recovers from anybody; the vocabulary only from somebody who was running it.

The harder case is a colleague who does install it and runs a different version. sgt recovers its records from code instead of being handed them, so the program that did the reading is part of what a record is, and an edit's identifier is computed from its content together with the version of the miner that produced it. Two versions therefore mint different identifiers for the same edit, and a sync across that gap would quietly alias records that mean different things, which sgt avoids by refusing the whole union and printing what to do. Two people using git have to agree about bytes and nothing else, and two people using sgt have to agree about a program. That refusal is correct and it is also the cost we are least able to speak to, because Section 9.4 reports five miner versions inside fifty-seven days and every one of those transitions would have stopped a collaborator. It stopped nobody, since there was nobody to stop, and a design decision that a solo repository charges nothing for is a decision that repository cannot evaluate.

The third case is the reviewer, and here we have no answer at all. A pull request that carries sgt records shows them as what they are on disk, a few hundred JSON files whose names are hashes, and no reviewer reads that; they review the code, as they should, and how anybody should read a large model-written change spanning many files is an open question being studied on its own [60]. The reason is structural, not incidental. Code review evaluates correctness: does this code do what it should? The sgt records encode provenance: which request produced which function? Evaluating provenance requires the reviewer to read the original requests, compare them against the grouping, and judge whether the inference is right—which is the comprehension work the tool was built to eliminate, performed by a person who did not write the code and has even less context than the developer who did. A reviewer who cannot evaluate the record cannot catch a wrong resolution or a mislabelled feature, so the record that makes `revert` work is not part of what anybody approved, and a resolution one developer reasoned through and a label another corrected both enter the shared branch unexamined. The rule in Section 4.3 asks a person to settle a fork, and on a team that person is still one person, while the review that would have caught them being wrong reads a diff their decision does not appear in.

9.8 The fork rule has never met two people

Section 4.3 says a resolved fork has to pass the project's test command before sgt will include it, and the summary above also prints `oracle: unconfigured`. In the repository we use every day, resolving a fork means the developer picks a version and sgt takes their word for it, because nobody ever pointed sgt at our test runner. A safety property a developer has to configure is a safety property most repositories will not have, so the strongest claim in Section 4.3 holds only where somebody did that setup, and the person who wrote the rule did not do it in the repository where the rule runs every day. The fix needs no research, because sgt could find the test command the project already uses and offer it during `sgt init`, and making the developer ask for it was the wrong default.

The condition the rule has been tested under is narrower still. This repository's history carries three git identities and one human being behind all of them, so every merge sgt has ever seen had the same person on both sides of it, which is the condition in which the rule is most obviously right, because the person a fork stops is the person who wrote both versions and can answer it in seconds. It is also the condition in which the rule is cheapest. On a repository where twenty people edit the same functions every day, the same rule produces a queue of decisions whose two sides were written by people who have not spoken, and Nelson et al.'s account of how teams work through merge conflicts describes the barriers a queue of that kind meets [99]. We expect that team would switch the rule off and take the textual merge, and our evidence cannot tell them not to.

The setting we would build applies git's rule at the grain of a function, merging when the two edits changed no overlapping lines and asking when they did, which would also let sgt report a divergence while both sides are still being written rather than when they meet [53]. We have not built it, because we cannot tell a reader which of the two should be the default, and the repository we would have tested it on has one person in it. This is the part of the design a study of larger teams would most usefully overturn.

9.9 The record the agent already had

The grouping sgt recovers by clustering existed before any of the code did. The agent held it while it worked, and every agent we have used discards it when the session ends. Hutchins's account of distributed cognition explains what disappeared [65]: the knowledge of which functions serve which purpose was formerly distributed across three supports—the developer's procedural memory of typing the code, the commit structure that recorded boundaries, and the naming conventions that made intent readable from identifiers. Agent-mediated development removes the first two supports (the developer typed sentences, not code; the commit happened once at the end, not at each boundary) and leaves the third intact but insufficient, since function names describe implementation rather than purpose. sgt reconstructs what was lost by mining the structural channels (coupling, co-change) that survive the transition. A record written by the agent, one entry per request naming the functions it changed, would make the inference in Section 4.4 unnecessary, and it would fix the case in Section 5 where a rename nobody asked for got a name derived from the code.

Half of that record we do take, and the half we take is the half a vendor cannot spoil. Where the developer's agent is Claude Code, sgt installs a hook on prompt submission and stores every sentence the developer typed, with its time. The words that named the work are on disk before any code exists, and `sgt now` answers with what was asked for rather than with a count of edits. That is twenty lines and one vendor's hook interface, and the same twenty lines have to be written again for the next agent, which is the price we decided we could pay, because what they capture is the developer's own English and a change of vendor cannot make yesterday's sentence wrong.

The half we do not take is the agent's account of which functions each request touched, and the reason is that a record of that kind is a claim about the files which we would have to either trust or check. Trusting it puts the correctness of a revert inside a transcript nobody validated against the code. Tang et al.'s finding that inaccurate self-reporting accounts for 22.6% of all developer–agent misalignment episodes—rising even as other failure classes decline [134]—says directly how often that trust would be misplaced. Checking it means reading the files and computing the answer, at which point we did not need the account. What the agent's account would improve is the label, which is the part of sgt that is allowed to be wrong, and improving a label is worth a hint and not worth a dependency. So the split we have arrived at is that membership is read from code and names are read from words, and the words come from the hook where there is a hook and from the save message where there is not. What we do not know is whether an agent's account of its own edits is accurate enough to move membership too, and that is the measurement we would ask a reader of this paper to make before any other.

Open science

sgt is open source, and the repository it versions is the repository that implements it, so the artefact and the evidence in this section are the same object: a reader can clone it, check out the commit the artefact names, run `sgt log --summary`, and read every number in this section off the output. Run at a later commit the counts will have moved, since the repository is the working one. The within-subjects study in Section 6.3 is pre-registered: its design, measures, and the thresholds in Table 1 were fixed before recruiting, so that the first row remains the first row after we have seen the data. Two moderated pilots (Section 7) validated the tasks, the rubric, and the scoring script; they found twelve tool defects and four design defects, all fixed before the full study ran. The two repositories the study uses, the transcripts that produced them, and the correctness script are released with the results. The study's recorded data are screen activity, task outcomes, and interview audio from consenting professional developers, with no personal data beyond what consent and compensation require.

10 What the Design Bought and What It Cost

A system paper owes the reader two things the evaluation alone does not provide: which design choices caused each failure class, and whether a different choice would have avoided the failure or merely relocated it. This section traces each of the four commitments of Section 4 back to its evaluation outcome, names the alternative the designer faced at each decision point, and states what the alternative would have cost instead. The goal is a map that a tool builder in the same design space can navigate without rerunning the experiment: here is where each path leads, here is what we found at the end of ours, and here is the evidence for turning earlier.

10.1 Three distinct safety properties

The evaluation surfaced three properties a developer needs from a tool that rewrites files on their behalf, and they require different kinds of guarantee.

(1) **Nothing is removed.** The store is append-only; every prior state is reachable by identifier. The append-only store guarantees this structurally, and all 10,237 operations in the harness preserved it.

(2) **The tool operates on what you named.** The revert removes the feature the developer pointed at, the restore brings back the thing they asked for, and neither touches anything else. Three findings in the robustness harness broke this property: an undo that silently dropped a *different* edit, a revert that printed success about an operation it had not performed, and a restore that reported "changed nothing" while emptying a file. All were recoverable—property 1 held throughout—and all were wrong in the direction hardest to detect, because the command's own report confirmed what the developer expected.

(3) **New work can still be recorded.** A revert that succeeds must leave the repository in a state where the next save is accepted. One sequence in the robustness harness reached a state where every write verb was refused, including `undo`, because a layout record outlived the entity it described and the rebuild's composed image disagreed with the file by one byte. The developer's work was intact and unreachable.

Property 1 is architectural and held universally. Properties 2 and 3 are behavioural and failed on sequences involving layout records (the gap text and ordering facts that sit between functions in a file). The append-only store makes durability cheap, so legibility becomes the remaining problem.

The distinction matters because the three properties fail with different visibility and different recovery costs. Property 1 failures are catastrophic and detectable: the developer's code is gone, and they know it. Property 3 failures are non-catastrophic and detectable: the developer's code is present but the tool refuses to record new work, and the refusal is loud. Property 2 failures are non-catastrophic and *undetectable without an external oracle*: the developer's code is present, the tool claims it did the right thing, and the claim is wrong. In every one of eight self-reporting failures we found across the evaluation, the data was present and the tool's account of how to reach it was wrong. The tool exhibited what Bainbridge called the fundamental irony: the humans most in need of monitoring are the ones least able to provide it, and here the tool most in need of checking is the one whose reports are the only thing the developer has to check against [5]. Sarter and Woods identified the mechanism by which this gap widens: an operator whose feedback confirms a state the system is not in stops monitoring, and every subsequent action builds on the wrong model until a later failure exposes the

divergence at a point far removed from its cause [118]. The green checkmark over corrupted output is exactly this structure—the developer's model ("the revert worked") diverges from reality ("three function headers are glued onto one line"), the divergence is invisible because the feedback confirmed the false state, and it surfaces only when a test runner or a colleague finds the damage.

The pattern is not unique to sgt. During this evaluation, a git merge resolution silently reverted six previously-landed fixes. Git reported the merge as successful, the working tree was clean, and no test failed—because the same resolution removed the tests that would have caught the reversion. The code was present in history and absent from the working tree, with no indication that anything had been lost. Two of the six were found only by comparing the names in the test roster against the names at the prior commit, a check nobody was running. This is property 2 failure in the tool the paper's baseline condition uses, and it manifests the same way: the system reports success while silently doing less than the developer intended.

The self-reporting pattern has a mirror that the evaluation found independently. Seventeen files are reported as "unrebuildable," but fourteen of them are files sgt decided never to record at function level—dot-paths and generated configuration that the tier filter excluded. A developer chasing seventeen reported breakages finds three. Over-reporting and under-reporting are the same defect for the same structural reason: the tool's self-report confuses what it *chose not to record* with what it *failed to record*, and a developer who trusts either direction—whether the news is good or bad—acts on a model the tool does not support.

The design implication generalises to any system that maintains a semantic record alongside a repository. Making the store append-only gives property 1 for free and makes property 3 failures recoverable (the developer can always fall back to the raw files). Property 2 requires a different kind of assurance: the tool's self-report must agree with an independently checkable outcome, and the only oracle that can verify agreement is a probe that recomputes the result without consulting the tool's own state. The robustness harness is exactly this probe—it applies the operation, then checks the outcome against a reference computed from git—and its 50% per-session hit rate (Section 9.1) is the measure of how far property 2 remains from holding. Testing durability (can the bytes be recovered?) answers a different question from testing agreement (did the tool do what it said?), and a test suite that checks only durability passes with flying colours on a system whose self-reports are wrong half the time.

Casserini et al. named this accumulation *agentic entropy*: the systemic drift between what an agent does and what the developer intended, compounding over time and invisible to diff-based review [20]. Their diagnosis is correct—the diff tells you what changed in one commit, not how twenty commits diverge from what was asked for—but their proposed remedy (conformity seeding, reasoning monitoring, a causal graph of agent decisions) observes the agent's internal trace, which is available only while the agent is running. The record sgt maintains operates at the complementary layer: it persists after the agent exits and expresses drift in terms the developer already holds ("this function was produced by that request"), so the entropy is visible at any future point and actionable through the same operations that produced the work. The distinction matters because an agent's reasoning trace is both proprietary and ephemeral—it disappears when the session ends—while the request-to-function map is durable and belongs to the developer.

Recoverability must be defined precisely, because each imprecise version produced a false violation during the harness's construction. The definition that survived calibration: after any sequence of verbs, the bytes of every prior state are reachable through documented commands. Reaching this statement required six successive revisions to the test oracle, each removing an assertion about the tool's *representation* (which operation-ids a set contains) that disagreed with the tool's *content* guarantee (which bytes a file holds). The revision sequence is itself a finding: operation-set identity and content identity are different equivalences, and every safety claim in this domain must name which one it means. A system can satisfy one while violating the other, and the one a developer cares about is bytes, not identifiers.

10.2 What each design commitment cost

We trace each of the four commitments of Section 4 back to its evaluation outcome to show where the design has already been tested and where it has not.

The function as unit (Section 4.1). Delivered entity coverage over 42% of files, with a threshold that loses function identity on roughly half of all renames that co-occur with a body rewrite (Section 9.3). On the corpus the decomposed rebuild rate is 0.23, meaning that three functions in four are absent from the composition at `HEAD`. Ninety percent of those absences come from the fork rule withholding records, and the parsing itself succeeds, so most of the cost at this level actually originates from the fork rule (the next commitment) rather than from the function-as-unit decision itself. The implication for a designer choosing a grain: the function works as a recording unit—parsing succeeds, identity tracking is imperfect but bounded, and the coverage spans every file tree-sitter can read—but the cost of acting on those records depends on what other commitments the design makes downstream.

Set-based versions (Section 4.2). Gave the round-trip reversibility the walkthrough demonstrates, which the pilot participant called "a fast, accurate first pass that leaves me a punch list." That reversibility breaks twice in 10,237 operations, both on layout targets no developer would name. The robustness harness found 340 operations that left the repository in an unexpected state, and 68% of those targeted an entity the developer would name—but the dominant violation in those cases (157 of 232) was a layout record being dragged along by the entity operation, while the entity logic itself worked correctly.

The set formalism fails in two distinct ways. First, layout records: layout facts are individually addressable because they are symbols, and individually revertible because they are records, but no mechanism keeps them consistent with the entities they sit between. Nine distinct defects trace to this decision, and every robustness-harness failure entered through it. The failure has a counterintuitive property: a lost separator is latent until both definitions flanking it are recovered, so the number of broken files *rises* as the record becomes more complete. A developer who assumes "more history mined,

fewer broken files" has it backwards, and the progress bar that shows mining advancing is not a progress bar toward a working write layer.

Second, upward closure: reverting a mid-history edit originally orphaned every later edit that depended on it, because the closure rule excluded them from the rebuilt set. On the study repository this demolished 13 files when the participant asked to remove one feature—the worst single defect we found. The fix replaced blanket exclusion with forward subtraction (splice the removed content out of later edits rather than dropping them), which preserves the set semantics while changing what "remove" means from "exclude this and everything above it" to "subtract this contribution from the tip." The lesson for a designer using set-based versions: closure is correct for the append-only store (nothing is lost) but too aggressive for the working tree (a developer removing one feature does not intend to remove everything built on top of it). The distinction maps onto fail-stop versus fail-silent behaviour: forward subtraction is fail-stop because the developer sees the splice and can reject it (the preview names every function that would change), whereas blanket exclusion is fail-silent because the developer asked to remove one feature and thirteen files disappeared without a question. Bainbridge's irony applies directly: the developer who installed automation to handle the decomposition is the developer least equipped to notice when the decomposition was too aggressive, because they have not been practising the skill of holding it themselves [5].

The fork rule (Section 4.3). Protected 162 actually-forked functions and blocked mutation on 27 of 28 outside repositories. The amplification factor is 18: each forked function withdraws records covering 17.7 function keys on average, and the total withdrawal of 2,864 keys is what blocks the write guards. On the self-hosted repository, switching the rule off raises the rebuild rate on function-level files from 0.23 to 0.83—a 3.7× gap that the pooled figure (which includes whole-file records that trivially reconstruct) had compressed to 2×. On the corpus, the median rebuild of 0.33 means the guard requires 1.00 and most repositories are at one third of that. The one corpus repository that reaches 1.00 holds 293 records where the others hold 1,382 to 7,951; it is also the only repository where `sgt save` succeeded and the only one the robustness harness flagged nothing on. Three separately reported positive results trace to the same unit—a repository too thinly recorded to trigger any of the failure modes the denser stores exhibit. The rule's read benefit is real (the pilot participant was stopped before two conflicting edits could be silently merged), but the write cost is eighteen times larger than the rule's description suggests.

The fork rule is a cognitive forcing function in Buçinca et al.'s sense [16]: a deliberate friction that interrupts the developer's acceptance flow and forces engagement with the decision the tool would otherwise make silently. Buçinca et al. found that the most effective cognitive forcing functions—those that reduced overreliance on AI recommendations the most—were also the ones participants rated least favourably, because the subjective cost of being stopped is felt immediately while the objective benefit (avoiding an error) is counterfactual and invisible. The pilot reproduced this finding exactly: the fork rule stopped the participant before two conflicting edits could be silently merged (the safety benefit held), and the participant declined to use the session feature afterwards (the goodwill cost was immediate). The mechanism is that a forcing function pays for itself only when it fires on an actual conflict, and the developer cannot distinguish a true positive (the function genuinely diverged) from a false positive (two edits touched opposite ends of a long function) until they have done the inspection the rule demands. In Buçinca et al.'s framework, the forcing function's value depends on the base rate of errors it prevents: if most firings are false positives, the developer learns to treat the interruption itself as noise, and the rare true positive arrives in a context where the developer has already learned to dismiss.

Nelson et al. found that the expensive part of a merge conflict is the semantic investigation required to understand whether both sides can coexist, not the textual resolution [99]. The fork rule moves that investigation from the moment the merge fails to the moment the second edit arrives, when the developer still holds both intents. This temporal shift is the rule's strongest claim: at the moment of the second edit, the developer still knows what both versions intended, so the investigation costs seconds rather than the minutes Nelson et al. measured at merge time. Kazemitabaar et al. showed the same structure in a pedagogical setting: cognitive engagement techniques that force students to think before accepting AI-generated code improve comprehension at the cost of completion speed [71]. The fork rule imposes the same trade: it stops passive acceptance of a merge and forces the developer to state which version they want, and the pilot confirms both sides of that exchange.

Ferino et al. proposed a reliance-control framework for AI in software engineering that uses the developer's *level of control* as the mechanism for identifying both over- and under-reliance [43]. The fork rule instantiates their framework concretely: it increases the developer's control at exactly the point where silent merging would decrease it (two edits to one function from different intents), and does so by making the developer state which version they want rather than accepting whatever the system computes. The cost is what Ferino et al. call unnecessary control retention—being forced to decide when both versions are clearly compatible —and the benefit is what they call appropriate reliance calibration: the developer knows which merges required their judgment, and which did not.

Duan et al. found that distrust propagates between teammates more readily than trust does: one person's negative experience, relayed by word of mouth, reduces a colleague's reliance even when the colleague's own experience has been positive [38]. The fork rule has been tested only with one person, and this contagion effect suggests that a single developer's frustration with it could suppress adoption across a team even if the rule's safety guarantee would have helped the others. The implication for a designer building forcing functions into developer tools: the function must fire rarely enough that most firings are true positives, because each false positive erodes the willingness to engage with the next interruption, and on a team that erosion propagates socially. On the study repository (162 forked functions in 346 commits), the fork rule fires on roughly one function per two commits; we do not know whether that rate feels like signal or like noise to a developer who is not the system's author. Figure 9 shows the amplification visually: the 153 files the

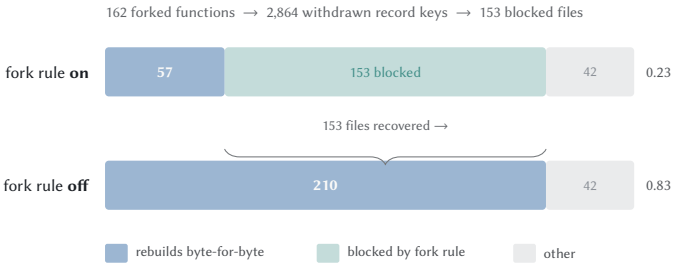

Figure 9: The fork rule's cost on sgt's own repository, measured over the 252 files sgt decomposes into functions. With the rule, 57 files rebuild; without it, 210 do. The 153 files the rule blocks are the transitive cost of 162 forked functions withdrawing 2,864 record keys through their dependency chains. The 42 files that fail in both cases are the residual losses of Section 9.4: missing separators, incomplete chains, and the one file whose pending record our own measurement apparatus appended. Whole-file records (104 files, not shown) rebuild at 0.97 in both conditions and move by exactly one file.

rule blocks come from just 162 forked functions, and switching the rule off recovers them but removes the safety guarantee.

Inferred names (Section 4.4). Recovered cross-file groupings that git cannot express, and systematically over-decomposed what a developer asked for by a factor of 2 to 6 at the median (up to 13 on the longest-session repository). On CodeNav (the one repository where the metric discriminates), F1 is 0.66 with precision 0.68 and recall 0.64. The margin over a file-only baseline comes entirely from cross-file grouping. The tool's vocabulary is finer than the developer's vocabulary, which means the operations of Section 4 require naming something smaller than what was originally asked for. The pilot participant saw five unnamed features for one coherent piece of work and asked why the tool was decomposing what they had done in one sitting. On the study repositories, the feature surface is a reliable index into history (every request's answer is reachable in at most three commands) but an unreliable decomposition of it. Fifteen percent of labels describe a minority of their members' actual content, and reverting a 5-symbol feature removes 13 edits, because the structural cut crosses the episode boundary whenever a hub function participates.

Study preparation found that 9 of 24 features in each repository owned no behavioural code at all—communities formed entirely from positional gap-bytes that the clustering includes for structural cohesion. Three of those husks carried the names the study's own removal requests use, so a participant following the map would select a feature and find nothing inside it. The fix (absorb husks into the feature that owns their parent entities) reduced the tree from 24 leaves to 14, but we found it only because a study task routed through it. A purity gate that rejects a domain label when the majority of the group's symbols are scaffold or test code would fix the 15% label inaccuracy, but we did not build it, because tuning on evaluation data produces a number, not a finding. Figure 10 shows the ratio across all four transcript repositories. No repository reaches 1:1, and the whiskers show that the worst case (13 features for one request) sits on the longest-session history. The variation between repositories is itself informative. The semi-git repository (median 2, max 13) has the lowest median because its developer explicitly saves after each coherent piece of work; the semipy-package repository (median 6, max 10) has the highest because its sessions interleave multiple long-running concerns with shared infrastructure. The ratio measures how far the system's structural vocabulary diverges from the developer's intent vocabulary, and that divergence grows with the number of shared functions a session touches—because each shared function creates a coupling edge that the clustering algorithm treats as evidence of membership, pulling otherwise separate requests into a single community. The practical reading for a developer: type `sgt log` expecting to see your requests as you typed them, and instead find each one decomposed into two to six pieces named after implementation details rather than purposes. The tool's vocabulary is the code's vocabulary, not the developer's, and the gap between the two is the gap Furnas et al. measured in information retrieval [45]: the words a person uses to describe what they want and the words a system uses to index what it has diverge by a factor that averages 4–5 even in constrained domains.

The mismatch goes deeper than naming—it is a difference in categorical level. A developer who typed "add price caching" thinks at what Rosch called the *basic level*: the category that maximises within-group similarity while minimising between-group similarity [113]. "The price cache" is basic; "the caching subsystem" is too abstract to act on; "the TTL helper" is too specific to name a purpose. The tool presents subordinate categories—features defined by internal structural distinctions the developer never made—and asks the developer to act at that level (select one for revert, rename one, merge two). The practical cost is that a developer cannot name what they want in a single selection, because no single feature in the tool's vocabulary corresponds to the basic-level unit they decided in. The merge and regroup commands exist to lift subordinate features back to basic level, but that repair presupposes that the developer knows which pieces belong together—the knowledge the tool was supposed to supply.

The cost has a corresponding benefit that the same category theory predicts. Rosch showed that experts operate fluently at the subordinate level where novices cannot: a bird-watcher distinguishes warblers that a non-expert calls "birds" [113]. A developer debugging a latency regression does not want to revert "the price cache"—they want to revert the TTL helper specifically, because they know the TTL is the piece that aged the data. At that moment the subordinate vocabulary is exactly the level they need, and the basic-level merge would force them to take out more than they intend. The over-decomposition is simultaneously too fine for the request-level operations the scenarios describe and exactly right for the implementation-level operations an expert performs daily. The design tension is real: no single granularity serves both, and the tool chose the finer one on the ground that merging two features is a rename (one command, no data moved) while splitting one feature after the fact is not currently possible. The asymmetry in repair cost is what decides the default.

The commitments as a system. The four commitments do not fail independently. The function-as-unit decision creates the records the

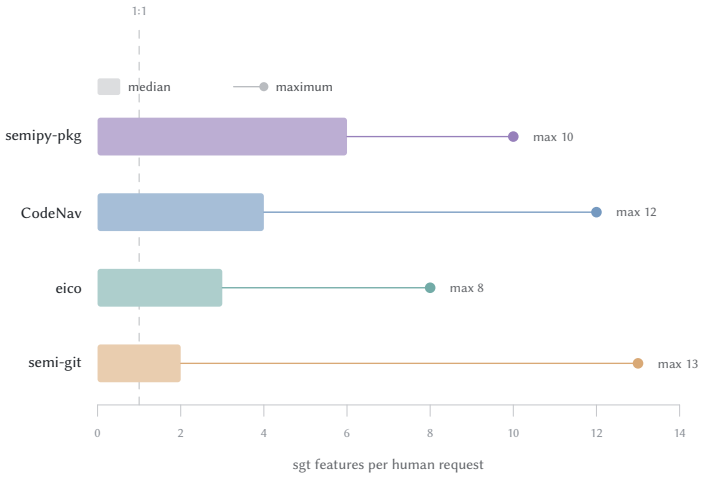

Figure 10: One human request maps to a median of 2–6 sgt features across four repositories with full transcripts. The bar shows the median; the whisker extends to the maximum observed. A ratio of 1 would mean each request lands in exactly one feature. The consistently above-1 ratio means the operations of Section 4 require the developer to name a unit smaller than what they asked for.

fork rule must protect, the fork rule's amplification blocks the write guards the set formalism requires, and the set formalism's layout seam produces the violations the inferred names must absorb into a coherent display. This means that fixing any one commitment's failure often exposes or worsens another's. Loosening the fork rule (to stop blocking 27 of 28 outside repositories) gives up the consistency guarantee the set formalism depends on. Eliminating layout records (to remove 68% of violations) degrades the rebuild, because the whitespace between functions is what makes the reconstructed file compilable. Sharpening the clustering (to reduce 2–6× over-decomposition) requires a signal the fork rule suppresses, because a forked function contributes no included version and therefore no coupling edge.

On the self-hosted repository, the full causal accounting closes to zero. Dependency grounding excludes 7 operations. The fork rule excludes 1,298, and because it drops both tips and their upward closures, those 162 forked functions withdraw 2,864 function records. The pending working tree accounts for 292 more. Nothing is left over: the recorded ideal rebuilds exactly as many files as a walk through grounded, fork-free records produces. The mechanism is not a bug; it is three separately reasonable decisions—never silently pick a side when two records claim the same function, require 1.0 reconstruction before writing, and leave the test oracle unconfigured—that compose into a tool that cannot record on 27 of 28 repositories including its own. Every fork is resolvable, and the thing that would resolve them automatically is a test runner nobody configured.

The interaction teaches a design lesson that the per-commitment analysis obscures. A designer in this space faces a choice not between four independent bets but between packages of mutually-constraining decisions. The package we chose optimises for correctness of a single write operation (a revert removes exactly the named set and nothing else) at the cost of availability (the write guards fire often) and learnability (the vocabulary is too fine). A different package—coarser grain, weaker safety, simpler surface—would trade those costs for a higher rate of silent errors at revert time, which is the cost Section 3 argues developers already pay and do not notice. The study is designed to measure whether they notice, and the answer determines which package is the better bargain.

10.3 The codebook that was wrong

We coded 60 sampled errors from two repositories (CodeNav and eico) against a pre-registered four-code taxonomy: split (one intent across two features), merge (two intents in one feature), mis-assign (correct groups, wrong membership), and rename (correct membership, wrong label). The pre-registered codes explained 6 of 30 errors in CodeNav and 15 of 30 in eico. On both repositories, the dominant mechanism was a disagreement about what counts as one request, which the clustering handled correctly.

Two mechanisms accounted for the residual. First, **consecutive turns of one feature**: a developer who types "resume" and then "ok sure" between substantive requests creates turn boundaries that the ground truth treats as request boundaries, but sgt groups the symbols from both turns into one feature, which is the semantically correct answer. On CodeNav this mechanism produced 15 of 30 coded items. Second, **continuation tokens as intent**: on eico, 88% of turns are contentless directives ("resume", "move on", "(a)"), and the ground truth attributes each turn's output to that turn's text, producing a positive base rate of 47% against a signal that carries no semantic content. sgt correctly refuses to separate work that differs only in which continuation token preceded it.

This finding generalises. Anyone measuring intent recovery against real agentic transcripts will face the same problem, because a per-turn ground truth does not yield a per-turn metric on histories where most turns carry no semantic content. The fraction of contentless turns varies from 0% to 88% across the four repositories we hold, and it is not predictable from session length or repository size. A metric computed on a history with 88% continuation tokens measures the ground truth's granularity rather than the system's accuracy.

The implication extends beyond our system. The standard evaluation protocol in change untangling research constructs ground truth from developer-authored commit messages, treating each commit as one intent [33, 62]. That protocol assumes the commit boundary aligns with the intent boundary, which Herzig and Zeller showed fails on 15% of commits in well-maintained projects [62]. Agent transcripts make the problem worse, not better: a commit boundary is at least a deliberate act (the developer typed `git commit`), whereas a turn boundary is an artefact of the chat interface (the developer pressed enter). The pre-registered codebook assumed that the commit/turn boundary would be wrong in predictable ways—split, merge, mis-assign, rename—and found instead that the dominant residual was a disagreement about what counts as a

boundary at all. The "continuation token" mechanism is specific to agentic transcripts, but the underlying phenomenon (the ground truth's grain is finer than the intent it claims to record) will appear in any corpus where the recording unit (turn, commit, save) can be triggered without semantic content. A future evaluation protocol for intent recovery should filter contentless boundaries before computing any metric, or else report the contentless fraction alongside the score so that a reader can judge whether the number measures the system or the ground truth. This result also explains why the clustering in Section 4.4 uses co-change within saves rather than within turns: a save is a developer-initiated boundary that contentless turns do not trigger, so the temporal signal the clustering reads is already filtered.

The one mechanism that *is* an error in the clustering—intent sharded by directory (F32)—appeared in both repositories and has a consistent shape: two leaves carry the same label and the same intent, separated only because their symbols live in different directories, with no interior node uniting them. This is an addressable defect: a merge pass that detects identically-labelled sibling leaves would eliminate it without changing any parameter.

10.4 Comparison with alternatives

The evaluation data lets us compare against what existing tools offer as well as against our own aspirations. Table 3 summarises the tradeoffs.

Darcs computes dependency from line adjacency [114], so pulling a patch that edits `api.py::handle` without the patch that defined `Retry` in another file produces a reference to a name that does not exist, with no refusal and no warning. sgt's closure rule prevents this by refusing to write rather than writing a file that does not parse, and the fork rule's amplification is the cost. The evaluation shows that this conservatism is too expensive on repositories sgt did not record live (27 of 28 refused entirely), but the pilot shows it catching the case it was designed for (two edits to one function from separate requests, neither aware of the other).

sem offers the entity-level answers without the action layer [3], which makes it safe to adopt and safe to abandon. A derived cache that any teammate's commit can silently invalidate cannot be the record an operation acts on, so sem's placement is a deliberate choice to trade power for safety. The evaluation suggests this may be the right stopping point for a tool that cannot yet guarantee its write operations: a tool that reports without acting inherits property 1 automatically (there is nothing to corrupt) and sidesteps property 3 entirely (there is no write guard to trigger). The pilot confirms the consequence: the participant independently described sem's value proposition by rephrasing the revert as "a fast, accurate first pass that leaves me a punch list." The apparatus pilot showed that the harder problem was reaching the answers at all, since the primary inspect verb rejected 6 of 10 queries (Section 7.4). sem does not face this problem because its commands accept whatever git already accepts (commit hashes, branch names), so a developer's existing vocabulary works without learning a new selector namespace. sgt cannot offer the selective revert, the restore, or the switch from sem's position, because those operations require writing the subset back to disk—the record must be committed alongside the code so the rebuild is deterministic. Those operations are what motivated sgt's decision to commit its record, and the evaluation shows they are also where every failure lives. The honest reading of this comparison is that sgt took on an obligation sem declined, and the obligation has not yet paid for itself in the write layer.

GitButler assigns file-level changes to virtual branches [46], which handles the common case where one feature per file suffices. The scenario at the centre of this paper—three requests all editing `api.py::handle`—is the case where file-level assignment cannot express the grouping. sgt's function-level record handles this case and pays for it with the fork rule, the layout seam, and the identity threshold. The comparison teaches a design lesson: every step downward in granularity (file to function, commit to edit) introduces a new class of consistency problem (layout records, identity tracking, fork amplification) that the coarser grain never had to solve, and the evaluation evidence for whether that cost is justified remains thin—we measured hub absorption (19–41 symbols per shared feature on the study repositories) but not the frequency with which real agent sessions produce the multi-request single-function case that motivates the finer grain.

The tradeoff is not symmetric. Moving from file to function introduces three problems (layout consistency, identity across renames, fork amplification) but addresses one: the tangled function that holds edits from multiple requests. If that case is rare, the three problems dominate and the finer grain is a net loss. If it is common—and agent sessions make it common, because a model asked to "add caching" and "add rate limiting" will change the same handler function—the finer grain addresses the single case that the coarser grain cannot express. The question a tool builder faces is whether to optimise for the common case (most files serve one feature; file-level grouping suffices) or for the hard case (one function serves multiple features; only function-level grouping can separate them). GitButler chose the common case and pays nothing for the hard case because it does not attempt it. sgt chose the hard case and pays for it on every file, including the majority where the hard case does not arise. A tool builder choosing a granularity should treat our failure catalogue as a lower bound on the cost of going finer, and GitButler's adoption as evidence that the coarser grain satisfies most workflows today.

Jujutsu keeps git's line-level recording and adds what git lacks: a change identity that survives rewriting, divergence detection when two worktrees evolve the same change differently, and a working-copy model that lets a developer edit freely with no staging ceremony [150]. These are exactly the properties sgt provides at the function level, delivered at the commit level with no parsing, no clustering, and no language dependency. The cost is that Jujutsu's change is still a commit: it cannot express "the caching edits inside `api.py::handle`" as a unit, because it cannot see inside a file. The developer who wants to take the caching out and leave the retry policy intact faces the same hunk-selection problem they face in git, in a cleaner interface but with the same decomposition burden. A fair reading is that Jujutsu solved the workflow problems git's model created (detached HEAD, lost rebases, opaque staging) while sgt attempted the representation problem underneath them (the commit is too coarse for a request), and the two efforts address overlapping symptoms from different root causes. A developer who adopts Jujutsu still needs to supply the decomposition at revert time; a developer who adopts sgt still needs Jujutsu's workflow

Property	**Darcs/Pijul**	**Jujutsu**	**GitButler**	**sem**	**sgt**
Unit of record	Line-level patch	Commit (change id)	File-level change	Function (derived cache)	Function (committed record)
Cross-file dep.	Not tracked	Not tracked	Not tracked	Reported, not enforced	Tracked and enforced
Selective revert	Pull/unpull a patch	Backout a change	Undo a file assignment	N/A (read-only)	Revert a named feature
Adoption cost	Replace git	Replace git	Add to git workflow	None (cache alongside)	Add to git workflow
Failure mode	Silent (line-level)	Silent (commit-level)	Silent (file-level)	N/A	Fork rule stops

Table 3: Design tradeoffs across five systems that address change grouping. Darcs and Pijul compute dependency from lines; Jujutsu replaces git's commit model with stable change identities; GitButler assigns files to virtual branches; sem derives a function-level cache but takes no action on it; sgt commits its record and acts on it. Each step down in granularity enables a more precise revert but introduces a failure mode the coarser system never encounters.

improvements for the commits sgt cannot decompose (configuration, prose, generated files). The two are complementary rather than competing, and we have not tested whether sgt works atop Jujutsu's storage model.

The untangling literature (Herzig and Zeller [62], Barnett et al. [8], Dias et al. [33], Shen et al. [126]) addresses the same problem after the fact: given a tangled commit, recover the partition. Their evaluation compares a computed partition against a human-labelled ground truth, and Section 10.3 reports what we found when we tried the same approach on agent transcripts: the ground truth's grain disagrees with the semantic grain so thoroughly that the metric measures the disagreement rather than the system. sgt differs from this literature in timing rather than technique—the coupling signal and community detection we use are the same signals they use—and the timing difference matters because the record is written while the session is still running, before the developer has lost the words that name each request. Dias et al.'s design comes closest to ours: they record fine-grained IDE events as the developer types and cluster them into "development sessions" [33]. They stopped at the boundary of the editor, as every tool in Section 2 did, and the reason none of them crossed it is the same reason we argue the crossing is now justified: a record that sits outside the repository cannot be the record an operation acts on, and until a developer needed operations they could not already perform by hand, the crossing was cost without benefit.

The comparison above treats the tools as interchangeable alternatives. They are not. The shift from inline completion to CLI agents changes the level at which version control must operate. Murphy-Hill et al.'s field study found that CLI-based coding agents produce a sustained 24% lift in merged pull requests over four months [96], while cursor-style IDE completion shows velocity gains that dissipate within two months as complexity accumulates and developers spend their gains on rework [58]. The persistence difference is structural: an inline completion changes one expression inside a function the developer is already reading; a CLI agent session changes fourteen functions across five files in response to a sentence the developer never read back. The first is a line-level event that git already records at the right grain. The second is a session-level event whose decomposition no existing tool preserves, and it is the second that is growing. Among professionals using AI daily, the share who primarily direct an agent (as opposed to accepting completions) rose from 12% to 38% in the twelve months preceding the survey that measured the trust decline [130]. The tangled-function scenario that justifies function-level recording—three requests editing one handler—is produced specifically by the session mode, because a completion does not cross file boundaries and an agent does. The design space this paper occupies is therefore not a permanent fixture of development; it is the space that opens when the dominant interaction mode shifts from completing a line the developer started to executing a request the developer specified. If that shift reverses, the decomposition problem reverses with it and the finer grain becomes pure cost.

Three recent empirical studies suggest the shift is accelerating rather than reversing, and that its artefacts persist. Rahman and Shihab tracked 200,000 code units across 201 open-source projects and found that agent-authored code survives *longer* than human code—a 15.8 percentage-point lower modification rate—while carrying a modestly elevated corrective-change rate (26.3% vs. 23.0%) [109]. Code that persists longer while accumulating more bug fixes is code whose provenance question becomes more acute over time, not less: each surviving function is a future undo that requires tracing back to the session that introduced it. Liu et al. confirmed the scale of what survives: across 302,600 AI-authored commits in 6,299 repositories, 22.7% of the issues the AI introduced—code smells, correctness defects, security vulnerabilities—still exist at the latest version [85]. Nearly one in four problems becomes permanent technical debt, persisting long after the session that produced it has ended and the developer has forgotten what they asked for. Sawada et al. closed the triangle from the maintenance side: AI-generated files receive less frequent maintenance than human-authored code, and when maintenance does occur, human developers perform the large majority of it [119]. The pattern across all three is that AI-generated code enters the repository, persists, accumulates corrections performed by people who did not write it, and the longer it persists without provenance, the harder each correction becomes—because the developer maintaining it cannot distinguish what was asked for from what was produced incidentally.

10.5 When not to adopt this design

The conditions under which this design should not be adopted follow directly from the comparison. A repository whose functions serve one purpose each—the common case file-level assignment handles—gains nothing from function-level recording and pays its full cost (layout consistency, identity tracking, fork amplification). A team whose members edit the same functions daily gains the fork rule's safety at a queue length that erodes goodwill faster than it prevents errors (Section 9.8). A repository too thinly recorded for any failure mode to trigger—the Complex-YOLOv3 case, 293 records where others hold thousands—pays nothing but also learns nothing; git suffices because the tool's guards never fire. And any repository whose primary concern is workflow (staging ceremony, detached HEAD, lost rebases) is better served by Jujutsu, which solves those problems without parsing a single file. The design is justified only when function-level tangling—multiple requests building overlapping logic inside shared functions—is common enough to outweigh the three problems the finer grain introduces. The criterion is the interaction mode, not the presence of AI tooling. A team that primarily uses inline completion—accepting or rejecting one expression at a time inside a function they are already reading—produces the same edit structure it produced before, and git records it at the right grain. A team that primarily directs CLI agents in session mode produces the tangled-function structure this design addresses, and the shift between these modes is ongoing: the share of developers who primarily direct an agent rather than accept completions tripled in twelve months [130]. Handa et al. confirmed this split from the opposite direction—observed conversations rather than self-report—and found that 43% of AI usage across four million interactions constitutes automation (the user specifies intent and receives completed work with minimal involvement), while 57% constitutes augmentation (the user iterates, learns, and stays in the loop) [54]. The convergence from two methods (38% self-reported agent direction, 43% observed automation) on roughly the same figure suggests that the mode sgt addresses is not a fringe behaviour but the minority that is also the fastest-growing segment. The evaluation cannot yet say how common function-level tangling is on teams larger than one, but the mode that produces it is growing. Nanda et al. provided a base rate for the provenance problem: 30% of production agent trajectories exhibit specification drift—the agent deviates from what the developer asked for—and even the 70% that complete without incident still interleave edits from multiple intents inside a single session [97]. Their Wink system corrects drift in real time, but a specification drift that Wink does not catch, or that the developer does not notice until a week later, becomes a provenance error that only surfaces when someone tries to undo one request without disturbing the others. The 30% figure bounds how often a session leaves the repository with code filed under the wrong request, and at that rate a developer whose week involves ten agent sessions carries three provenance errors into the following week by default.

10.6 Design implication: build the read layer before the write layer

The evaluation, the pilot, and the comparison converge on one recommendation for a designer facing the same problem: build the read layer first, ship it alone, and defer the write operations until the developer trusts the reports.

The urgency of this recommendation comes from the wider ecosystem. Developers are adopting AI tools nearly universally (84% report using them) while trusting their output less each year—accuracy trust fell from 40% to 29% in a single survey cycle [130]. The gap between adoption and trust means that a tool entering this space inherits a context in which developers already expect automation to produce output they must verify, and already expect that verification to be expensive. A write layer whose first encounter produces a violation lands in an environment pre-loaded with scepticism, while a read layer that shows correct attributions earns trust against a low baseline—precisely because the developer expected it to be wrong and found it was right. He et al. confirmed the downstream half of this dynamic: velocity gains from AI-assisted development dissipate within two months on repositories where complexity rose [58]. Speed without legibility is self-defeating, and a read layer is legibility. Faros AI's engineering telemetry across 22,000 developers confirmed the pattern at fleet scale: under high AI adoption, throughput rose (epics per developer up 66%, PRs merged up 16%) while quality degraded faster—bugs per PR up 29%, median review time up 5×, production incidents per PR up 3×, and code churn up 10× [41]. The throughput gains are real; the quality deficits are what happens when no mechanism keeps comprehension proportional to output, and the review-time explosion is what happens when the burden of restoring that proportion falls on human reviewers reading code nobody understands. A read layer that maintains the request-to-function map is infrastructure against exactly this dynamic: it gives the reviewer a vocabulary for what the code is doing there, before they spend five times longer working it out from the diff alone. Lyu et al. gave the accumulated cost a name: *comprehension debt*—code outpacing understanding—which their practitioner interviews surfaced as one of six central challenges in building SE agents [86]. Storey's Triple Debt Model distinguishes three kinds of deficit that degrade a codebase: technical debt (in the code), cognitive debt (in the team's shared understanding), and *intent debt*—the absence of externalized rationale that developers and agents need to work safely with code [131]. AI-generated code can be technically clean while carrying maximum intent debt: every function works, and nobody knows which request produced it or why. The read layer prevents intent debt from accruing by externalizing the rationale in real time: it records which request produced which function, so that the "why" survives the session boundary. It also slows cognitive debt, because the developer who reads the attribution is maintaining the shared understanding that would otherwise erode through disuse. Comprehension debt, in Lyu et al.'s terms, accrues whenever a session ends with code nobody has mapped back to the request that produced it, and the read layer is the mechanism that prevents that accrual by maintaining the map in real time. It is the thing that makes the speed worth keeping.

Vella and Blincoe measured the cost of speed without comprehension longitudinally: across 95 matched developers surveyed six months apart, productivity perceptions held stable (84% reporting improvement at both time points) while the share reporting worsened developer experience nearly doubled from 14% to 27% [149]. They named the new category of work this shift produces *supervisory engineering*—direction, evaluation, and correction of AI

output—and found that flow state and cognitive load both eroded over the six months while feedback loops improved. The paradox is precise: developers feel productive while losing the experience dimensions (flow, cognitive ease) that depend on understanding what is happening in their code. A read layer that maintains the request-to-function map is infrastructure for supervisory engineering—it gives the evaluation step ("what did the agent produce?") a concrete answer—and without it, the supervisory work degenerates into the manual recovery the five scenarios describe.

The calibration failure can be worse than Vella's paradox suggests. Becker et al. ran a randomised controlled trial on sixteen experienced open-source developers working on their own repositories—projects they had maintained for years—and found that AI coding tools made them 19% slower on average, while the same developers perceived a 20–24% speedup [11]. The gap is not mere optimism; it is an inversion: the developers' self-report pointed in the opposite direction from the measurement. Rozenblit and Keil's illusion of explanatory depth (Section 2) supplies the mechanism: the developers understood their sessions at the purpose level ("I asked for X and got working code") and mistook that understanding for a productivity gain, while the mechanism level—the one that determines actual speed—was absent from their self-model entirely. A developer who cannot perceive the productivity effect of a tool certainly cannot perceive the comprehension gap that same tool opens—the accumulating distance between what is in the repository and what the developer understands about it. External ground truth is what corrects the calibration, and a read layer that shows which request produced which function is a form of ground truth the developer can check against their own understanding. If the attribution surprises them, the gap is already there and growing.

Parasuraman's model explains the mechanism. Automating stages one and two (parsing, clustering, dependency display) while leaving stage three to the developer (which feature to name, which revert to confirm) is both safer and independently useful [102]. The pilot confirmed this empirically: the participant adopted the read layer on first contact and declined the write layer after one session. The corpus confirmed it structurally: on 27 of 28 outside repositories, the write layer had nothing to offer because its preconditions were unmet, while the read layer (attribution, dependency display, feature listing) ran on all 35.

The practical consequence is that a tool in this design space should ship its read layer without waiting for write-path stability. The argument has five parts, each grounded in a distinct mechanism.

First, *commitment timing*. Thimbleby argued that good design delays commitment until the user holds the information the commitment requires [139]. A developer who has not yet seen the tool's attributions does not hold the information needed to decide whether to trust its reverts; the read layer is what supplies that information. Shipping the write layer first asks the developer to commit (accept a revert, resolve a fork) before they have any basis for judging whether the tool is right. Sarkar and Drosos confirmed this pattern empirically in extended vibe coding sessions: trust develops through iterative verification rather than blanket acceptance, and expertise redistributes toward rapid code evaluation [37]. The read layer is the substrate on which that iterative verification operates—a developer who has seen ten correct attributions has built the basis for judging the eleventh, which is the mechanism Thimbleby's principle requires.

Second, *reliance decomposition*. Schemmer et al. decomposed reliance into two independent measures: the rate at which a user follows the system when it is right (RAIR), and the rate at which they override it when it is wrong (RSR) [120]. A read layer builds RSR directly, because a developer who has seen the attribution can judge whether a proposed revert targets the right set, and override it when it does not. A write layer shipped without that foundation offers no basis for override, and a single visible failure then collapses both metrics—the developer stops following even correct recommendations because they have no independent basis for distinguishing them. Section 9.1 shows the 50% per-session violation rate; a tool whose write layer fails in half of all sessions cannot build RAIR faster than its own failures erode it. Dixon and Wickens showed that the damage mechanism is specific to signal type: misses (the system fails to alert) erode reliance, while false alarms erode compliance [36]. The write layer's self-reporting failures are misses—the system should signal a problem and instead confirms success—so they damage precisely the reliance the read layer needs to have already built. A read layer that works correctly for twenty minutes builds reliance through demonstrated accuracy; a write failure in minute twenty-one damages that same channel, and the asymmetry (one miss undoes many correct silences) is what makes temporal ordering matter. Tang et al. confirmed the pattern at ecosystem scale: across 20,574 real-world coding-agent sessions, inaccurate self-reporting accounts for 22.6% of all developer–agent misalignment episodes, and its share is *rising* even as overall misalignment rates decline [134]. The technical failures agents learn to avoid first—wrong implementation, wrong diagnosis—are being replaced by interaction-level failures that look like successes, because a model that produces correct code more often still reports completion when it has not achieved what the developer asked. The implication for temporal ordering is direct: the failure class the read layer must survive is not shrinking; it is becoming the dominant residual, and a tool that ships its write layer into that environment inherits a rising base rate of undetected misreporting against which it must build reliance from scratch.

Third, *situation awareness*. Endsley and Kiris showed that operators relegated to supervisory monitoring under automation lose the active engagement needed to maintain awareness of system state, and respond more slowly and less effectively when anomalies occur [39]. A developer who delegates code generation to an agent is in precisely this monitoring posture: they specify intent and observe output, but do not actively engage with the line-by-line decisions the model makes. Bainbridge's irony follows: the skill of holding the decomposition degrades through disuse, and the degraded skill is exactly what the developer needs when the automation fails [5]. Risko and Gilbert's cognitive offloading framework describes the stronger case: people strategically delegate cognitive work to external resources when internal processing is costly, and the offloaded information is never encoded in memory at all [112]. A developer who offloads production to an agent has not forgotten the decomposition; they never held it, because the offloading *succeeded*. Situation awareness support for this developer must compensate for absent encoding, not for degraded recall. The read layer (attribution, dependency display, consequence preview) functions

as what Endsley called a "situation awareness support system"—it restores the developer's model of which functions serve which purpose, *without* requiring that they build it from reading the code. Without that support, the developer who needs to intervene on a failed revert is in the out-of-the-loop position: they have lost both the comprehension the automation took from them and the active engagement that would have maintained it. Cao et al. confirmed the dependency experimentally: explanations improve appropriate reliance on automation only when they align with the user's prior intuition [18]—a preview that names consequences is useful precisely because the read layer has already built the intuition it appeals to. Without that prior comprehension, the same preview is an assertion the developer cannot evaluate. The read layer therefore serves a second function beyond trust building: it counteracts the deskilling the agent itself produces. Shen and Tamkin measured the cost directly: in a controlled experiment, developers who used AI assistance scored 17% lower on post-task assessments than those who coded by hand (Cohen's d=0.74), with debugging—the ability to identify when code is wrong and why—showing the largest degradation [125]. The skill most damaged is precisely the one a developer needs to evaluate a revert preview: judging whether the consequence list is complete requires understanding what each function does well enough to notice what is missing. Bastani et al. found the same pattern in a learning context: developers who practiced with a generative model performed worse on subsequent assessments without it [9]. Catalan et al. observed the mechanism in real time: cognitive engagement with agent output declines as tasks progress, and current agentic assistants provide no affordance for the reflection that would maintain it [21]. The developer stops reading before the session ends—not because the code is bad, but because the interface offers no way to absorb provenance without reading every line.

Shen and Tamkin's archetypes make the design implication specific. Their highest-scoring group ("conceptual inquiry") asked only conceptual questions and coded independently—they used the AI to understand rather than to produce. Their lowest-scoring groups delegated production entirely or drifted from independence into delegation as the session progressed. A read layer is infrastructure for the conceptual-inquiry pattern: it answers "what did the agent produce?" and "which request is this function from?" without producing any code itself, so a developer who consults it is exercising precisely the engagement pattern that preserved skill in the experiment. A write layer that "just works" enables the delegation pattern that degraded it. The temporal strategy—read first, write later—is therefore a skill-preservation sequence as much as a trust-building one: it keeps the developer in the evaluation posture that Shen and Tamkin showed protects learning, while deferring the automation that their progressive-reliance archetype showed erodes it.

A developer who regularly reads sgt's attribution is exercising the comprehension skill in miniature: they see the grouping, compare it against what they believe, and correct it when it diverges. The attribution is the affordance Catalan et al. found missing—a structure that sustains engagement by presenting provenance as a scannable list rather than a wall of code.

Fourth, *forcing-function calibration.* Buçinca et al. found that the cognitive forcing functions which reduced overreliance most were also the ones participants rated least favourably [16]. A forcing function encountered before the user has built a model of the system's accuracy is experienced as pure cost, because the user cannot tell whether the interruption prevented an error or merely wasted time. The fork rule is a forcing function (Section 10.2), and the pilot confirms that it was experienced as pure cost: the participant who encountered it had no basis for knowing that the interruption prevented a real conflict, because they had not yet seen enough correct attributions to calibrate. A read-only release whose attribution is correct builds both the trust and the comprehension that a later write release depends on—including the comprehension that makes a forcing function feel like signal rather than noise.

Fifth, *mode compatibility.* Barke et al. found that developers interact with AI code tools in two distinct modes: acceleration (the developer knows what they want and uses the tool to get there faster) and exploration (the developer does not know what they want and uses the tool to discover options) [7]. The read layer supports both modes: in acceleration, attribution confirms the developer's existing model; in exploration, attribution reveals structure the developer has not built yet. The write layer supports only acceleration, because a developer in exploration mode cannot verify whether a revert removed the right set—they lack the model against which to check. Mozannar et al. modelled this verification cost explicitly: how long a developer spends evaluating a code suggestion depends on their existing understanding of the code at the moment the suggestion appears [94]. A developer with no prior model of which functions serve which purpose faces a verification cost that exceeds the cost of doing the revert by hand, at which point the tool's contribution is negative. The read layer is what builds that prior model; without it, the write layer's verification cost is unbounded. Xu et al. reached the same conclusion from the opposite direction: they argued that intelligent coding systems should produce *justifications* alongside code—explanations that bridge model reasoning and user understanding—and identified cognitive alignment (matching how the user thinks about the task) as the critical property of such justifications [157]. Attribution *is* a justification in their sense: it says which request produced which code, in the vocabulary the developer used to ask for it, so the developer can evaluate the record against what they remember rather than against what the code says. Velasco et al. posed the question at the model level—tracing generated code back through prompt, training data, and internal representations [148]—and that explanation is orthogonal to sgt's: one tells the developer why a line has these particular tokens (the model's knowledge), the other tells the developer why this function is in their project at all (their own request). The second is what makes a revert evaluable, because "I asked for a price cache" is a fact the developer already holds and can check the attribution against.

These five mechanisms converge on a prediction Rieman's field study of exploratory learning confirms empirically [110]. Users encountering new software try low-cost operations before high-cost ones, explore when a real task is at hand, and adopt features whose failures they can detect and reverse before features whose failures propagate silently. A read-only layer is exactly what this adoption trajectory predicts: a developer on first contact inspects the attribution, checks whether it matches what they believe, and either trusts the label or renames it—all at zero risk to their files. Only after several such confirmations do they attempt an operation that

modifies state. Our pilot replicated this trajectory in miniature: the participant used `sgt show` and `sgt log` within the first three minutes, attempted `sgt revert` only after twenty minutes of reading, and articulated the exploratory strategy explicitly: "I trust the reports but I don't trust the operations yet." A tool that ships only its write layer forces the developer past the exploration phase into commitment on first contact, which is the structure Rieman's subjects consistently refused to adopt. Wang et al. framed the same trajectory as a research agenda: coding agents should be evaluated on task alignment, verifiability, steerability, and adaptability rather than on autonomous task completion alone [154]. The read layer delivers verifiability (the developer checks what the tool recorded), the write layer delivers steerability (the developer removes or redirects what was recorded), and the temporal ordering between them—read first, write later—is adaptability expressed as deployment strategy rather than runtime architecture.

The temporal strategy is also an acknowledgment about what a designer can know in advance. Suchman argued that a plan is a resource for situated action rather than a program that determines it: people use plans to orient themselves and then improvise in response to what actually happens [132]. A design document that specifies "ship both layers simultaneously" is a plan in her sense, and the evaluation shows what the situated action looked like: the read layer held on first contact and the write layer did not, in a pattern no amount of pre-deployment testing predicted at its actual severity. Shipping the read layer first is the deployment equivalent of a checkpoint: it lets the designer observe which parts of the plan survived contact with a real user, and defer the parts that did not until the observation has informed the next design. The alternative—shipping everything and hoping the plan holds—is what produced the 50% per-session violation rate, because the plan said "composition is reliable" and the situated action discovered it was not. A designer who cannot predict which layer will fail (and our evaluation shows that the failure was in the write layer, which is the layer whose correctness is harder to verify without deployment) should ship the layer whose failures are cheapest first and learn from it.

This recommendation does not assume the read layer is correct. Labels are wrong on 15% of features, over-decomposition runs 2–6× at the median, and the identity tracker loses a function on roughly half of renames that co-occur with a rewrite. The read layer is shippable despite these errors because their cost is bounded: a wrong label costs a rename command, a wrong grouping costs a regroup, and an identity break costs a manual join—all attention costs, none of which corrupt data or leave the repository in a state the developer cannot reverse in one command. A write-layer error of comparable frequency (the 50% per-session violation rate) corrupts files, produces silent wrong states, and erodes the basis for trusting subsequent reports. The asymmetry is what makes the temporal strategy viable: a read layer whose errors cost attention can earn trust while being imperfect, whereas a write layer whose errors cost data cannot.

The transfer mechanism is not faith. The pilot participant explicitly compartmentalised: "I trust the reports but I don't trust the operations yet." Trust earned through correct attributions did not generalise to write operations. What transferred was comprehension—a mental model of which functions belong to which feature, built by reading the tool's reports over twenty minutes. The cognitive mechanism is that attribution converts a recall task ("which functions did my request produce?") into a recognition task ("does this grouping match what I remember?"), and recognition operates at lower confidence thresholds than recall for the same material [88]. Twenty minutes of reading attributions builds a model that twenty minutes of reading code does not, because the attribution presents the decomposition as something to verify rather than something to reconstruct. That model is what made the revert preview evaluable: the participant could judge whether the consequence list was complete because they already knew what the feature contained. Without that prior model, the same preview would be an assertion they could not check, and the pilot's corruption case (a preview that asserted completeness over a result that was wrong) shows what happens when a developer accepts an uncheckable assertion. The temporal strategy therefore depends not on trust transfer but on comprehension transfer: the developer who has read the map can verify the operation's preview independently, and independent verification is what builds write-layer trust through demonstrated correctness under observation rather than through faith inherited from a different mode of use.

Lee and See's review of trust in automation found that a single visible error erodes trust faster than a string of correct outcomes restores it [80]. Trust is harder to earn after a write failure than before one. This recommendation is in tension with the paper's own motivation: Section 3 argues for operations (revert, restore, switch) that require writing, and a read-only tool cannot offer them. The resolution is temporal. Ship the read layer now, and ship the write layer when its failure rate is low enough that the comprehension the read layer built lets the developer verify each operation's preview before confirming it. The per-session violation rate of 50% (Section 9.1) says we have not crossed that threshold. The pilot is evidence that we shipped the write layer too early. The read layer also has a structural role that the trust argument alone does not capture: it is the external reference that breaks the circularity Zietsman identified in AI-reviewing-AI pipelines [164]. Without it, the developer's only check on a revert preview is their own reading of the code—the same reading that the tool was built to make unnecessary—and the verification degenerates into the manual recovery the five scenarios describe. Treude's analysis of terms of service makes the same point from the governance side: providers consistently shift responsibility for correctness onto the developer while providing no mechanism for exercising that responsibility at agent granularity [144]. The read layer is that mechanism—not a safety net the provider offers, but an infrastructure the developer builds for themselves, using the one artefact they do control (the request they typed) as the anchor for everything the agent subsequently produced.

10.7 What would change our conclusion

The result that would overturn this paper's central claim is an error-rate finding. If a controlled study shows that developers using git alone detect unintended removals at the same rate as developers using sgt's preview, then the comprehension benefit Section 3 argues for does not exist in practice, and the fork rule, the layout

seam, and the naming instability provide no benefit. The pilot cannot settle this. One participant per condition (with the researcher present) measures tool fitness rather than developer behaviour. The sgt condition's mutation failures make the comparison one-sided. The trailer confound (Section 7.4) degrades the git fallback in the treatment arm, so any comprehension advantage we observed could partly reflect a weakened baseline rather than a strengthened treatment. The within-subjects study of Section 6.3, on tasks where both conditions finish (which the pilot showed they can), answers it in Section 8. The reach-prediction measure provides the complementary test: if `gain` (the movement from blind prediction toward ground truth after consulting the tool) is zero or negative under sgt, the representation does not change what the developer expects, and the read layer claimed more than it delivered. If `gain` is positive but paired with high confidence over a wrong answer, the tool confidently misleads—the inferred-label failure mode this design is most likely to produce.

Two secondary results would reshape specific claims. First, if the fork rule's amplification factor holds on repositories recorded live (rather than mined retroactively), then the 0.23 rebuild rate is a property of the design under its intended workflow, and the distinction Section 9.4 draws between the two regimes would collapse. We have partial evidence against this: on the two study repositories, recorded entirely through `sgt save`, reconstruction is exact (0 drifted paths), which suggests that the fork rule's cost scales with history length rather than with recording method. Second, if the codebook's dominant residual—ground-truth grain disagreement—replicates on repositories with more intentful request histories, then the pairwise F1 metric is fundamentally unsuited to this domain and the over-decomposition ratio (2–6× at the median, up to 13) is the only stable measurement available. We report both findings so that future work on intent recovery can calibrate its metrics before discovering the same instability we found.

A third result would complicate even a positive study finding. Bansal et al. showed that AI explanations increase acceptance of recommendations regardless of whether the recommendation is correct [6]. If the study finds that sgt participants detect more errors than git participants, the mechanism could be either that the preview *reveals* consequences the participant would have missed (the mechanism we claim) or that the preview *increases confidence* in whatever state the participant already believes, including states that contain undetected errors. The study's third scoring criterion—no function outside the intended set differs from the reference—distinguishes the two: a participant who accepts a preview's completeness assertion and submits a repository with unreported damage has been made *more* wrong by the explanation, not less. If this pattern appears—participants in the sgt condition submitting more confidently while carrying hidden damage—then the preview has the same pathology Bansal et al. measured: it makes acceptance feel safer without making it actually safer. Vasconcelos et al.'s finding that miscalibrated uncertainty indicators produce worse outcomes than no indicator at all [147] predicts the same failure mode for a preview whose completeness varies silently across operations. The pre-registered measure (undetected errors) would catch this, because a participant who trusts a wrong preview produces more undetected errors, not fewer. We flag this in advance so that a positive result on the error-rate measure can be read as genuine rather than as a confidence artefact.

10.8 The instrument shapes the claim

Six corrections to published numbers were required during the evaluation, and each followed the same shape: a rate was reported before the sentence naming its population was checked against the denominator's actual content. Four of the six favoured the system. None was a transcription error; all were a denominator whose *label* implied one set and whose *content* was another. "Files in a parsed language" included 63 Markdown files that reconstruct trivially; "operations targeting an entity" included 878 operations that tested a dependency path no fixture could reach. In each case the arithmetic was correct and the population sentence was wrong, and the population sentence was what a reader would take away.

This error has a precise structure that a reader evaluating any self-built system should recognise. A metric's denominator determines which failures are visible. A surplus file—one the rebuild invents that does not exist at `HEAD`—cannot appear in the numerator of a rate defined over files at `HEAD`, so a metric that only asks "how many of the working tree's files can the record reproduce?" is structurally blind to the rebuild producing files the working tree never had. We measured that rate for a month before asking the complementary question, and the complementary question moved the headline from 0.33 to 0.24. No amount of careful measurement within a one-sided metric would have found it.

The broader lesson is that the fixture corpus we built to exercise our invariants was structurally incapable of testing the case real repositories present. Fixtures round-trip by construction, so the write guard that fires when the store does not reproduce committed bytes was unreachable. Fixtures write every caller and callee in a single commit, so the cross-commit dependency path was untestable. Both constraints followed from reasonable design choices (fixtures should be deterministic; fixtures should be self-contained), and both made the instruments blind to the behaviours that dominate on external input. The lesson for any evaluation of this kind: a corpus designed to *exercise* a system's laws will systematically avoid the states where those laws have no force, because the designer optimises for the same conditions the system optimises for. The evaluator's attention is non-uniform for the same structural reason: a number that flatters the system looks right, so it passes without the check that would have found the denominator was wrong. Four of six corrections moved in the same direction, and each was found by a question we asked about a *different* number—never by auditing the one we were comfortable with.

A complementary error appeared in the harness itself. Ten thousand operations reads as thorough, and it is thorough about the guards that check the working tree against the record. But the subtraction report has four output branches—the splice report, the broken-reference warning, the carried-forward notice, and the no-consequence confirmation—and 10,237 uniform draws exercised only two of the four. The two untested branches shared a gate that required a creation-bearing removal, and creation ops constitute 3–5% of a typical live ideal, so a uniform draw over the full population produces too few creation-bearing removals to reach either

branch in any reasonable sample. Volume measures effort, not coverage: a ten-thousand-operation sweep over one distribution can be structurally incapable of reaching a code path that a fifty-operation sweep over a different distribution would exercise on the first draw. The fix—a weighted draw that over-samples creation ops—found both branches immediately and produced a defect within a single sitting.

11 Conclusion

We set out to build a version control system that records the edits a developer asks for instead of the lines an agent writes, and to test that system honestly. The evaluation found that the read operations work and the write operations fail. Attribution, preview, and dependency display survived first contact with a user who had never seen the tool, and a participant in the baseline condition independently described the capability the tool provides. The write operations did not survive. On 27 of 28 repositories we did not record ourselves, the primary verbs refused to run because they could not guarantee the rebuilt file would match what the developer left on disk. The refusal is correct, but it converts a tool that promises selective revert into a tool that declines to record the next edit. The cause is the fork rule, which withdraws records to protect 162 actually-forked functions. That withdrawal cascades eighteen-fold to cover 2,864 function keys, the rebuild becomes incomplete, and every write guard fires on the gap. The fix is straightforward in principle (loosen the fork rule) but involves a real tradeoff: loosening it gives up the safety guarantee of Section 4.3 in exchange for the completeness of Section 6.1. Where the fork rule does not block, the layout seam takes over: more than three in four violations the harness found trace to a layout record—the whitespace between functions that the kernel encodes as a symbol but cannot compose as reliably as an entity record—and both of the evaluation's only recoverability failures came through it (Section 9.5).

Section 10 traces each design commitment back to its evaluation outcome. The pattern across all four commitments is the same: the part that shows information to the developer held up under first contact, and the part that modifies the developer's files either failed or refused to act.

Three findings generalise past this system. First, automating the earlier stages of Parasuraman's model (parsing, clustering, dependency tracking) delivers value independently of whether the later stages work, and a developer who trusts the reports can use them with git directly. Second, the failure mode that cost us the most was a command that reported success while doing nothing. A silent lie is worse than a crash because it damages a specific dimension of automation trust. Dixon and Wickens showed that misses (the system fails to signal a problem) and false alarms (the system signals a problem that does not exist) erode trust through independent mechanisms: misses damage *reliance*—the willingness to trust that silence means safety—while false alarms damage *compliance*—the willingness to act on a positive alert [36]. Every one of our self-reporting failures is a miss in this framework: the system should signal that something went wrong and instead confirms success, so the damage lands on reliance. A developer whose reliance is damaged stops trusting that a clean report means a clean state, and that erosion did not recover within a session (Section 7.5). Any tool that mediates between a developer and code they did not write faces this same constraint: the developer's inability to verify independently is the reason the tool exists, and a tool whose reports cannot be trusted leaves the developer worse off than having no tool, because they have spent their verification budget on a false assurance. Third, evaluating intent recovery against real agentic transcripts is unstable in a way the metric hides. The share of contentless turns varies from 0% to 88% across our four repositories, and a pairwise F1 computed on a history with 88% continuation tokens measures the ground truth's grain rather than the system's accuracy (Section 10.3).

The second finding generalises further. Durability and legibility require different kinds of guarantee, and the second is harder to provide than the first. The append-only store ensures that every prior state is reachable, and it does so structurally—no sequence of operations can destroy a record once written. But the developer still has to know which command to type and whether its output told the truth, and the architecture provides no structural guarantee of that. Every one of our eight self-reporting failures demonstrated this gap: the data was present, and the tool's account of how to reach it was wrong. A tool builder who ships an append-only store and considers persistence solved has solved the easier half; the harder half is that the human trusts what the tool says about what is stored, and trust is not a structural property but a calibration that each interaction either strengthens or damages.

The deeper lesson is about what the tool recovers. We set out to recover intent from history. What we built recovers sub-intent structure: one developer request becomes a median of 4–6 features, reliably, across four repositories with different authors. The reason is that the signals available to the tool—co-change, structural coupling, call references—cut along the code's own joints, not along the developer's purposes: a community detection algorithm finds densely connected subgraphs, and a subgraph boundary falls wherever the coupling is weak, which is where the code's structure has a seam, not where the developer's request ended. The tool's vocabulary is the code's vocabulary (files, coupling, structural communities), not the developer's (requests, purposes, decisions). That vocabulary is useful—it is the cross-file grouping git cannot express, and it answers provenance questions git cannot pose—but calling it intent recovery would overstate the evidence. A designer building on this work should treat the structural signal as a backbone and ask where the intent signal (the developer's own words at save time, the request boundary in the transcript) enters, because the evaluation shows it does not enter through coupling alone.

The recommendation that survives the entire evaluation is temporal: build the read layer and ship it alone. The reason is architectural as well as empirical. A read operation has one precondition—that the file can be parsed—and produces a display the developer checks against their own understanding. A write operation has a second precondition—that the store reproduces the committed bytes exactly—and produces a file the developer checks against a test suite they may not have. The first precondition held on every repository we pointed the tool at; the second held on one of twenty-eight. Attribution, dependency display, and consequence preview ran everywhere, including the 27 repositories that refused every write verb, because they never needed the reconstruction to be complete. A tool builder in this design space faces the same

tradeoff we face and should learn from the order in which we built: the write layer was shipped too early, and the evidence for that is not a hypothesis about trust formation but a participant who said it explicitly and a harness that measured it at 50% per session.

Winograd and Flores argued that a breakdown is productive: by making visible an assumption that was previously invisible, it creates the conditions for redesigning around the absence rather than restoring the assumption to its hidden state [155]. The assumption that broke—that the developer holds a model of what changed and why—is not coming back. The velocity gains are real and the developers who benefit from them are not going to reconstruct each function by hand in order to satisfy a tool that assumed they would. The productive response is to externalize what the assumption provided: make the decomposition a record rather than a memory, and offer it to the developer as something to verify rather than something to reconstruct. That is what the read layer does. The write layer attempts the stronger claim—acting on the record without the developer's verification—and the evaluation shows that claim is premature.

The temporal strategy is also a response to context. A tool entering the developer ecosystem in 2026 inherits a trust deficit it did not create [130]: developers already expect AI-mediated output to be unreliable, and already expect that verifying it will cost more than producing it [94]. The velocity gains from agent-mediated development are real [29, 96] and accompanied by complexity that accumulates downstream [58]—and the natural organisational response, deploying a second model to review what the first wrote, is structurally circular when no external reference breaks the loop [164], while per-reviewer load has already doubled under the throughput pressure these tools create [57]. A tool whose purpose is to make that accumulation legible—to give the developer a vocabulary for what the agent deposited in their repository—addresses the gap between speed and comprehension that the broader ecosystem is now measuring at scale. That tool should arrive as comprehension first, not as another automation whose correctness must be verified. The version control system a developer needs now is one that shows them what they have, not one that acts on it before they know.

References

[1] Abdullah Al Mujahid and Mia Mohammad Imran. 2026. "TODO: Fix the Mess Gemini Created": Towards understanding GenAI-induced self-admitted technical debt. *arXiv preprint arXiv:2601.07786* (2026).

[2] Sven Apel, Jörg Liebig, Benjamin Brandl, Christian Lengauer, and Christian Kästner. 2011. Semistructured merge: rethinking merge in revision control systems. In *Proceedings of the 19th ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 190–200.

[3] Ataraxy Labs. 2026. sem: Semantic version control built on Git. https://github.com/ataraxy-labs/sem. Accessed 2026.

[4] Alberto Bacchelli and Christian Bird. 2013. Expectations, outcomes, and challenges of modern code review. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*. 712–721. doi:10.1109/ICSE.2013.6606617

[5] Lisanne Bainbridge. 1983. Ironies of automation. *Automatica* 19, 6 (1983), 775–779. doi:10.1016/0005-1098(83)90046-8

[6] Gagan Bansal, Tongshuang Wu, Joyce Zhou, Raymond Fok, Besmira Nushi, Ece Kamar, Marco Tulio Ribeiro, and Daniel Weld. 2021. Does the whole exceed its parts? The effect of AI explanations on complementary team performance. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. Article 81. doi:10.1145/3411764.3445717

[7] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How programmers interact with code-generating models. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1 (2023), 85–111. doi:10.1145/3586030

[8] Mike Barnett, Christian Bird, João Brunet, and Shuvendu K. Lahiri. 2015. Helping developers help themselves: Automatic decomposition of code review changesets. In *Proceedings of the 37th International Conference on Software Engineering (ICSE)*. 134–144. doi:10.1109/ICSE.2015.35

[9] Hamsa Bastani, Osbert Bastani, Alp Sungu, Haosen Ge, Özge Kabakcı, and Rei Mariman. 2025. Generative AI can harm learning. *Proceedings of the National Academy of Sciences* 122, 26 (2025), e2422633122. doi:10.1073/pnas.2422633122

[10] Michel Beaudouin-Lafon. 2000. Instrumental Interaction: An Interaction Model for Designing Post-WIMP User Interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI)*. ACM, 446–453.

[11] Joel Becker, Nate Rush, Rosie Saunders, Lauro Langosco, Kaylee Burns, Beth Barnes, and David Rein. 2025. Evidence on the Impact of Generative AI on Experienced Open-Source Developer Productivity. *METR Technical Report* (2025). https://metr.org/blog/2025-07-10-ai-experienced-os-developers/.

[12] Ted J. Biggerstaff, Bharat G. Mitbander, and Dallas Webster. 1993. The concept assignment problem in program understanding. In *Proceedings of the 15th International Conference on Software Engineering (ICSE)*. 482–498. doi:10.1109/ICSE.1993.346017

[13] Alan F. Blackwell and Thomas R. G. Green. 2003. Notational systems: The Cognitive Dimensions of Notations framework. In *HCI Models, Theories, and Frameworks: Toward a Multidisciplinary Science*, John M. Carroll (Ed.). Morgan Kaufmann, 103–134.

[14] Virginia Braun and Victoria Clarke. 2006. Using thematic analysis in psychology. *Qualitative Research in Psychology* 3, 2 (2006), 77–101. doi:10.1191/1478088706qp063oa

[15] Max Brunsfeld and the tree-sitter contributors. 2026. tree-sitter: An incremental parsing system for programming tools. https://tree-sitter.github.io/. Accessed 2026.

[16] Zana Buçinca, Maja Barbara Malaya, and Krzysztof Z. Gajos. 2021. To trust or to think: Cognitive forcing functions can reduce overreliance on AI in AI-assisted decision-making. In *Proceedings of the ACM on Human-Computer Interaction*, Vol. 5. Article 188. doi:10.1145/3449287

[17] Raymond P. L. Buse and Westley R. Weimer. 2010. Automatically documenting program changes. In *Proceedings of the 25th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 33–42. doi:10.1145/1858996.1859005

[18] Ruijia Cao, Zhuoren Lu, and Chun-Wei Huang. 2024. Toward appropriate reliance: complementary explanations reduce both over- and under-reliance on AI. *Proceedings of the ACM on Human-Computer Interaction* 8, CSCW2 (2024). doi:10.1145/3687054

[19] John M. Carroll and Mary Beth Rosson. 1987. Paradox of the active user. *Interfacing Thought: Cognitive Aspects of Human-Computer Interaction* (1987), 80–111.

[20] Matteo Casserini, Alessandro Facchini, and Andrea Ferrario. 2026. Beyond the 'Diff': Addressing Agentic Entropy in Agentic Software Development. In *HCXAI Workshop at CHI*. arXiv:2604.16323.

[21] Carlos Rafael Catalan, Lheane Marie Dizon, Patricia Nicole Monderin, and Emily Kuang. 2026. "I'm Not Reading All of That": Understanding Software Engineers' Level of Cognitive Engagement with Agentic Coding Assistants. In *CHI Workshop on Tools for Thought*. arXiv:2603.14225.

[22] Guilherme Cavalcanti, Paulo Borba, and Paola Accioly. 2017. Evaluating and improving semistructured merge. *Proceedings of the ACM on Programming Languages* 1, OOPSLA (2017), 1–27. doi:10.1145/3133883

[23] Ruijia Cheng, Ruotong Wang, Thomas Zimmermann, and Denae Ford. 2024. "It would work for me too": How online communities shape software developers' trust in AI-powered code generation tools. *ACM Transactions on Interactive Intelligent Systems* (2024). doi:10.1145/3651990

[24] Mark C. Chu-Carroll and Sara Sprenkle. 2000. Coven: brewing better collaboration through software configuration management. In *Proceedings of the 8th ACM SIGSOFT Symposium on Foundations of Software Engineering (FSE)*. 88–97.

[25] Mark C. Chu-Carroll, James Wright, and David Shields. 2002. Supporting aggregation in fine grained software configuration management. In *Proceedings of the 10th ACM SIGSOFT Symposium on Foundations of Software Engineering (FSE)*. 99–108.

[26] Luke Church, Emma Söderberg, and Bhargava Elango. 2014. A case of computational thinking: The subtle effect of hidden dependencies on the user experience of version control. In *Proceedings of the 25th Annual Workshop of the Psychology of Programming Interest Group (PPIG)*.

[27] Herbert H. Clark. 1996. *Using Language*. Cambridge University Press, Cambridge, UK.

[28] Mihai Codoban, Sruti Srinivasa Ragavan, Danny Dig, and Brian Bailey. 2015. Software history under the lens: A study on why and how developers examine it. In *Proceedings of the 2015 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 1–10. doi:10.1109/ICSM.2015.7332446

[29] Zheyuan (Kevin) Cui, Mert Demirer, Sonia Jaffe, Leon Musolff, Sida Peng, and Tobias Salz. 2025. The effects of generative AI on high-skilled work: Evidence from three field experiments with software developers. *SSRN Electronic Journal* (2025). doi:10.2139/ssrn.4945566

[30] Shamse Tasnim Cynthia, Ratnadira Widyasari, Banani Roy, Ting Zhang, and David Lo. 2026. "Go Home Copilot, You're Drunk": Understanding Developer Responses to Agent-Generated Code Review Comments. *arXiv preprint arXiv:2607.21997* (2026).

[31] Yuanzhe Deng, Shutong Zhang, Kathy Cheng, Alison Olechowski, and Shurui Zhou. 2026. Untangling the Timeline: Challenges and Opportunities in Supporting Version Control in Modern Computer-Aided Design. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*. ACM.

[32] Shipi Dhanorkar, Samir Passi, and Mihaela Vorvoreanu. 2026. Human oversight of agentic systems in practice: Examining the oversight work, challenges, and heuristics of developers using software agents. *arXiv preprint arXiv:2606.05391* (2026).

[33] Martín Dias, Alberto Bacchelli, Georgios Gousios, Damien Cassou, and Stéphane Ducasse. 2015. Untangling fine-grained code changes. In *Proceedings of the 22nd IEEE International Conference on Software Analysis, Evolution, and Reengineering (SANER)*. 341–350. doi:10.1109/SANER.2015.7081844

[34] Danny Dig, Kashif Manzoor, Ralph Johnson, and Tien N. Nguyen. 2007. Refactoring-aware configuration management for object-oriented programs. In *Proceedings of the 29th International Conference on Software Engineering (ICSE)*. 427–436.

[35] Bogdan Dit, Meghan Revelle, Malcom Gethers, and Denys Poshyvanyk. 2013. Feature location in source code: A taxonomy and survey. *Journal of Software: Evolution and Process* 25, 1 (2013), 53–95. doi:10.1002/smr.567

[36] Stephen R. Dixon and Christopher D. Wickens. 2007. On the Independence of Compliance and Reliance: Are Automation False Alarms Worse Than Misses? *Human Factors* 49, 4 (2007), 717–729. doi:10.1518/001872007X215656

[37] Ian Drosos and Advait Sarkar. 2025. Vibe coding: Programming through conversation with artificial intelligence. *arXiv preprint arXiv:2506.23253* (2025).

[38] Ruoxi Duan, Jina Suh, Alex Gu, Zijian Liu, Hussein Mozannar, and Gagan Bansal. 2026. I don't trust it because my friend said so: trust contagion in human–AI teams. In *Proceedings of the 2026 ACM Conference on Computer-Supported Cooperative Work and Social Computing*. doi:10.1145/3706598.3713903

[39] Mica R. Endsley and Esin O. Kiris. 1995. The out-of-the-loop performance problem and level of control in automation. *Human Factors* 37, 2 (1995), 381–394. doi:10.1518/001872095779064555

[40] Jean-Rémy Falleri, Floréal Morandat, Xavier Blanc, Matias Martinez, and Martin Monperrus. 2014. Fine-grained and accurate source code differencing. In *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering (ASE)*. 313–324. doi:10.1145/2642937.2642982

[41] Faros AI. 2026. The 2026 Engineering Acceleration Report. https://www.faros.ai/report. Telemetry from 22,000 developers across 4,000 teams.

[42] Kasra Ferdowsi, Ruanqianqian Huang, Madeline B. James, Nadia Polikarpova, and Sorin Lerner. 2024. Validating AI-generated code with live programming. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. doi:10.1145/3613904.3642495

[43] Samuel Ferino, Rashina Hoda, John Grundy, and Christoph Treude. 2026. Towards an appropriate level of reliance on AI: A preliminary reliance-control framework for AI in software engineering. *arXiv preprint arXiv:2604.10530* (2026).

[44] Beat Fluri, Michael Würsch, Martin Pinzger, and Harald C. Gall. 2007. Change distilling: Tree differencing for fine-grained source code change extraction. *IEEE Transactions on Software Engineering* 33, 11 (2007), 725–743. doi:10.1109/TSE.2007.70731

[45] George W. Furnas, Thomas K. Landauer, Louis M. Gomez, and Susan T. Dumais. 1987. The vocabulary problem in human-system communication. *Commun. ACM* 30, 11 (1987), 964–971. doi:10.1145/32206.32212

[46] GitButler. 2026. GitButler documentation: Parallel branches. https://docs.gitbutler.com/features/virtual-branches/virtual-branches. Accessed 2026.

[47] GitClear. 2025. AI copilot code quality: 2025 look back at 12 months of data. https://www.gitclear.com/ai_assistant_code_quality_2025_research. 211 million changed lines of code authored January 2020 to December 2024.

[48] GitHub. 2026. Stacked pull requests are now in public preview. https://github.blog/changelog/2026-07-30-stacked-pull-requests-are-now-in-public-preview/. Accessed 2026.

[49] Michael W. Godfrey and Lijie Zou. 2005. Using origin analysis to detect merging and splitting of source code entities. *IEEE Transactions on Software Engineering* 31, 2 (2005), 166–181. doi:10.1109/TSE.2005.28

[50] Graphite. 2026. Stacked diffs and how to use them. https://graphite.com/guides/stacked-diffs. Accessed 2026.

[51] Thomas R. G. Green and Marian Petre. 1996. Usability analysis of visual programming environments: A 'cognitive dimensions' framework. *Journal of Visual Languages and Computing* 7, 2 (1996), 131–174. doi:10.1006/jvlc.1996.0009

[52] Tovi Grossman, Justin Matejka, and George Fitzmaurice. 2010. Chronicle: Capture, exploration, and playback of document workflow histories. In *Proceedings of the 23rd Annual ACM Symposium on User Interface Software and Technology (UIST)*. 143–152. doi:10.1145/1866029.1866054

[53] Mário Luís Guimarães and António Rito Silva. 2012. Improving early detection of software merge conflicts. In *Proceedings of the 34th International Conference on Software Engineering (ICSE)*. 342–352. doi:10.1109/ICSE.2012.6227180

[54] Kunal Handa, Alex Tamkin, Miles McCain, Saffron Huang, Esin Durmus, Sarah Heck, Jared Mueller, Jerry Hong, Stuart Ritchie, Tim Belonax, Kevin K. Troy, Dario Amodei, Jared Kaplan, Jack Clark, and Deep Ganguli. 2025. Which Economic Tasks are Performed with AI? Evidence from Millions of Claude Conversations. *arXiv preprint arXiv:2503.04761* (2025).

[55] Björn Hartmann, Loren Yu, Abel Allison, Yeonsoo Yang, and Scott R. Klemmer. 2008. Design as exploration: Creating interface alternatives through parallel authoring and runtime tuning. In *Proceedings of the 21st Annual ACM Symposium on User Interface Software and Technology (UIST)*. 91–100. doi:10.1145/1449715.1449732

[56] Hideaki Hata, Osamu Mizuno, and Tohru Kikuno. 2011. Historage: fine-grained version control system for Java. In *Proceedings of the 12th International Workshop on Principles of Software Evolution and the 7th Annual ERCIM Workshop on Software Evolution (IWPSE-EVOL)*. 96–100.

[57] Hao He, Shyam Agarwal, Yegor Denisov-Blanch, Pavel Azaletskiy, Sanmi Koyejo, and Bogdan Vasilescu. 2026. AI writes faster than humans can review: A longitudinal study of an enterprise 2x mandate. *arXiv preprint arXiv:2607.01904* (2026).

[58] Junwen He, Luís Cruz, and Meiyappan Nagappan. 2026. The velocity mirage: AI-assisted development speed and its downstream costs. In *Proc. MSR*. IEEE.

[59] Andrew Head, Fred Hohman, Titus Barik, Steven M. Drucker, and Robert DeLine. 2019. Managing messes in computational notebooks. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. 1–12. doi:10.1145/3290605.3300776

[60] Lars Heander, Agnia Sergeyuk, Ilya Zakharov, et al. 2026. Trust-calibrated code review: A participatory design study of review workflows for LLM-generated multi-file changes. *arXiv preprint* (2026).

[61] Kim Herzig, Sascha Just, and Andreas Zeller. 2016. The impact of tangled code changes on defect prediction models. *Empirical Software Engineering* 21, 2 (2016), 303–336. doi:10.1007/s10664-015-9376-6

[62] Kim Herzig and Andreas Zeller. 2013. The impact of tangled code changes. In *Proceedings of the 10th Working Conference on Mining Software Repositories (MSR)*. 121–130. doi:10.1109/MSR.2013.6624018

[63] Yoshiki Higo, Shinpei Hayashi, and Shinji Kusumoto. 2020. On tracking Java methods with Git mechanisms. *Journal of Systems and Software* 165 (2020), 110571.

[64] Susan Horwitz, Jan Prins, and Thomas Reps. 1989. Integrating noninterfering versions of programs. *ACM Transactions on Programming Languages and Systems* 11, 3 (1989), 345–387.

[65] Edwin Hutchins. 1995. *Cognition in the Wild*. MIT Press, Cambridge, MA.

[66] Daniel Jackson. 2021. *The Essence of Software: Why Concepts Matter for Great Design*. Princeton University Press.

[67] Judah Jacobson. 2009. A formalization of Darcs patch theory using inverse semigroups. *Computational and Applied Mathematics Reports, University of California, Los Angeles* (2009). CAM report 09-83.

[68] Jiun-Yin Jian, Ann M. Bisantz, and Colin G. Drury. 2000. Foundations for an empirically determined scale of trust in automated systems. *International Journal of Cognitive Ergonomics* 4, 1 (2000), 53–71. doi:10.1207/S15327566IJCE0401_04

[69] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can language models resolve real-world GitHub issues?. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.

[70] Harmanpreet Kaur, Hanna Nori, Samuel Jenkins, Rich Caruana, Hanna Wallach, and Jennifer Wortman Vaughan. 2020. Interpreting Interpretability: Understanding Data Scientists' Use of Interpretability Tools for Machine Learning. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*. 1–14. doi:10.1145/3313831.3376219

[71] Majeed Kazemitabaar, Olivia Huang, Sangho Suh, Austin Z. Henley, and Tovi Grossman. 2025. Exploring the design space of cognitive engagement techniques with AI-generated code for enhanced learning. In *Proceedings of the 30th International Conference on Intelligent User Interfaces*. doi:10.1145/3708359.3712104

[72] Mik Kersten and Gail C. Murphy. 2006. Using task context to improve programmer productivity. In *Proceedings of the 14th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*. 1–11. doi:10.1145/1181775.1181777

[73] Mary Beth Kery, Amber Horvath, and Brad A. Myers. 2017. Variolite: Supporting exploratory programming by data scientists. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*. 1265–1276. doi:10.1145/3025453.3025626

[74] Mary Beth Kery, Marissa Radensky, Mahima Arya, Bonnie E. John, and Brad A. Myers. 2018. The story in the notebook: Exploratory data science using a literate programming tool. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*. 1–11. doi:10.1145/3173574.3173748

[75] Sunghun Kim, Kai Pan, and E. James Whitehead. 2005. When functions change their names: Automatic detection of origin relationships. In *Proceedings of the 12th Working Conference on Reverse Engineering (WCRE)*. 143–152. doi:10.1109/WCRE.2005.33

[76] Martin Kleppmann, Victor B. F. Gomes, Dominic P. Mulligan, and Alastair R. Beresford. 2019. Interleaving anomalies in collaborative text editors. In *Proceedings of the 6th Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC)*. 1–7. doi:10.1145/3301419.3323972

[77] Andrew J. Ko, Brad A. Myers, Michael J. Coblenz, and Htet Htet Aung. 2006. An exploratory study of how developers seek, relate, and collect relevant information during software maintenance tasks. *IEEE Transactions on Software Engineering* 32, 12 (2006), 971–987. doi:10.1109/TSE.2006.116

[78] Raula Gaikovina Kula and Christoph Treude. 2025. The shift from writing to pruning software: A Bonsai-inspired IDE for reshaping AI generated code. In *SE 2030 Software Engineering Roadmap Workshop*. arXiv:2503.02833.

[79] Thomas D. LaToza and Brad A. Myers. 2010. Hard-to-answer questions about code. In *Evaluation and Usability of Programming Languages and Tools (PLATEAU)*. 1–6. doi:10.1145/1937117.1937125

[80] John D. Lee and Katrina A. See. 2004. Trust in automation: Designing for appropriate reliance. *Human Factors* 46, 1 (2004), 50–80. doi:10.1518/hfes.46.1.50.30392

[81] Stanley Letovsky. 1987. Cognitive processes in program comprehension. In *Journal of Systems and Software*, Vol. 7. 325–339. doi:10.1016/0164-1212(87)90032-X

[82] Jenny T. Liang, Chenyang Yang, and Brad A. Myers. 2024. A large-scale survey on the usability of AI programming assistants: Successes and challenges. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. 1–13. doi:10.1145/3597503.3608128

[83] Ernst Lippe and Norbert van Oosterom. 1992. Operation-based merging. In *Proceedings of the Fifth ACM SIGSOFT Symposium on Software Development Environments (SDE)*. 78–87.

[84] Geoffrey Litt, Sarah Lim, Martin Kleppmann, and Peter van Hardenberg. 2022. Peritext: A CRDT for collaborative rich text editing. *Proceedings of the ACM on Human-Computer Interaction* 6, CSCW2 (2022), 1–29. doi:10.1145/3555644

[85] Yue Liu, Ratnadira Widyasari, Yanjie Zhao, Ivana Clairine Irsan, Junkai Chen, and David Lo. 2026. Debt Behind the AI Boom: A Large-Scale Empirical Study of AI-Generated Code in the Wild. *arXiv preprint arXiv:2603.28592* (2026).

[86] Yunbo Lyu, David Williams, Jieke Shi, Zhensu Sun, Chao Peng, Zhou Yang, Federica Sarro, and David Lo. 2026. How Do Practitioners Build SE Agents? Insights from a Mixed-Methods Study. *arXiv preprint arXiv:2607.10856* (2026).

[87] Poornima Madhavan and Douglas A. Wiegmann. 2007. Similarities and differences between human–human and human–automation trust: An integrative review. *Theoretical Issues in Ergonomics Science* 8, 4 (2007), 277–301. doi:10.1080/14639220500337708

[88] George Mandler. 1980. Recognizing: The judgment of previous occurrence. *Psychological Review* 87, 3 (1980), 252–271. doi:10.1037/0033-295X.87.3.252

[89] Pierre-Étienne Meunier and Florent Becker. 2024. The Pijul manual. https://pijul.org/manual/. Accessed 2026.

[90] Hiroaki Mikami, Daisuke Sakamoto, and Takeo Igarashi. 2017. Micro-versioning tool to support experimentation in exploratory programming. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*. 6208–6219. doi:10.1145/3025453.3025597

[91] Samuel Mimram and Cinzia Di Giusto. 2013. A categorical theory of patches. *Electronic Notes in Theoretical Computer Science* 298 (2013), 283–307. doi:10.1016/j.entcs.2013.09.018

[92] Scott Mishler and Jamy Chen. 2023. The effect of automation errors on trust: A meta-analysis. In *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, Vol. 67. 279–284. doi:10.1177/21695067231192235

[93] Burt L. Monroe, Michael P. Colaresi, and Kevin M. Quinn. 2008. Fightin' words: Lexical feature selection and evaluation for identifying the content of political text. *Political Analysis* 16, 4 (2008), 372–403. doi:10.1093/pan/mpn018

[94] Hussein Mozannar, Valerie Chen, Mohammed Alsobay, Subhro Das, Sebastian Zhao, Dennis Wei, Manish Nagireddy, Prasanna Sattigeri, Ameet Talwalkar, and David Sontag. 2024. The RealHumanEval: Evaluating large language models' abilities to support programmers. *arXiv preprint arXiv:2404.02806* (2024).

[95] Bonnie M. Muir. 1994. Trust in automation: Part I. Theoretical issues in the study of trust and human intervention in automated systems. *Ergonomics* 37, 11 (1994), 1905–1922. doi:10.1080/00140139408964957

[96] Emerson Murphy-Hill, Ciera Jaspan, Caitlin Sadowski, David Shepherd, Brendan Dolan-Gavitt, Kivanç Muşlu, and Kathryn T. Stolee. 2026. How developers use AI coding assistants: A field study of CLI agents in professional practice. *IEEE Transactions on Software Engineering* (2026). Preprint: arXiv:2607.01418.

[97] Rahul Nanda, Chandra Maddila, Smriti Jha, Euna Mehnaz Khan, Matteo Paltenghi, and Satish Chandra. 2026. Wink: Recovering from Misbehaviors in Coding Agents. *arXiv preprint arXiv:2602.17037* (2026).

[98] Peter Naur. 1985. Programming as Theory Building. *Microprocessing and Microprogramming* 15, 5 (1985), 253–261. Reprinted in *Computing: A Human Activity*, ACM Press, 1992.

[99] Nicholas Nelson, Caius Brindescu, Shane McKee, Anita Sarma, and Danny Dig. 2019. The life-cycle of merge conflicts: Processes, barriers, and strategies. *Empirical Software Engineering* 24, 5 (2019), 2863–2906. doi:10.1007/s10664-018-9674-x

[100] Tuan Anh Nguyen and Elena L. Glassman. 2024. How beginning programmers and code LLMs (mis)read each other. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. doi:10.1145/3613904.3642706

[101] Donald A. Norman. 1988. *The Design of Everyday Things*. Basic Books. Originally published as *The Psychology of Everyday Things*, 1988; revised edition 2013.

[102] Raja Parasuraman, Thomas B. Sheridan, and Christopher D. Wickens. 2000. A model for types and levels of human interaction with automation. *IEEE Transactions on Systems, Man, and Cybernetics—Part A: Systems and Humans* 30, 3 (2000), 286–297. doi:10.1109/3468.844354

[103] Chris Parnin and Spencer Rugaber. 2011. Resumption strategies for interrupted programming tasks. In *Proceedings of the 2011 IEEE 19th International Conference on Program Comprehension (ICPC)*. IEEE, 81–90. doi:10.1109/ICPC.2011.5970167

[104] Santiago Perez De Rosso and Daniel Jackson. 2013. What's wrong with git? A conceptual design analysis. In *Proceedings of the 2013 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software (Onward!)*. ACM, 37–52. doi:10.1145/2509578.2509584

[105] Santiago Perez De Rosso and Daniel Jackson. 2016. Purposes, concepts, misfits, and a redesign of git. In *Proceedings of the 2016 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*. ACM, 292–310. doi:10.1145/2983990.2984018

[106] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do users write more insecure code with AI assistants?. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2785–2799. doi:10.1145/3576915.3623157

[107] Peter Pirolli and Stuart Card. 1999. Information foraging. *Psychological Review* 106, 4 (1999), 643–675. doi:10.1037/0033-295X.106.4.643

[108] Profir-Petru Pârțachi, Santanu Kumar Dash, Christoph Treude, and Earl T. Barr. 2020. Flexeme: Untangling commits using lexical flows. In *Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 63–74. doi:10.1145/3368089.3409693

[109] Musfiqur Rahman and Emad Shihab. 2026. Will It Survive? Deciphering the Fate of AI-Generated Code in Open Source. In *Proc. EASE*. arXiv:2601.16809.

[110] John Rieman. 1996. A field study of exploratory learning strategies. In *ACM Transactions on Computer-Human Interaction*, Vol. 3. 189–218. doi:10.1145/234526.234527

[111] Peter C. Rigby and Christian Bird. 2013. Convergent contemporary software peer review practices. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. 202–212. doi:10.1145/2491411.2491444

[112] Evan F. Risko and Sam J. Gilbert. 2016. Cognitive Offloading. *Trends in Cognitive Sciences* 20, 9 (2016), 676–688. doi:10.1016/j.tics.2016.07.002

[113] Eleanor Rosch. 1978. Principles of categorization. In *Cognition and Categorization*, Eleanor Rosch and Barbara B. Lloyd (Eds.). Lawrence Erlbaum, Hillsdale, NJ, 27–48.

[114] David Roundy. 2005. Darcs: Distributed version management in Haskell. In *Proceedings of the 2005 ACM SIGPLAN Workshop on Haskell*. 1–4. doi:10.1145/1088348.1088349

[115] Leonid Rozenblit and Frank Keil. 2002. The misunderstood limits of folk science: An illusion of explanatory depth. *Cognitive Science* 26, 5 (2002), 521–562. doi:10.1207/s15516709cog2605_1

[116] S. Sabouri, P. Eibl, X. Zhou, M. Ziyadi, N. Medvidovic, L. Lindemann, and S. Chattopadhyay. 2025. Trust dynamics in AI-assisted development: definitions, factors, and implications. In *Proceedings of the 47th International Conference on Software Engineering*. doi:10.1109/ICSE55347.2025.11029928

[117] Advait Sarkar, Andrew D. Gordon, Carina Negreanu, Christian Poelitz, Sruti Srinivasa Ragavan, and Ben Zorn. 2022. What is it like to program with artificial intelligence?. In *Proceedings of the 33rd Annual Workshop of the Psychology of Programming Interest Group (PPIG)*.

[118] Nadine B. Sarter and David D. Woods. 1995. How in the world did we ever get into that mode? Mode error and awareness in supervisory control. *Human Factors* 37, 1 (1995), 5–19. doi:10.1518/001872095779049516

[119] Shota Sawada, Tatsuya Shirai, Yutaro Kashiwa, Ken'ichi Yamaguchi, Hiroshi Iwata, and Hajimu Iida. 2026. To What Extent Does Agent-generated Code Require Maintenance? An Empirical Study. In *Proc. 30th International Conference on Evaluation and Assessment in Software Engineering (EASE)*.

[120] Max Schemmer, Niklas Kühl, Carina Benz, and Gerhard Satzger. 2022. Appropriate reliance on AI advice: conceptualization and the effect of explanations. In *Proceedings of the 2022 AAAI/ACM Conference on AI, Ethics, and Society*. doi:10.1145/3514094.3534089

[121] Richard D. Schlichting and Fred B. Schneider. 1983. Fail-Stop Processors: An Approach to Designing Fault-Tolerant Computing Systems. *ACM Transactions on Computer Systems* 1, 3 (1983), 222–238. doi:10.1145/357369.357371

[122] Michael Sedlmair, Miriah Meyer, and Tamara Munzner. 2012. Design Study Methodology: Reflections from the Trenches and the Stacks. In *IEEE Transactions on Visualization and Computer Graphics*, Vol. 18. 2431–2440. doi:10.1109/TVCG.2012.213

[123] Agnia Sergeyuk, Eric Huang, Dariia Karaeva, Anastasiia Serova, Yaroslav Golubev, and Iftekhar Ahmed. 2026. Evolving with AI: A longitudinal analysis of developer logs. *arXiv preprint arXiv:2601.10258* (2026).

[124] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. 2011. Conflict-free replicated data types. In *Proceedings of the 13th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS)*. 386–400. doi:10.1007/978-3-642-24550-3_29

[125] Alex Shen and Alex Tamkin. 2026. How AI Impacts Skill Formation: Evidence from a Coding Experiment. *Anthropic Research* (2026). https://www.anthropic.com/research/AI-assistance-coding-skills

[126] Bo Shen, Wei Zhang, Christian Kästner, Haiyan Zhao, Zhao Wei, Guangtai Liang, and Zhi Jin. 2021. SmartCommit: A graph-based interactive assistant for activity-oriented commits. In *Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 379–390. doi:10.1145/3468264.3468551

[127] Frank M. Shipman and Catherine C. Marshall. 1999. Formality Considered Harmful: Experiences, Emerging Themes, and Directions on the Use of Formal Representations in Interactive Systems. *Computer Supported Cooperative Work* 8, 4 (1999), 333–352.

[128] Jonathan Sillito, Gail C. Murphy, and Kris De Volder. 2008. Asking and answering questions during a programming change task. *IEEE Transactions on Software Engineering* 34, 4 (2008), 434–451. doi:10.1109/TSE.2008.26

[129] Herbert A. Simon. 1962. The Architecture of Complexity. *Proceedings of the American Philosophical Society* 106, 6 (1962), 467–482.

[130] Stack Overflow. 2025. 2025 Developer Survey. Accessed 2026-08-20. https://survey.stackoverflow.co/2025/

[131] Margaret-Anne Storey. 2026. From Technical Debt to Cognitive and Intent Debt: Rethinking Software Health in the Age of AI. *arXiv preprint arXiv:2603.22106* (2026).

[132] Lucy A. Suchman. 1987. *Plans and Situated Actions: The Problem of Human-Machine Communication*. Cambridge University Press.

[133] Florian Tambon, Arghavan Moradi Dakhel, Amin Nikanjam, Foutse Khomh, Michel C. Desmarais, and Giuliano Antoniol. 2024. Bugs in large language models generated code: An empirical study. *Empirical Software Engineering* 29, 3 (2024), 69. doi:10.1007/s10664-024-10468-0

[134] Ningzhi Tang, Chaoran Chen, Gelei Xu, Yiyu Shi, Yu Huang, Collin McMillan, Tao Dong, and Toby Jia-Jun Li. 2026. How Coding Agents Fail Their Users: A Large-Scale Analysis of Developer–Agent Misalignment in 20,574 Real-World Sessions. *arXiv preprint arXiv:2605.29442* (2026).

[135] Ningzhi Tang, Meng Chen, Zheng Ning, Aakash Bansal, Yu Huang, Collin McMillan, and Toby Jia-Jun Li. 2024. Developer behaviors in validating and repairing LLM-generated code using IDE and eye tracking. In *Proceedings of the 2024 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. 40–46.

[136] Yida Tao and Sunghun Kim. 2015. Partitioning composite code changes to facilitate code review. In *Proceedings of the 12th Working Conference on Mining Software Repositories (MSR)*. 180–190. doi:10.1109/MSR.2015.24

[137] Michael Terry, Elizabeth D. Mynatt, Kumiyo Nakakoji, and Yasuhiro Yamamoto. 2004. Variation in element and action: Supporting simultaneous development of alternative solutions. In *Proceedings of the 2004 CHI Conference on Human Factors in Computing Systems*. 711–718. doi:10.1145/985692.985782

[138] The Mercurial contributors. 2026. Mercurial wiki: Changeset evolution. https://wiki.mercurial-scm.org/ChangesetEvolution. Accessed August 2026.

[139] Harold Thimbleby. 1988. Delaying commitment. *IEEE Software* 5, 3 (1988), 78–86. doi:10.1109/52.2027

[140] Yingchen Tian, Yuxia Zhang, Klaas-Jan Stol, Lin Jiang, and Hui Liu. 2022. What makes a good commit message?. In *Proceedings of the 44th International Conference on Software Engineering (ICSE)*. 2389–2401. doi:10.1145/3510003.3510205

[141] Linus Torvalds, Junio C. Hamano, and the Git contributors. 2026. Git: Distributed version control system. https://git-scm.com/. Accessed 2026.

[142] V. A. Traag, P. Van Dooren, and Y. Nesterov. 2011. Narrow scope for resolution-limit-free community detection. *Physical Review E* 84 (2011), 016114. doi:10.1103/PhysRevE.84.016114

[143] V. A. Traag, L. Waltman, and N. J. van Eck. 2019. From Louvain to Leiden: Guaranteeing well-connected communities. *Scientific Reports* 9 (2019), 5233. doi:10.1038/s41598-019-41695-z

[144] Christoph Treude. 2026. Accountable Agents in Software Engineering: An Analysis of Terms of Service and a Research Roadmap. In *AIware '26: Proc. 3rd ACM International Conference on AI-powered Software*. arXiv:2605.04532.

[145] Nikolaos Tsantalis, Matin Mansouri, Laleh M. Eshkevari, Davood Mazinanian, and Danny Dig. 2018. Accurate and efficient refactoring detection in commit history. In *Proceedings of the 40th International Conference on Software Engineering (ICSE)*. 483–494. doi:10.1145/3180155.3180206

[146] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts*. 1–7. doi:10.1145/3491101.3519665

[147] Helena Vasconcelos, Gagan Bansal, Adam Fourney, Q. Vera Liao, and Jennifer Wortman Vaughan. 2025. Generation probabilities are not enough: Uncertainty highlighting in AI code completions. *ACM Transactions on Computer-Human Interaction* (2025). doi:10.1145/3702320

[148] Alejandro Velasco, Nathan Wintersgill, Trevor Stalnaker, Oscar Chaparro, and Denys Poshyvanyk. 2026. On Automated and Explainable Provenance of AI-Generated Code. *arXiv preprint arXiv:2608.02329* (2026).

[149] Annie Vella and Kelly Blincoe. 2026. The Impact of AI Coding Assistants on Software Engineering: A Longitudinal Study. *arXiv preprint arXiv:2605.23135* (2026).

[150] Martin von Zweigbergk and the Jujutsu contributors. 2026. Jujutsu documentation: Comparison with Git. http://docs.jj-vcs.dev/latest/git-comparison/. Accessed August 2026.

[151] Martin von Zweigbergk and the Jujutsu contributors. 2026. Jujutsu documentation: Glossary. http://docs.jj-vcs.dev/latest/glossary/. Accessed August 2026.

[152] Min Wang, Zeqi Lin, Yanzhen Zou, and Bing Xie. 2019. CoRA: Decomposing and describing tangled code changes for reviewer. In *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 1050–1061. doi:10.1109/ASE.2019.00101

[153] Ruotong Wang, Ruijia Peng, Chin-Wei Cheng, Yun-Chien Chen, Chia-Yi Kuo, and Hao-Chuan Hsieh. 2024. Trust calibration in AI-assisted code generation. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. doi:10.1145/3613904.3642503

[154] Zora Z. Wang, John Yang, Kilian Lieret, Alexa Tartaglini, Valerie Chen, Yuxiang Wei, Zijian Wang, Lingming Zhang, Karthik Narasimhan, Ludwig Schmidt, Graham Neubig, Daniel Fried, and Diyi Yang. 2026. Humans are Missing from AI Coding Agent Research. *arXiv preprint arXiv:2608.12355* (2026).

[155] Terry Winograd and Fernando Flores. 1986. *Understanding Computers and Cognition: A New Foundation for Design*. Ablex Publishing, Norwood, NJ.

[156] Xin Xia, Lingfeng Bao, David Lo, Zhenchang Xing, Ahmed E. Hassan, and Shanping Li. 2018. Measuring program comprehension: A large-scale field study with professionals. *IEEE Transactions on Software Engineering* 44, 10 (2018), 951–976. doi:10.1109/TSE.2017.2734091

[157] Xiangzhe Xu, Shiwei Feng, Zian Su, Chengpeng Wang, and Xiangyu Zhang. 2025. Position: Intelligent coding systems should write programs with justifications. *arXiv preprint arXiv:2508.06017* (2025).

[158] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[159] Annie T. T. Ying, Gail C. Murphy, Raymond Ng, and Mark C. Chu-Carroll. 2004. Predicting source code changes by mining change history. *IEEE Transactions on Software Engineering* 30, 9 (2004), 574–586. doi:10.1109/TSE.2004.52

[160] YoungSeok Yoon and Brad A. Myers. 2014. A longitudinal study of programmers' backtracking. In *Proceedings of the 2014 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. 101–108. doi:10.1109/VLHCC.2014.6883030

[161] YoungSeok Yoon and Brad A. Myers. 2015. Supporting selective undo in a code editor. In *Proceedings of the 37th International Conference on Software Engineering (ICSE)*. 223–233. doi:10.1109/ICSE.2015.43

[162] YoungSeok Yoon, Brad A. Myers, and Sebon Koo. 2013. Visualization of fine-grained code change history. In *Proceedings of the 2013 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. 119–126. doi:10.1109/VLHCC.2013.6645254

[163] Andreas Zeller and Gregor Snelting. 1997. Unified versioning through feature logic. *ACM Transactions on Software Engineering and Methodology* 6, 4 (1997), 398–441.

[164] Christo Zietsman. 2026. The specification as quality gate: Three hypotheses on AI-assisted code review. *arXiv preprint arXiv:2603.25773* (2026).

[165] Thomas Zimmermann, Peter Weißgerber, Stephan Diehl, and Andreas Zeller. 2004. Mining version histories to guide software changes. In *Proceedings of the 26th International Conference on Software Engineering (ICSE)*. 563–572. doi:10.1109/ICSE.2004.1317478